A Practical Guide To Securing Open Source In The Quantum Era

Stopping Dangerous AI Agent Actions Before Execution

₹120
ISSN-2456-4885

An EFY GROUP Publication

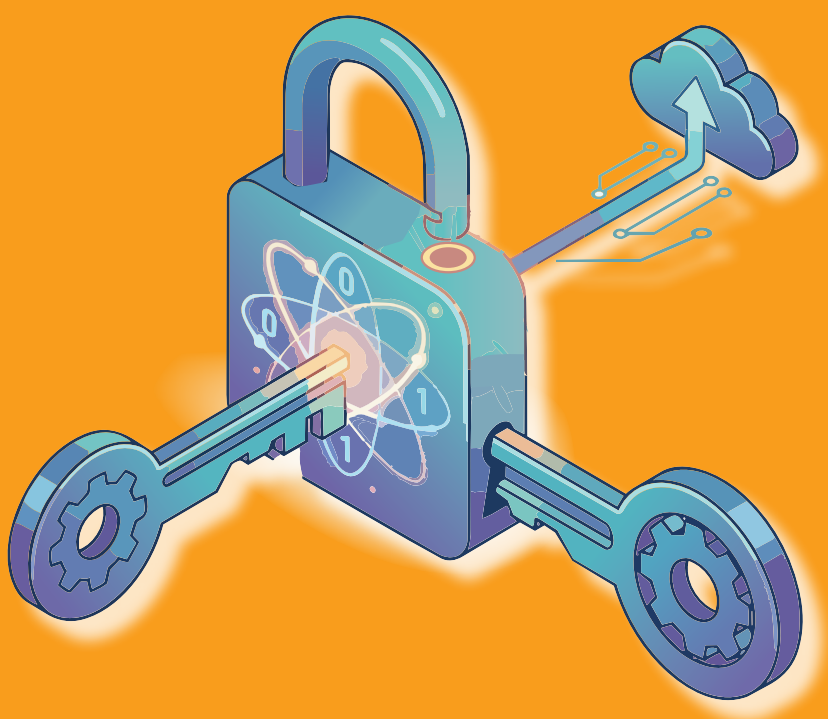
Volume: 14 | Issue: 12 | Pages: 100 | October 2026

OpenSource ForYou

THE COMPLETE MAGAZINE ON OPEN SOURCE

Cracking The Quantum Code

How Open Source Is Unlocking The Next Era Of Computing

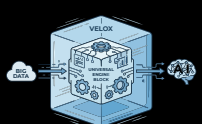


Open Source Quantum Compilers: A Technical Overview Of The Current Landscape

Quantum Computing: Architecture, Access, And Open Source Tools

PennyLane: Building A Bridge Between Machine Learning And Qubits

Quantum Cryptography: Ensuring Secure Digital Communication



Inside Velox, The 'Universal Engine Block' For Big Data And AI

Agentic DevOps In Action

Wanna Be Your Own Boss?

DO OPEN SOURCE.



Demand for Open Source is sky rocketing. Be it for managing IT infrastructure or development of software--Open Source solutions are what customers are seeking.

All you need to do is develop expertise in an Open Source stack, and then build a team around it!

And, Open Source For You can be your friend and a guide through this journey.

TO READ OUR PRINT EDITION Visit: https://subscribe.efyindia.com

TO READ OUR EZINE EDITION Visit: https://ezine.lfymag.com



WORLD'S LEADING PUBLICATION ON OPEN SOURCE

Looking for marketing solutions to engage with cutting edge techies?
Contact us at growmybiz@efy.in OR call us at +91-9811155335.

Hetzner Cloud

No surprises.
No limits.

SAVE NOW!
$10
with the code:
HetznerXOSFY26



LOCATION IN
SINGAPORE

HETZNER

- ✓ No hidden costs
- ✓ Easy to start & full API access
- ✓ High data protection standards

htznr.li/OSFY/cloud-sgp

Get started now and
deploy your server in seconds:
Hetzner Cloud CPX22
2 vCPU
4 GB RAM
80 GB SSD

Test Hetzner Cloud now!

Promo code only for new customers - activate before 31 January 2027 - valid for 3 months after activation.
All products are subject to the terms and conditions of Hetzner Online GmbH. Subject to errors.

CONTENTS

OCTOBER 2026 | ISSN-2456-4885

FOR U & ME

22 Infrastructure Processing Units: The Unsung Heroes of the Data Centre

33 The Internet and AI: Multipliers of National Progress

FOCUS

40 Quantum Computing: Ringing the Bell States

45 Open Source Quantum Compilers: A Technical Overview of the Current Landscape

49 Quantum Cryptography: Ensuring Secure Digital Communication

51 The Road Ahead: Quantum Computing in the Next Decade

56 NVIDIA CUDA-Q: Accelerating the Development

DEVELOPERS

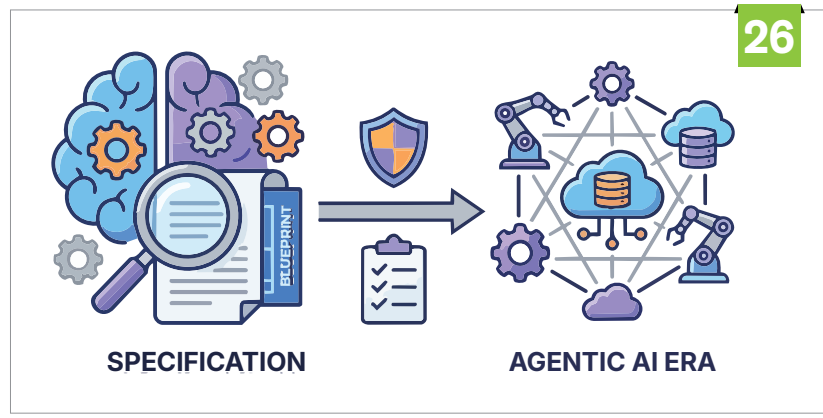
73 The Benefits and Uses of Open Source LLMs

75 Moving Average Crossover Strategy: Algorithmic Trading with Python

ADMIN

91 Kubernetes Security: Threats a nd Controls

95 Agentic DevOps in Action



The Need for Spec-Driven Development in the Agentic AI Era



Quantum Computing: Architecture, Access, and Open Source Tools

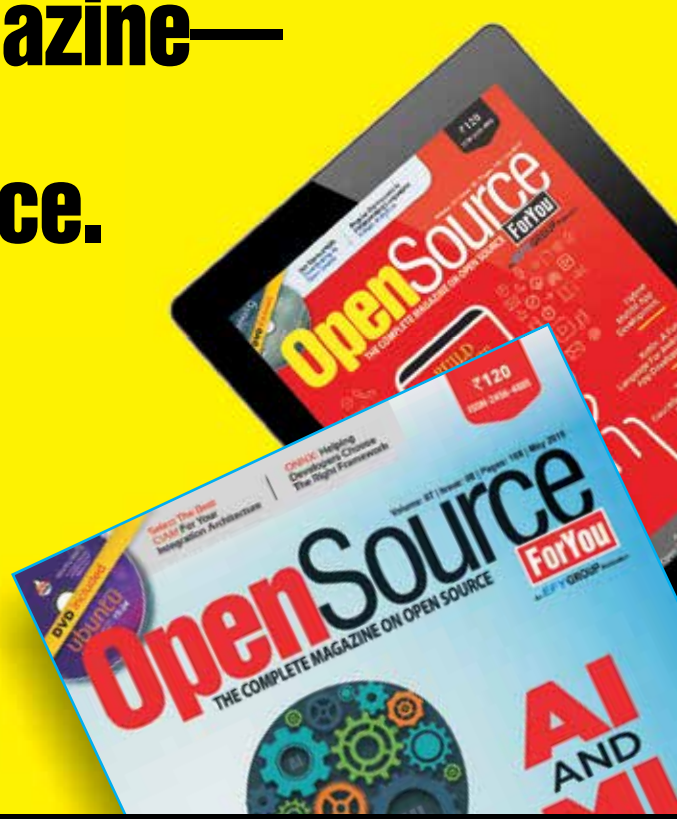
REGULAR FEATURES

07 FossBytes

EFYGROUP
YOURS SINCE 1969

Wanna Support Open Source For You?

Subscribe to the Magazine— so that we can keep promoting Open Source.



AMAZING OFFER

BUY 1, GET 1 FREE

WORLD'S LEADING PUBLICATION ON OPEN SOURCE

TEAR OUT — **ORDER FORM** — TEAR OUT

Please Tick (✓) Your Choice

Pay for 12 Issues Get 24 Issues	Pay for 24 Issues Get 48 Issues	Pay for 36 Issues Get 72 Issues
2 Years (24 copies) + 24 Online Tracked Deliveries worth ₹480	4 Years (48 copies) + 48 Online Tracked Deliveries worth ₹960	6 Years (72 copies) + 72 Online Tracked Deliveries worth ₹1440
₹1920	**₹3840**	**₹5760**

To subscribe online, visit **https://bit.ly/osfybogo** OR SCAN THIS CODE

Name_____ Organisation_____ Mailing Address_____

_____ City_____

Pin Code_____ State_____ Phone No._____ Email_____

Subscription No. (for existing subscribers only)_____. **I would like to subscribe to the above (✓)marked Open Source For You magazine starting with the next issue.** Please find enclosed a sum of Rs_____ by DD/MO/crossed cheque bearing the No._____ dt._____ in favour of **EFY Enterprises Pvt Ltd,** payable at Delhi. *(Please add Rs 50 on non-metro cheque)*

Please mark one (nearest) relating to your subscription: ☐ Indian Company ☐ MNC ☐ R&D organisation ☐ Engineering institute ☐ College/School ☐ Any other (specify):_____

Send this filled-in form or its photocopy to : **EFY Enterprises Pvt Ltd,** D-87/1 Okhla Industrial Area, Phase 1, New Delhi 110 020 | Ph: **011-40596600** | e-mail: **support@efy.in**

Terms:- # These rates are applicable for new subscribers as well as renewal by existing subscribers. # The rates are valid for subscribers within India only. # Please allow 4-6 weeks for processing of your subscription. # The subscription copies will be dispatched through ordinary post only # Subscription Agents will not get agency commission against this scheme # Disputes, if any, are subject to exclusive jurisdiction of competent courts and forums in Delhi/New Delhi only. * Replacement will be made if intimation of damaged / non-receipt of copies is received within 30 days of its publication ** After three months, if you are not satisfied with the magazine, your balance amount will be returned (Not applicable for gift offer)

EDITOR
RAHUL CHOPRA

EDITORIAL, SUBSCRIPTIONS & ADVERTISING
Delhi (HQ)
D-87/1, Okhla Industrial Area, Phase I, New Delhi 110020
Phone: +91-9811155335
E-mail: info@efy.in

MISSING ISSUES
Phone: +91-9811155335
E-mail: support@efy.in

BACK ISSUES
Phone: +91-9811155335
E-mail: support@efy.in

NEWSSTAND DISTRIBUTION
Phone: +91-9811155335
E-mail: efycirc@efy.in

ADVERTISEMENTS
NEW DELHI (HEAD OFFICE)
Phone: +91-9811155335
E-mail: efyenq@efy.in

MUMBAI
E-mail: rmowest@efy.in

BENGALURU
E-mail: rmosouth@efy.in

CHINA
Worldwide Focus Media
E-mail: china@efy.in

GERMANY
pms Plantenberg Media Service GmbH
E-mail: germany@efy.in

JAPAN
Tandem Inc.
E-mail: japan@efy.in

TAIWAN
J.K. Media
E-mail: taiwan@efy.in

UNITED KINGDOM
ASA Media
E-mail: uk@efy.in

UNITED STATES
E & Tech Media
E-mail: usa@efy.in

Printed, published and owned by Ramesh Chopra. Printed at Tara Art Printers Pvt Ltd, A-46,47, Sec-5, Noida, on 28th of the previous month, and published from D-87/1, Okhla Industrial Area, Phase I, New Delhi 110020. Copyright © 2026. All articles in this issue, except for interviews, verbatim quotes, or unless otherwise explicitly mentioned, will be released under Creative Commons Attribution-NonCommercial 3.0 Unported License a month after the date of publication. Refer to http://creativecommons.org/licenses/by-nc/3.0/ for a copy of the licence. Although every effort is made to ensure accuracy, no responsibility whatsoever is taken for any loss due to publishing errors. Articles that cannot be used are returned to the authors if accompanied by a self-addressed and sufficiently stamped envelope. But no responsibility is taken for any loss or delay in returning the material. Disputes, if any, will be settled in a New Delhi court only.

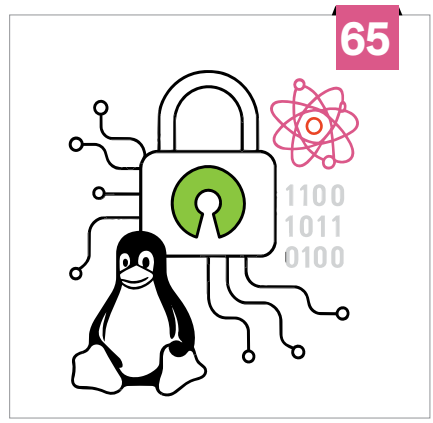
SUBSCRIPTION RATES			
Year	**Newstand Price (₹)**	**You Pay (₹)**	**Overseas**
Five	7200	4320	—
Three	4320	3030	—
One	1440	1150	US$ 120

Kindly add ₹ 50/- for outside Delhi cheques.
Please send payments only in favour of EFY Enterprises Pvt Ltd.
Non-receipt of copies may be reported to **support@efy.in**—do mention your subscription number.

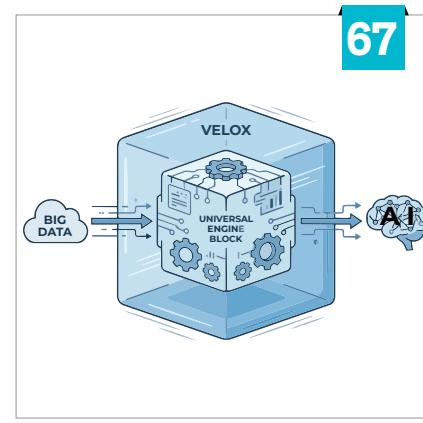
CONTENTS



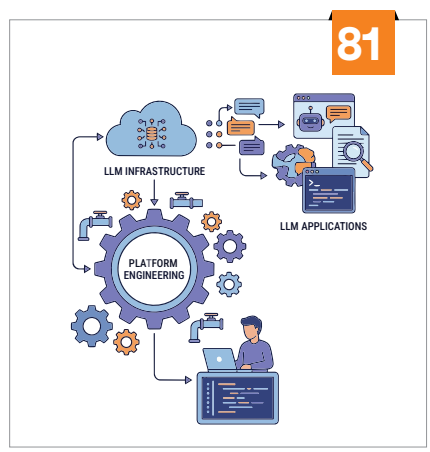
PennyLane: Building a Bridge Between Machine Learning and Qubitsof Quantum Computing



A Practical Guide to Securing Open Source in the Quantum Era



Inside Velox, the 'Universal Engine Block' for Big Data and AI



The Benefits and Uses of Open Source LLMs



Stopping Dangerous AI Agent Actions Before Execution

Bodhan AI launched for Indic languages



Indian Institute of Technology (IIT) Madras-incubated Bodhan AI – a Centre of Excellence focused on artificial intelligence (AI) for education -- has introduced four key AI models designed for Indian languages, in partnership with AI4Bharat and Nvidia. The models cover speech recognition, speech generation, machine translation, and optical character recognition (OCR). They are available as open weights and hosted application programming interfaces (APIs) on sovereign digital infrastructure, enabling startups, edtech firms, researchers and government bodies to build customised applications.

Together, these models form part of the Bharat EduAI Stack, which Bodhan AI envisions as a shared digital public infrastructure for India's multilingual education system. The goal is to prevent duplication of effort, so institutions don't have to rebuild the same AI tools separately for different languages. As Prof. Mitesh Khapra, principal investigator at Bodhan AI explained, Bodhan AI wants to "build with the ecosystem, not compete with it," offering a common technology layer for education-focused AI.

The models were trained using Nvidia's Nemotron open libraries, including the NeMo framework and optimised to handle Indian languages, dialects, and accents. They are deployed with Nvidia TensorRT-LLM and vLLM microservices. Bodhan AI and Nvidia are also collaborating on datasets and training recipes for future language models.

Potential applications include voice-enabled AI tutors and tools that can read and interpret educational material in multiple Indian languages. Bodhan AI is already launching two products based on these models: the 'Student Tutor Bot' and the 'Teacher Assistant Bot'. Importantly, teachers remain in control, with AI-generated content serving only as a draft to be reviewed or edited. Prof. V. Kamakoti, director of IIT Madras, emphasised that India's AI efforts must prioritise technology that respects its linguistic diversity.

India's top institutes spent ₹1.34 billion on proprietary software

India's top universities and technical institutes have spent at least ₹ 1.34 billion in the past five years on proprietary software licences, according to figures compiled through Right to Information queries by a free and open source software (FOSS) collective. The advocacy group warns the real number is likely far higher, since the data covers only 15-20 per cent of premier public institutes and excludes the Indian Institutes of Technology (IITs).

The spending spans email systems, engineering and electronic design automation (EDA) tools for chip design, and digital assignment platforms. "Institutions spend crores every year teaching students software they will never own," the group noted, arguing that dependence on proprietary platforms undermines both technology sovereignty and student learning. FOSS United CEO Sai Rahul Poruri cautioned that vendors could revoke access at any time, exposing universities to serious risks. He added that India's push for sovereignty is being applied too narrowly, ignoring broader dependence on foreign tech.

Despite government policies encouraging open source adoption, proprietary 'lock-in' continues to dominate campus decisions. FOSS advocacy groups call for a shift towards open source alternatives in curricula, positioning them as critical for India's long-term independence and innovation.

As costs rise and sovereignty debates intensify, the choice between proprietary and open source software is set to become a defining issue for the future of Indian higher education.

Nvidia hands over Open Secure AI Alliance to The Linux Foundation

Nvidia is transitioning the Open Secure Artificial Intelligence Alliance it propounded (along with Cisco, HPE, IBM, and dozens of other firms) to The Linux Foundation. The Alliance, launched in July 2026, was built off The Linux Foundation's Akrites initiative and OpenSSF. The idea behind this collaboration is to collectively learn from AI incidents such as the recent hack of Hugging Face by rogue OpenAI agents – an occurrence that was instrumental in the Alliance's formation.

With Hugging Face now a part of Nvidia's growing AI ecosystem, handing over the Alliance to The Linux Foundation helps retain the open identity that the AI industry is now seeking to evolve. Under the Linux Foundation umbrella, the Open Secure AI Alliance will develop open source tools, shared standards, and defensive practices that help organisations inspect, audit, and secure AI systems.

The Linux Foundation says that under its influence, "this community can accelerate the development of shared, open security stack for AI." According to SDx Central, this defensive stack includes initiatives like the shared AI findings exchange (SAFE), which enables organisations to share threat intelligence and collectively bolster defences against emerging AI security risks.

Project Lightwell uses AI-assisted remediation to secure open source software

Open source software is becoming a critical security concern for enterprises as artificial intelligence (AI) accelerates both software development and vulnerability discovery. IBM, Red Hat, and LTM are working through Project Lightwell to address this challenge by using AI-assisted remediation to move from identifying vulnerabilities to delivering validated fixes that can be deployed in production environments.



Lightwell is designed as an enterprise-focused security layer for the open source software supply chain. Instead of stopping at vulnerability scanning, its approach combines AI capabilities with engineering expertise to analyse affected software, develop or identify suitable fixes, validate them, and support their deployment. This is particularly relevant for organisations that depend on large numbers of open source libraries and components across cloud, enterprise, and embedded software environments.

The technology is backed by IBM and Red Hat's broader Lightwell initiative, which includes a global engineering workforce of more than 20,000 engineers and a reported $5-billion commitment towards strengthening open source software security. AI is used to assist vulnerability triage, code analysis, remediation, validation and maintenance, while engineering processes provide additional checks before fixes reach production.

LTM's role expands the technology into enterprise implementation. The company plans to support dependency analysis, risk-based prioritisation, remediation planning, DevSecOps integration, testing, validation and large-scale deployment. This can help engineering teams determine which vulnerabilities require immediate attention instead of treating every software flaw with the same priority.

The approach could be particularly useful for electronics and embedded-system developers, where firmware, operating systems, development tools and software libraries often depend on open source components. Faster remediation can reduce the time that vulnerable components remain in connected products and infrastructure.

Microsoft's WinUI becomes fully open source

Microsoft has completed the planned open source migration of WinUI, the native user interface framework used for Windows 11 applications. Mainline development is now taking place publicly on GitHub, where branches, pull requests and code reviews can be tracked openly.

The move marks the completion of a multi-stage effort that Microsoft began after announcing plans to open source WinUI more than a year ago. The company followed a four-phase migration process, with the third phase completed in May 2026 and the final stage now bringing mainline development into the public repository.

For now, pull requests are primarily being submitted by Microsoft developers as the company tests its end-to-end development processes. This is intended to establish a stable workflow before the project begins accepting contributions from the wider developer community.

Making WinUI development public gives developers greater visibility into the evolution of the framework and allows them to follow changes affecting Windows application development. WinUI is used for building native Windows applications

INDIA'S #1 ELECTRONICS MAGAZINE

electronics FOR YOU

Stay **Connected.**
Stay **Informed.**
Stay **Ahead.**



SUBSCRIBE AND **SAVE**

YOUR TWO SUBSCRIPTION OPTIONS:

Electronics For You Offer

- 12 Issues of Electronics For You worth ₹1200
- Free delivery by courier worth ₹600

SPECIAL OFFER...

You Pay ~~₹1800~~ ₹960

AVAIL THIS OFFER

Electronics For You Magazine + Electronics Industry Directory (EID) Offer



- 12 Issues of Electronics For You worth ₹1200
- Free delivery by courier worth ₹600
- Free Electronics Industry Directory (EID) 2026-27 worth ₹1000
- Free Delivery by Courier (EID) worth ₹200

SPECIAL OFFER...

You Pay ~~₹3000~~ ₹1500

KNOW ABOUT EID 2026 / AVAIL OFFER

TEAR OUT

ORDER FORM

TEAR OUT

Name_____ Organisation_____ Mailing Address_____

City_____

Pin Code_____ State_____ Phone No._____ Email_____

Subscription No. (for existing subscribers only)_____. **I would like to subscribe to the above (✓)marked magazine(s) starting with the next issue.** Please find enclosed a sum of Rs_____ by DD/MO/crossed cheque bearing the No._____ dt._____ in favour of EFY Enterprises Pvt Ltd, payable at Delhi.

Please mark one (nearest) relating to your subscription: ☐ **Indian Company** ☐ **MNC** ☐ **R&D organisation** ☐ **Engineering institute** ☐ **College/School** ☐ **Any other (specify):**_____

Send this filled-in form or copy to : EFY Enterprises Pvt Ltd, D-87/1 Okhla Industrial Area, Phase 1, New Delhi 110 020 | Ph: 011-40596600 | e-mail: support@efy.in

Terms:- # These rates are applicable for new subscribers as well as renewal by existing subscribers # Please allow 4-6 weeks for processing of your subscription.
Please include your pincode for prompt delivery of your copy.

X takes legal action against the Nitter project

X has sent cease-and-desist letters demanding the permanent shutdown of Nitter, an open source project that lets users read public X posts without logging into the platform or using its app. The letters, sent on August 24, 2026, reportedly also targeted multiple Nitter instances. Nitter.net is now offline and development has stopped temporarily while its creator, developer Zedeus, seeks legal advice.

A letter viewed by *TechCrunch* accuses Nitter of the 'unlawful use and circumvention of X's application programming interface (API) and associated data', as well as scraping X data and accessing accounts and session tokens in violation of X's rules.

Nitter said: "On 24 August 2026 cease and desist letters have been sent by X Corp. demanding a permanent takedown of Nitter instances and the project's repository."

The project fetched public X posts while removing ads, tracking cookies and JavaScript. Its code also powers services such as XCancel. In 2024, X API restrictions had already forced Nitter.net offline temporarily and required hosts to connect instances to real X accounts.

The latest action shifts the pressure from technical restrictions to legal enforcement against an open source project, its repository and independent instances. Nitter has said that it would respond to this action.



and remains an important part of Microsoft's Windows App software developer's kit (SDK) ecosystem.

Perplexity open sources AI inference engine Lily

Perplexity has open sourced Lily, a local artificial intelligence inference engine developed for Apple Silicon. The engine powers the local computing component of Hybrid Compute in Perplexity Computer and is designed specifically to run the Qwen3.6-35B-A3B model

Unlike general-purpose AI frameworks designed to support multiple models and hardware platforms, Lily takes a highly specialised approach. It uses a Rust runtime and handwritten Metal kernels optimised around the architecture of Qwen3.6-35B-A3B and Apple Silicon. Neither PyTorch nor Apple's MLX framework is part of Lily's execution path.

The engine provides an OpenAI-compatible chat-completions API for local text generation. Qwen3.6-35B-A3B is a mixture-of-experts model with 35 billion total parameters, although roughly 3 billion parameters are activated for each token. Its architecture combines mixture-of-experts processing with grouped-query attention and Gated DeltaNet layers, creating different computational workloads that Lily specifically optimises for.



According to Perplexity's reported benchmarks, Lily achieved an average 1.23 times higher prefill throughput and 1.35 times faster decoding than MLX-LM when tested across different context lengths on a 40-core, 128GB M5 Max system. The optimisations include keeping expert routing operations on the graphics processing unit (GPU) and combining weight dequantisation with grouped matrix operations to reduce memory overhead.

By open sourcing Lily, Perplexity is making its specialised local inference technology available to developers interested in running large AI models efficiently on Apple Silicon. The project demonstrates an alternative to general-purpose inference frameworks by focusing deeply on the requirements of a specific model and hardware architecture, while supporting Perplexity's broader approach of combining local and cloud-based AI computing.

DeepSeek open sources its first native vision model

DeepSeek has open sourced V4-Flash-Vision-Exp, described as the company's first native vision model. The experimental release expands DeepSeek's artificial intelligence capabilities into multimodal computing, enabling the model to process and understand visual information alongside its broader AI functions.

A key part of the release is its open source availability. DeepSeek has made V4-Flash-Vision-Exp available through Hugging Face, giving developers and researchers a platform to access the model and experiment with its capabilities. The model is released under the MIT License, which permits broad use, modification and redistribution.

The experimental designation indicates that V4-Flash-Vision-Exp is part of DeepSeek's continuing work on multimodal and next-generation AI technologies. The release provides an early look at the company's approach to combining language and visual understanding within its AI ecosystem.

By releasing its first native vision model openly, DeepSeek is extending its open source AI strategy into multimodal systems. Availability through Hugging Face and the use of the MIT License allow the wider developer and research community to access, test, modify and potentially build new applications around the model's vision capabilities.

Multikernel Linux's new release provides kernel isolation without hypervisors

Multikernel Linux has released mklinux v7.0-mk2, its first public test release, as an open source project aimed at providing an alternative to conventional virtual machines (VMs) and containers. It allows multiple independent Linux kernels to run simultaneously on a single bare-metal machine without a hypervisor or container service.

A host kernel divides pools of central processing units (CPUs), memory, and peripheral component interconnect (PCI) devices into instances, with each instance booting and running its own kernel natively on allocated resources. There is no emulation or hypervisor layer intercepting operations, while only explicitly provided resources are shared.



Instances are defined through device trees, with planned overlays allowing CPUs, memory, and devices to move dynamically between the resource pool and running instances without restarting. Resources can also be reclaimed when an instance is shut down.

The project is designed to provide stronger isolation than containers while avoiding some VM overhead. It can potentially assign separate kernels to applications and physical-device drivers, with virtual network and block devices enabling communication.

Initial measurements from Cong Wang, founder and CEO, Multikernel Project, show performance gains of 1.07× to 2.5× over kernel-based virtual machine (KVM) guests, depending on the workload. Context switching was 2.5× faster, while memory bandwidth and latency were reportedly similar. "Compared to VMs, there is no second layer of page tables and no device model," Wang said.

The project targets artificial intelligence (AI) training and inference, latency-sensitive services, and workloads requiring strong isolation. It is initially available for x86_64, based on Linux Kernel 7.0, through its public GitHub repository.

Linux Foundation takes AIRSEAI into its fold

The LF AI & Data Foundation, an umbrella organisation under the Linux Foundation, has welcomed AIRSEAI, an open source embodied artificial intelligence project initiated by the Shenzhen Institute of Artificial Intelligence and Robotics for Society (AIRS). The move provides AIRSEAI with a neutral environment for global collaboration on deployable embodied AI, where artificial intelligence systems interact with the physical world.

AIRSEAI aims to address fragmentation in the robotics ecosystem, where proprietary platforms, hardware-specific software stacks, isolated datasets and models can make systems difficult to evaluate, reuse and deploy across different machines. Under the LF AI & Data Foundation, AIRSEAI will provide a unified framework intended to allow researchers, developers and manufacturers to test and deploy AI across diver**se** hardware architectures without vendor lock-in.

The project follows a three-stage roadmap covering modular single-robot intelligence, data-driven dual-arm manipulation and collective adaptive intelligence. AIRSEAI 2.0 adds production-oriented capabilities such as vision-based tactile sensing. Future AIRSEAI 3.0 updates are planned to enable different types of robots to coordinate across distributed environments.

OpenClaw 2.0 revamps personal AI agent setup

OpenClaw has released version 2026.8.1, described as OpenClaw 2.0, with major changes to the setup and operation of its open source personal artificial intelligence agent. The update introduces a more guided onboarding process, expands the role of the browser interface and adds shared cloud sessions that allow authorised users to continue work through the same Gateway.

The new setup process can detect supported AI access already available on a user's machine, including existing ChatGPT or Claude command-line sign-ins, API keys, configured model routes and compatible local models running through Ollama or LM Studio. OpenClaw tests the selected route with an actual completion request before saving the configuration, helping users confirm that their chosen model connection works correctly.

OpenClaw 2.0 also makes its browser application a central workspace for conversations, settings and ongoing agent activities. Users can return to existing work and monitor agent actions through the browser, while the Gateway continues to hold the main conversation state, credentials, permissions and other durable information.

A significant technical change involves session storage. Existing file-based sessions and transcripts can be migrated to SQLite during the update process. The release also introduces shared cloud sessions, allowing authorised collaborators to join or take over work stored by a Gateway. Remote workers can execute tasks using the shared session state without directly receiving provider credentials. OpenClaw's role and permission system controls access, although the project makes clear that users sharing a Gateway remain within the same trust domain rather than operating in fully isolated environments.

3RD EDITION

aiDevCon INDIA



2026 AT A GLANCE

Delegates
3,637

Speakers	Sessions
62	**38**

Workshop	Exhibitors & Sponsors
10	**15**









INDIA'S ONLY CONFERENCE FOR DEVELOPERS WHO...

WORK ON **AI** & WORK WITH **AI**.

3RD EDITION

ai DevCon INDIA

4-5 March 2027
NIMHANS Convention Centre, Bengaluru

Looking to hire, sell to, or influence AI developers?
Call +91-9811155335 or growmybiz@efy.in
to sponsor or exhibit

Claude designs 1320 protein binders with open source software

Anthropic's Claude autonomously designed 1320 protein binders using only open source, freely downloadable design software, with Anthropic paying no software licensing fees. The technical report says AlphaFold 3 weights and two other packages were excluded because of restrictive licences.

Given 16 protein targets, cloud compute and a roughly 16,000-word protocol, Claude researched each target, selected binding sites, ran the software and produced 30 ranked designs per target. Two independent laboratories then synthesised and tested the candidates.

Of the 1320 designs, 354 bound their targets, producing a 27 per cent hit rate, while 49 per cent of Claude's top-ranked designs succeeded. At least one design worked for 14 of the 15 targets with readable measurements. Against RBX1, Claude produced 28 binders from 90 designs, compared with nine from 245 entries in a human open design competition. Its strongest design bound roughly 10 times more tightly than the competition winner in side-by-side testing.

However, the experiment also exposed the next bottleneck: physical synthesis and experimental screening. Adaptyv Bio tested all 1320 designs, while Twist Bioscience tested 1260 independently. Experimental screening remains the only way to learn which targets a campaign has succeeded on and the affinity of the designed binders.

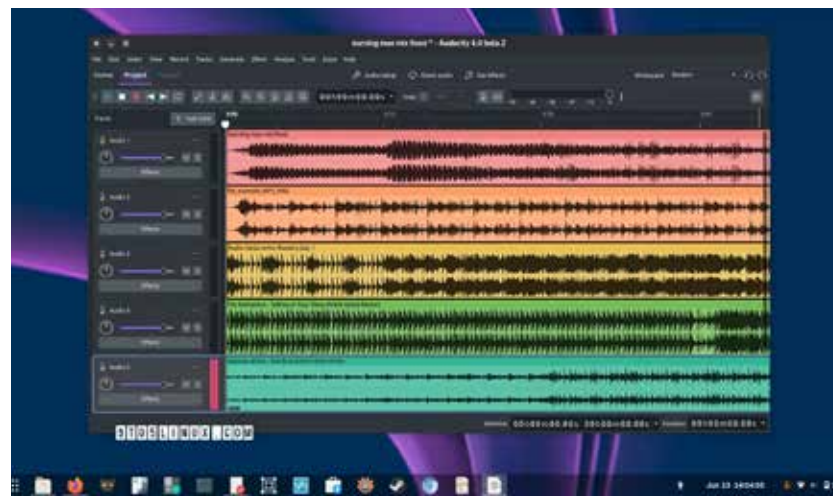
Anthropic has published the prompts and data on Hugging Face for others to rerun. The study demonstrated binding, not biological function, and does not establish that Claude outperformed human experts.

The open framework is intended to connect data, models, software and different robotic platforms on a common foundation. This could allow robots to continuously learn, adapt and improve through real-world deployment while giving the wider community an environment for developing and sharing embodied intelligence.

The addition of AIRSEAI strengthens LF AI & Data Foundation's efforts to establish open and interoperable infrastructure for embodied AI. By bringing the project into a neutral open source ecosystem, the foundation aims to reduce proprietary barriers and support collaboration around robotics systems that can operate across different hardware architectures and deployment environments.

Audacity 4.0 brings a new look to audio editing

The open source audio editor Audacity 4.0 has officially been released, marking one of the biggest updates in the project's history. The new version rebuilds the application's interface using Qt and introduces a redesigned editing experience, more flexible recording controls and major changes to how projects and audio clips are handled.



One of the biggest changes is the new clip-editing model. Users can directly select individual clips or multiple clips and edit them together. Clips can also be grouped so they remain together when moved, copied, or duplicated. The update further allows clips to move more freely between compatible mono and stereo tracks, while new splitting tools, alignment guides and sample-boundary snapping expand editing options.

The interface has also received a complete overhaul with native high-DPI rendering. Toolbars and panels can now be moved, docked, floated, shown or hidden, while users can save different layouts as 'workspaces'. Audacity 4.0 includes 'Modern', 'Classic' and 'Music' workspaces, alongside light, dark and high-contrast themes, accent colours, track colours, and different clip styles.

Audacity 4.0 also changes several core workflows. Recording can begin from anywhere on the timeline, while playback can seek to another position without stopping. Punch and roll, lead-in recording, latency compensation, software playthrough and per-track input monitoring have also been redesigned. The older 'Select', 'Envelope, 'Draw', and 'Multi-tool' modes have been replaced with context-sensitive controls.

However, some Audacity 3 features, including 'Time Tracks', 'Note/MIDI tracks', 'Mixer', 'Macro Manager', 'Scripting pipe', 'VAMP' and 'LADSPA' plugin hosting and 'Play-at-Speed' are not yet available in version 4.0.

Hugging Face launches Microduck robot

Hugging Face has unveiled Microduck, a 25cm-tall open source robot developed with its robotics subsidiary Pollen Robotics. The duck-shaped robot is designed as an accessible platform for experimenting with physical AI and reinforcement learning, allowing developers to teach it new behaviours and deploy them on the robot. Microduck can walk, crouch, recover from several common falls, pick up objects weighing up to 800 grams with its beak and even roller-skate.

The robot uses a camera, lidar sensors and two inertial measurement units (IMUs) to perceive its surroundings and monitor movement. Its compact design weighs less than 800 grams, making it small enough for desktop use while still allowing it to move through its environment. The design focuses more on physical interaction and movement than conversational interaction.

Microduck's software development kit, simulation environment and complete reinforcement-learning training stack are available on GitHub. Developers can train behaviours in simulation, transfer them to the physical robot, evaluate the results



and then retrain and redeploy the behaviour. This workflow is intended to make experimentation with physical AI more accessible and allow developers to share robotics behaviours in a manner similar to sharing AI models.

Pollen Robotics has positioned Microduck as a platform for learning how AI behaviours move from simulation into real-world machines. Rather than attempting to solve every robotics challenge, the project intends to provide developers, researchers and enthusiasts with a relatively accessible physical platform for experimenting with movement, reinforcement learning and robot control. The open software stack also gives users greater visibility into how behaviours are developed and deployed.

Microduck is available for pre-order at US$ 399, before taxes and shipping, with first deliveries targeted before Christmas 2026 in North America, Europe and the UK.

Arduino's VENTUNO Q board boosts edge AI

Arduino has opened pre-orders for the VENTUNO Q, a high-performance single-board computer designed to push the company deeper into edge artificial intelligence (AI), robotics and real-world automation. The board combines an eight-core Qualcomm processor with a dedicated microcontroller, creating a platform that handles both computationally intensive AI workloads and time-critical hardware control on the same system.

At the centre of the board is Qualcomm's Dragonwing IQ-8275 processor, featuring eight 64-bit Kryo central processing unit (CPU) cores running at up to 2.35GHz, an Adreno 623 graphics processing unit (GPU) and a Hexagon DSP-based AI accelerator.

Nvidia adds vGPU support to open source Nova driver

Nvidia continues to expand the capabilities of its open source Nova kernel graphics driver for Linux. The company has posted a series of 13 patches introducing an Nvidia vGPU manager and a virtual function I/O (VFIO) variant driver, extending the functionality being developed around the upstream Nova driver.

The new patches are designed to enable vGPU management, allowing a physical GPU to support virtualised workloads. They also add support for VFIO users working with quick emulator (QEMU), enabling graphics processing unit (GPU) virtualisation for both Linux and Windows guest operating systems while using the upstream Nova kernel driver.

VFIO is commonly used in Linux virtualisation environments to provide virtual machines with more direct access to hardware. In this case, the new Nvidia work is aimed at integrating virtual GPU functionality into the Nova driver stack rather than relying solely on separate proprietary driver components.

However, the initial support is expected to focus primarily on Nvidia's workstation and data-centre GPUs, rather than consumer GeForce graphics cards. The patch series provides an early indication of how Nvidia plans to support virtualised GPU workloads as development of the open source Nova driver continues.

The Nova driver is still under development and is not yet ready for general end user deployment in the mainline Linux kernel. Nevertheless, the addition of vGPU management and VFIO support shows Nvidia continuing to build functionality around its open source Linux graphics driver, with future developments expected to expand its role in the company's Linux GPU software ecosystem.

openKylin 3.0 offers deeper integration with AI agents

Open source operating system project openKylin has released version 3.0, introducing deeper artificial intelligence (AI)-agent integration while expanding its open Linux ecosystem across developers, hardware and computing environments.

The release brings KylinBot into the OS layer, allowing it to understand voice and text requests, retain context across interactions, and perform system tasks. Integration with Kylin-CUA, a computer-use automation tool, gives KylinBot access to functions such as system settings and device management.

Rather than limiting AI capabilities to its own agent, openKylin also supports third-party tools including OpenClaw and WorkBuddy, allowing developers to combine AI capabilities for different use cases. The project says its open ecosystem and large-scale AI resources can lower barriers to developing and using AI agents.

"The shift from 'AI as an application' to 'AI as an operating system' is not simply a feature upgrade, but a transformation of the underlying architecture," said Wu Qingbo, chair of the openKylin technical committee. "An open ecosystem will play an important role in this process, allowing developers to build and extend intelligent capabilities in a more transparent and secure way while adapting to a growing hardware ecosystem."

openKylin 3.0 also upgrades to the Linux 7.0 kernel, optimises more than 180 core components, expands reduced instruction set computer five (RISC-V) support and increases Rust use in system tools. The project targets AI-powered devices, robotics, industrial automation, high-performance computing and cloud computing.

The architecture delivers up to 40 trillion operations per second (TOPS) of AI performance, targeting workloads such as computer vision, machine learning, local large language models and agentic AI applications running directly at the edge.

However, the board's key electronics feature is its 'dual-brain' architecture. Alongside the Qualcomm processor sits an STMicroelectronics STM32H5 microcontroller dedicated to real-time workloads. This separation allows the high-performance processor to manage AI inference and operating system tasks while the microcontroller handles deterministic operations such as sensing, actuation and motor control.

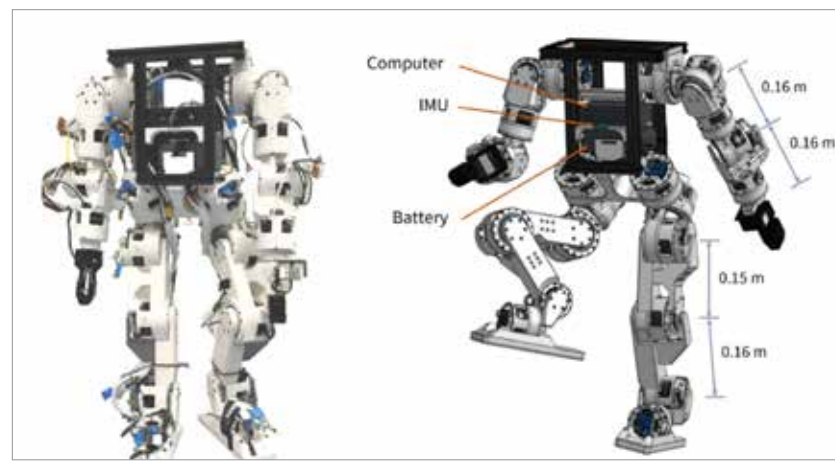


The board includes 16GB of LPDDR5 RAM and 64GB of eMMC storage, with an onboard M.2 slot supporting NVMe SSD expansion. Unlike the compact microcontroller boards traditionally associated with Arduino, the board is physically larger and requires active cooling with a heatsink and fan to sustain demanding workloads. The platform is also intended to bridge the gap between prototyping and commercial deployment. Through Arduino's 'Works with Arduino' programme, developers can potentially move applications developed on the board towards production-grade system-on-modules based on the same Qualcomm architecture without redesigning the complete software stack.

Berkeley Humanoid Lite cuts robot costs using modular actuators

Humanoid robots, including smaller-scale designs, can be expensive to develop and build. The Berkeley Humanoid Lite takes a low-cost approach by using 3D-printed actuators that form a significant part of the robot's structure. The project is designed as an open source and customisable humanoid robot reference platform.

At the centre of the design are modular actuators that combine motors, 3D-printed cycloidal gearboxes and embedded magnetic encoders. The modular approach means that the actuators can potentially be used independently in other robotics projects, without requiring developers to reproduce the complete humanoid robot.



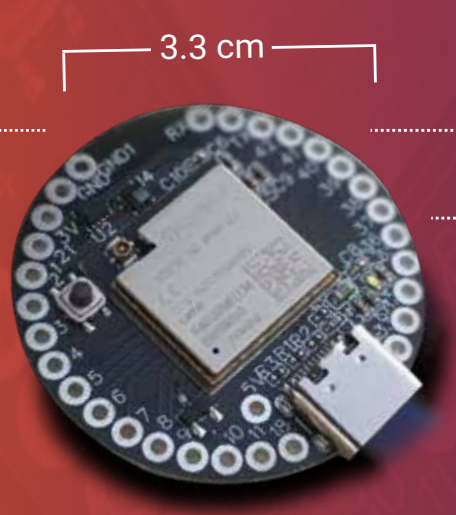
Rather than being presented as a finished commercial product, Berkeley Humanoid Lite is intended to serve as an accessible platform for experimentation and modification. The project's GitHub repository contains the resources needed to build and customise the robot, while demonstrations include virtual reality (VR)-driven teleoperation.

India's Only Open Source Wearable IoT Dev Board & Kits

INDUSBOARD COIN

Ultra-compact 3 cm development board with built-in multi-sensor array

- CAP-Touch Sensor
- 3D Compass
- 3D Mag Flux
- Lux Light Sensor



- Chip + Acc Temperature
- RealTime Clock
- Acc Sensor
- Proximity Sensor

Indus Board Coin

~~₹2,199~~

₹2,089

BUY NOW

IoT & Embedded Systems: Starter Kit



~~₹3,149~~ **₹2,991**

BUY NOW

Industrial IoT (IIoT) Kit



~~₹3,199~~ **₹3,039**

BUY NOW

NFC & RFID Embedded Systems IoT Kit



~~₹2,449~~ **₹2,326**

BUY NOW

Health Tech & Medical Device IoT Kit



~~₹2,499~~ **₹2,374**

BUY NOW

Agriculture IoT & AI DIY Kit



~~₹2,599~~ **₹2,469**

BUY NOW

KNOW MORE

SAVE 5%

Use Promo Code: EFY5

Prices includes delivery charges and taxes

IndusBoard COIN

OpenSSL 4.0.2 patches multiple security flaws across core components

OpenSSL, the widely used open source transport layer security/secure sockets layer (TLS/SSL) and cryptography library, has released OpenSSL 4.0.2, its second maintenance and security update for the OpenSSL 4.0 series, fixing multiple security vulnerabilities.

Among the key flaws addressed is a heap buffer overflow in CMS key unwrapping (CVE-2026-63072), alongside excessive memory use while buffering DTLS records (CVE-2026-54874) and a client-side memory leak during OCSP response checking (CVE-2026-54876).

The update also fixes an invalid pointer dereference in the CMP server through a crafted protectionAlg (CVE-2026-63076), unbounded memory growth in the QUIC server incoming channel queue (CVE-2026-14456), and a QUIC server double-free issue when processing an INITIAL packet (CVE-2026-18798).

Further fixes address RPK certificate handling, an untrusted Sender DN being used as a format string during CMP response validation, indefinite extraCerts cache growth, QUIC ACK-only packet memory exhaustion, and the possibility of AEAD forgeries with empty ciphertext when using EVP_Cipher().

Released two and a half months after OpenSSL 4.0.1, the updates underline the need for organisations using OpenSSL to apply upstream security fixes across the versions they operate. Users are advised to patch as soon as possible or when the updates reach stable GNU/Linux distribution repositories.

The complete hardware system costs under US$ 5,000, making it considerably less expensive than many other humanoid robots, although it still represents a substantial investment for hobbyists and researchers. The use of DIY actuators and 3D-printed components is central to reducing the overall cost of the platform.

By making the robot design and its modular actuator technology openly available, Berkeley Humanoid Lite provides developers with a platform for experimenting with humanoid robotics. Its design also demonstrates how open source hardware and DIY manufacturing techniques can help make complex robotic systems more accessible for research and development.

Modellix releases plugin for DeepSeek Harness

Modellix.ai has released dsh-modellix, a beta plugin for DeepSeek Harness, an open source artificial intelligence (AI) coding agent that has reportedly attracted nearly 200,000 GitHub stars within two weeks of its August 13, 2026, launch.



The plugin connects Harness to Modellix's large language model (LLM) gateway through a single application programming interface (API) key, with free LLM access as its key offering. Developers can select modellix-ai/free-llm, which automatically routes requests to current state-of-the-art open source LLMs, or use zai/glm-4.7-flash at $0 per call.

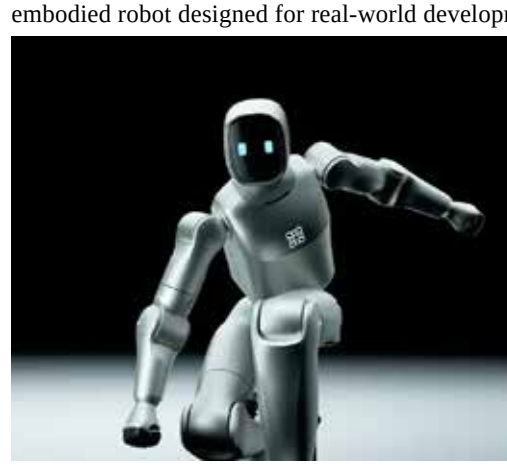
Beyond the free options, developers can access 28+ models spanning OpenAI, Anthropic, Google, xAI, GLM, Qwen, DeepSeek, and Kimi without changing API keys.

The integration reflects Harness's plugin-first architecture, built around the motto 'Everything is a plugin'. Modellix's plugin integrates into Harness's native model selector, allowing third-party providers to extend the open source coding workflow.

The plugin provides three independently toggled capabilities: LLM access, web tools through web_search and web_fetch, and design tools for image, video, and audio generation.

Zeroth launches Bridge and OpenBridge ecosystem

Zeroth has launched Bridge, a compact artificial intelligence (AI)-native embodied robot designed for real-world development and experimentation.



Alongside the robot, the company has introduced OpenBridge, a fully open source ecosystem for embodied intelligence that aims to give developers a shared environment for creating, testing and distributing robot capabilities.

Bridge stands 88cm tall and weighs approximately 13kg, combining a compact humanoid form with 30-40cm arms and two-finger grippers. The robot is designed to provide stable bipedal walking, obstacle avoidance, and a library of natural movements and choreography. Zeroth describes its

architecture as AI-native, with its hardware, motion system and developer interfaces built for AI-driven physical interaction rather than relying only on fixed, pre-programmed movements.

The newly launched OpenBridge ecosystem is intended to connect Bridge with a broader open source developer community. It provides unified development tools, shared training resources and a public Skill Hub, allowing developers to contribute, share and build upon robot capabilities created by others. Zeroth's goal is to move robot development towards a more collaborative model in which capabilities can be developed by the wider community rather than remaining isolated within individual robotic systems.

Zeroth has also introduced the 'Bridge Builder Programme' to bring developers into the OpenBridge ecosystem. In its first phase, 500 developers will be shortlisted for a co-creation challenge, with the top 200 receiving complimentary access to Bridge. The remaining 300 participants will receive priority access to purchase the Bridge Geek Edition, along with SDK access and eligibility for future programme opportunities. The first batch of the Bridge Geek Edition is also being made available to developers and technology enthusiasts.

DataAgent raises US$ 10M to help cut observability costs

DataAgent has launched with US$ 10 million in pre-seed funding to expand an open source artificial intelligence (AI) agent that diagnoses and repairs production faults directly inside customers' Kubernetes clusters. Founded in January 2026 by Ishay Yaari, chief executive officer, and Nati Shalom, chief technology officer, the company plans to use the funding to accelerate customer adoption in North America.



The agent analyses live system state, topology and configuration drift where the data already resides, determines the likely cause of an incident, and applies customer-pre-approved fixes through defined guardrails. Deeper root-cause analysis runs offline after service restoration, which DataAgent says can reduce mean time to resolution.

The agent can run standalone at no cost, while a paid SaaS tier provides fleet management and orchestration. It operates inside cloud-native control planes alongside existing observability tools, with data leaving the customer environment only when deeper external inspection is required.

"In 10 years, no one has ever heard an engineering leader say their organisation's observability costs went down and that is no accident," Yaari said. "When a vendor's revenue is your data ingest, it cannot cut your bill without cutting its own."

Shalom said the industry has spent 15 years "building better ways to watch production and charging more for it every year," adding that AI copilots have not changed the outcome. He described autonomy as an architectural shift rather than an observability feature.

DataAgent claims customers can cut observability costs by up to 90 per cent.

For more news, visit *www.opensourceforu.com*

Spooqy opens door to quantum programming with Tyche

Spooqy Technology has released Tyche, an open source Python library designed to provide the foundation for working with quantum states and measurements. The library forms part of the company's broader software roadmap, which also includes Eris, another open source library intended to build contextual quantum applications on top of Tyche.

Tyche provides the algorithms and data structures needed to represent programmable quantum objects. The framework acts as a basic programming layer for quantum states and measurements, while Eris builds on this foundation to address problems related to contextuality, which Spooqy identifies as an important aspect of quantum mechanics for developing applications.

Both platforms have been developed with the aim of keeping the software freely available to users. According to Spooqy, the company does not impose a licence on the use of these platforms. The open source approach allows users to work with and manipulate quantum-information data structures. Spooqy also provides an application programming interface (API) for connecting with the platforms, along with a web portal that provides access to AI tools and educational material.

Tyche and Eris form part of a larger compilation stack being developed by Spooqy. The company plans to use this stack to convert high-level application designs into hardware blueprints based on contextual design and quantum computation. Some features of the compilation stack may be provided through Spooqy Cloud services, while more advanced functionality is planned for designing customised applications.

EFY interactive

Asia's #1 **Digital Marketing Agency** for Electronics and Tech Industry

WE SPECIALISE IN B2B + TECH MARKETING

Digital Marketing | Events | Publications | Databases



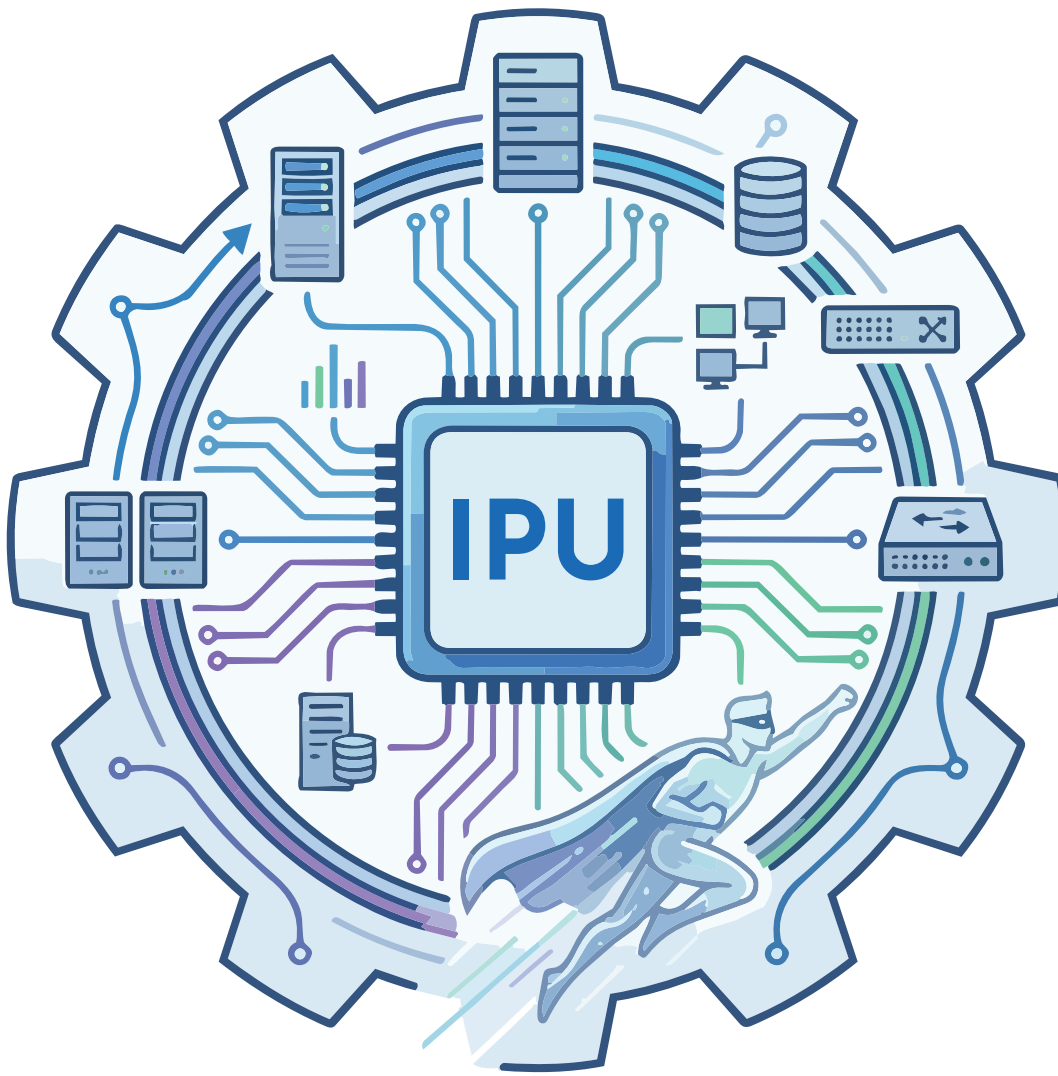
An EFY GROUP DIVISION

BRANDS WE WORK WITH...



For more information:
+91-98111-55335 | growmybiz@efy.in

Infrastructure Processing Units: The Unsung Heroes of the Data Centre



The future of cloud computing rests on the lesser-known infrastructure processing units. Here's why...

In the relentless race of modern computing, the spotlight is almost exclusively dominated by the heavy weights of processing power. If you read the headlines, you would assume that the entire digital world rests solely on the shoulders of central processing units (CPUs), graphics processing units (GPUs), and tensor processing units (TPUs). These chips are the celebrated rockstars of the artificial intelligence revolution. They are the brains orchestrating

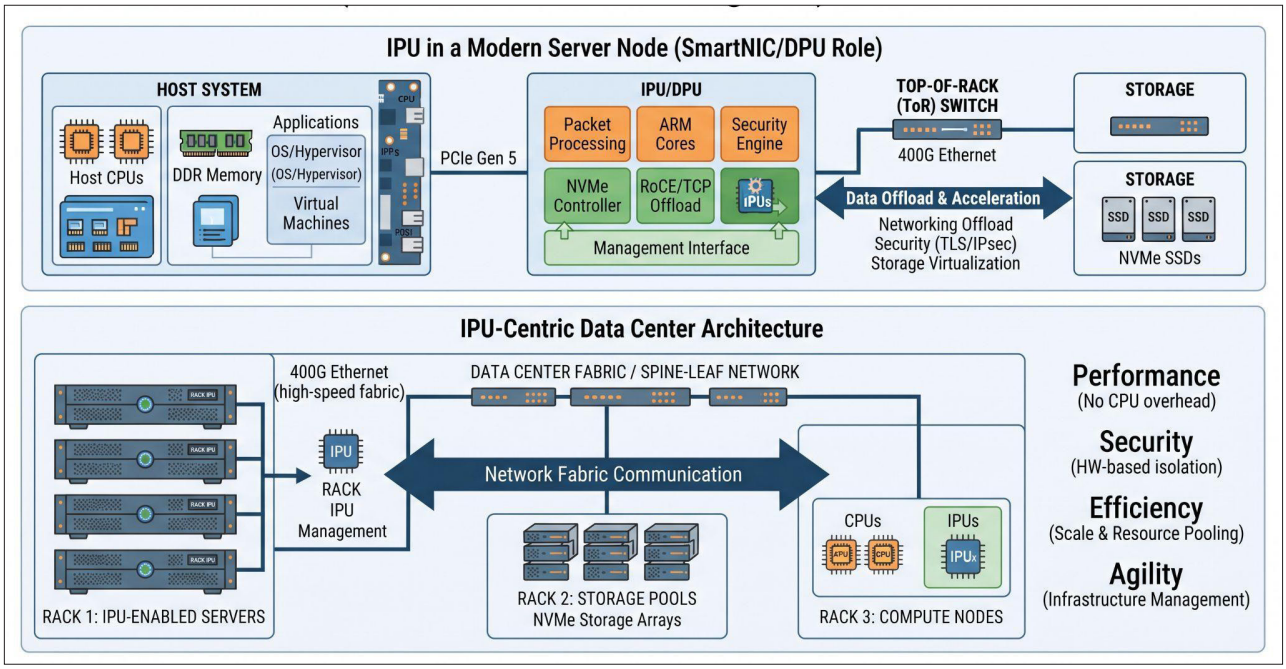


Figure 1: The role of the IPU in a server and data centre

operating systems, the raw muscle rendering immersive digital worlds, and the complex matrix-crunchers training the next generation of massive large language models (LLMs).

However, beneath the glamour and the staggering teraflops of computational power, there is a quiet, profound revolution occurring in the networking racks and server chassis of the modern data centre. Behind every highly publicized GPU cluster and every hyper-efficient TPU supercomputer, there is a vital layer of invisible infrastructure keeping the entire ecosystem from collapsing under its own weight.

Enter the infrastructure processing unit (IPU) — the unsung hero of the data centre. While CPUs, GPUs, and TPUs receive all the attention, IPUs are the dedicated workhorses managing the chaotic, high-volume flow of data, securing the perimeter, and ensuring that the high-profile processors can do exactly what they were designed to do: compute.

The hidden cost of the cloud: The 'data centre tax'

To understand why IPUs are becoming the most critical component in hyperscale architecture, one must first understand the fundamental bottleneck of the modern cloud.

In a traditional, CPU-centric data centre, the central processing unit is the undisputed orchestrator of everything. Not only is it responsible for running the revenue-generating workloads, it is also burdened with managing the underlying infrastructure.

As cloud environments have evolved from monolithic applications to highly distributed, microservices-based architectures, the volume of internal 'East-West' traffic within the data centre has exploded. To manage this, hyperscalers rely on software-defined networking (SDN), software-defined storage, and complex hypervisors. The problem is that running these virtualised infrastructure services requires a massive amount of computational power.

The infrastructure overhead tax: In a conventional cloud server, up to 30% to 50% of the CPU's core capacity can be consumed by background infrastructure tasks. This includes managing virtual switches (vSwitches), routing network packets, encrypting and decrypting data, and managing storage input/output (I/O).

For a cloud service provider (CSP), this is a devastating inefficiency. If a server is equipped with a high-end 64-core processor, but 20 of those cores are busy just keeping the network and storage running, those are 20 cores that cannot be rented out to a paying customer. In an industry where revenue is directly tied to the utilisation of available compute, this 'data centre tax' represents billions of dollars in lost monetisation potential.

Decoding the IPU: What it is and how it works

An IPU is a highly advanced, programmable network device designed specifically to intelligently manage system-level infrastructure resources. It intercepts the data before it ever reaches the CPU, acting as a dedicated brain for the data centre's plumbing.

While earlier iterations of network cards (like standard NICs) simply moved packets from a cable to the host memory,

and SmartNICs added basic offloading capabilities, the IPU represents a total architectural paradigm shift. It is essentially a specialised computer inside a computer.

A modern IPU typically integrates several key hardware components:

- *High-speed network interfaces:* Capable of handling line-rate speeds of 200Gbps, 400Gbps, or even 800Gbps.
- *Programmable compute cores:* Dedicated processors (often highly efficient ARM-based cores) designed to run the cloud provider's infrastructure software independently of the host CPU.
- *Hardened hardware accelerators:* Custom silicon dedicated to executing highly specific, repetitive tasks at blistering speeds - such as cryptography (IPsec/TLS encryption), data compression, and packet routing.
- *High-bandwidth memory:* Fast, localised memory to prevent bottlenecks during intensive data manipulation.

By placing this powerful, specialised device at the edge of the server motherboard, cloud operators can securely shift the entire burden of a fully virtualised storage and network architecture away from the CPU.

The transformational impact of IPUs in the data centre

The integration of IPUs is not merely an incremental upgrade; it completely redefines how data centres are designed, operated, and monetised. Here are some specific, game-changing advantages that IPUs deliver to modern cloud environments.

Unlocking 100% compute monetisation: The primary financial driver for IPU adoption is CPU reclamation. By offloading the hypervisor, storage management, and networking stack to the IPU, the 'data centre tax' is effectively reduced to zero.

For the cloud service provider, this means that every single core of the expensive, power-hungry server CPU can be dedicated entirely to the tenant's workload. If a user pays for a bare-metal instance or a massive virtual machine, they receive the unhindered, raw performance of the entire CPU. The IPU allows CSPs to achieve maximum density and profitability out of every square foot of their server racks, dramatically lowering the total cost of ownership (TCO).

Bulletproof security and air-tight isolation: In a multi-tenant cloud environment, security is paramount. In a traditional setup, the infrastructure software (the hypervisor) and the tenant's application share the same physical CPU. If a malicious actor successfully compromises the hypervisor or exploits a side-channel vulnerability in the CPU cache, they could potentially access other tenants' data or compromise the entire host system.

The IPU introduces a profound layer of physical and logical separation. Think of a cloud server as a luxury hotel. You want your high-paying guests (the tenant applications) to have a wonderful experience in their suites. However, you absolutely do not want them having the keys to the boiler room, the security cameras, or the plumbing systems.

With an IPU, the infrastructure control plane is entirely physically separated from the tenant data plane. The cloud provider's security policies, firewalls, and management software run exclusively on the IPU. Even if the tenant completely compromises the host CPU, they cannot breach the underlying network or access the infrastructure management layer, because it is running on a separate processor. This creates an impenetrable root of trust.

Pioneering the diskless data centre: Storage management has traditionally been a rigid and localised problem. Servers have physical hard drives or solid-state drives (SSDs) installed directly within their chassis. This leads to stranded resources: one server might run out of storage while having excess compute, while the server next to it has unused terabytes of storage but a maxed-out processor.

IPUs solve this by enabling the 'diskless data centre'. Through protocols like NVMe-over-Fabrics (NVMe-oF), an IPU can connect a server to a massive, centralised pool of storage located across the network. The IPU manages the complex routing and data compression so efficiently that, to the host CPU, this remote storage behaves as if it is a physical SSD plugged directly into its own motherboard. This disaggregation allows cloud providers to scale compute and storage completely independently, eliminating over-provisioning and drastically reducing hardware waste.

Feeding the AI behemoths: While GPUs and TPUs are the engines of the AI era, they are only as effective as the data pipeline feeding them. AI training requires pulling petabytes of unstructured data from storage and pushing it into the memory of parallel accelerators as fast as possible. If the network or the CPU bottlenecks, the incredibly expensive GPUs end up sitting idle, waiting for data — a phenomenon known as 'data starvation'.

IPUs are the perfect traffic controllers for AI clusters. They handle the East-West network traffic with ultra-low latency, bypass the CPU entirely through Remote Direct Memory Access (RDMA), and ensure that data is encrypted, compressed, and delivered directly to the GPU or TPU memory banks precisely when needed. By keeping the accelerators fed at line rate, IPUs ensure that the multimillion-dollar investments in AI compute are fully utilised.

Comparing architectures: The paradigm shift

To visualise the sheer impact of the IPU, consider how the fundamental architecture of the server changes.

Feature	Traditional CPU-centric architecture	IPU-accelerated architecture
Workload placement	Infrastructure and tenant applications compete for the same CPU cores.	Clean separation: CPU handles tenant applications, IPU handles infrastructure.
Server monetisation	70% - 80% CPU utilisation available for billing.	100% CPU utilisation available for billing.
Security model	Software-based hypervisor isolation; vulnerable to host CPU exploits.	Hardware-based air-gap; infrastructure control plane is physically isolated.
Storage strategy	Localised, physical disks prone to stranded capacity.	Diskless servers utilising pooled, network-attached NVMe via the IPU.
Data throughput	Bottlenecked by CPU interrupts and software switching.	Line-rate hardware acceleration (400Gbps+) with zero CPU intervention.

Programmability: Moving at the speed of software

Hardware acceleration is not a new concept — application-specific integrated circuits (ASICs) have been doing it for decades. However, ASICs are famously inflexible. Once manufactured, their logic is permanently etched into the silicon. If a cloud provider wants to update a network protocol or deploy a new security encryption standard, an ASIC becomes obsolete.

The genius of the modern IPU lies in its programmability. By combining hardened, fixed-function accelerators (for tasks that will never change, like basic packet routing) with programmable cores and open software frameworks (such as P4 for data plane programming), IPUs offer the best of both worlds.

Cloud operators can customise their data pipelines, deploy advanced telemetry, and roll out new hyper-scale microservices through software updates that directly manipulate the IPU's behaviour. This allows data centres to adapt to shifting AI workloads, new cybersecurity threats, and evolving cloud orchestration frameworks without needing to physically replace the hardware in millions of servers.

IPUs: The backbone of the next-generation cloud

We have officially reached the physical limits of the monolithic, CPU-does-it-all server architecture. As the world demands exponentially more compute power to fuel generative AI, real-time massive data analytics, and ultra-low-latency edge applications, the infrastructure must evolve to keep pace.

While the fascination will undoubtedly remain fixated on the raw intelligence of CPUs, the parallel processing power of GPUs, and the deep-learning mastery of TPUs, the true scalability of the cloud relies on an entirely different piece of silicon. IPUs are the dedicated guardians, the efficient traffic directors, and the tireless workers that allow the data centre to operate securely and profitably at a staggering, global scale.

The industry is waking up to this reality. The shift from compute-heavy nodes to heavily disaggregated, IPU-centric architectures is no longer just a theoretical roadmap — it is a financial and operational imperative. Because of their profound ability to reclaim revenue, harden security, and accelerate data pipelines, IPUs are incredibly important and are going to get a significant amount of investment from all cloud service providers in the next few years. END

References

- *https://community.intel.com/t5/Blogs/Tech-Innovation/Data-Center/The-IPU-A-New-Strategic-Resource-for-Cloud-Service-Providers/post/1335081#:~:text=Offloading%20infrastructure%20tasks%20to%20the,driven%20the%20development%20of%20IPUs*
- *https://simplyblock.io/glossary/infrastructure-processing-unit/#:~:text=for%20higher%20efficiency.-,How%20an%20IPU%20Handles%20Infrastructure%20Workloads,traffic%20routing%2C%20and%20tenant%20isolation*
- *https://www.presidio.com/blogs/intel-ipus-and-smartnics-boost-data-center-utilization-and-lower-tco/#:~:text=Introduced%20in%20June%202021%2C%20the,intelligently%20accelerates%20and%20manages%20them*
- *https://medium.com/intel-tech/the-infrastructure-processing-unit-for-the-next-generation-cloud-services-3d813d1d4ed4*
- *https://www.youtube.com/watch?is=pXjE-x2L8ipNjSN8&v=6Z20LAbHy48&feature=youtu.be*
- https://www.trentonsystems.com/en-*us/resource-hub/blog/what-is-an-ipu#:~:text=It%20allows%20cloud%20operators%20to,architecture%20in%20the%20data%20center*
- *https://semiwiki.com/semiconductor-manufacturers/intel/361110-intels-ipu-e2200-redefining-data-center-infrastructure/#:~:text=It%20also%20plays%20a%20crucial,Summary*

By: Ishaan Gupta

The author leads a team working on compute accelerators for AI workloads at Google. In his 13+ years of experience in the software and AI industry, he has worked in the augmented reality AI glasses organisation at Meta, and in Square, the payments company.

The Need for Spec-Driven Development in the Agentic AI Era

Spec-driven development is a critical discipline for the age of agentic software engineering. Organisations that define the right intent and turn specifications into durable engineering assets will do well in this age.



Software development is entering a phase where the most valuable engineering artifact is no longer only the codebase. In AI-assisted and agentic development, the real source of leverage is the clarity of intent that guides humans, tools, and autonomous coding agents. Spec-driven development, often shortened as SDD, responds to this shift by treating the specification as a living contract that defines what must be built, why it matters, how success will be verified, and where the boundaries of acceptable implementation lie. This approach is becoming important because AI coding agents can produce code faster than most teams can review, but speed without shared intent can quickly turn into confusion, rework, technical debt, and governance exposure.

Understanding spec-driven development

Spec-driven development is a disciplined software engineering approach in which development begins with a clear, versioned, reviewable specification rather than with an informal prompt or a half-formed implementation idea. The specification describes the problem, user value, functional behaviour, non-functional expectations, constraints, assumptions, interfaces, acceptance criteria, and verification strategy. In conventional projects, specifications often became static documents that were written early and forgotten later. In SDD, the specification remains active throughout the lifecycle. It informs design, drives task decomposition, guides AI-generated code, anchors testing, supports review, and provides traceability for operational and compliance needs.

The rise of coding agents has made this practice more than a documentation preference. When an AI agent receives an ambiguous instruction, it must still make implementation choices. It may select a design pattern, create a data model, call a library, introduce an API behaviour, or add assumptions that were never discussed. A well-written specification reduces these silent decisions. It gives the agent a bounded problem space and gives human reviewers a stable reference against which the output can be judged. In that sense, SDD is not anti-agility. It is a way of making agility safer when software is produced by a combination of human judgment and machine execution.

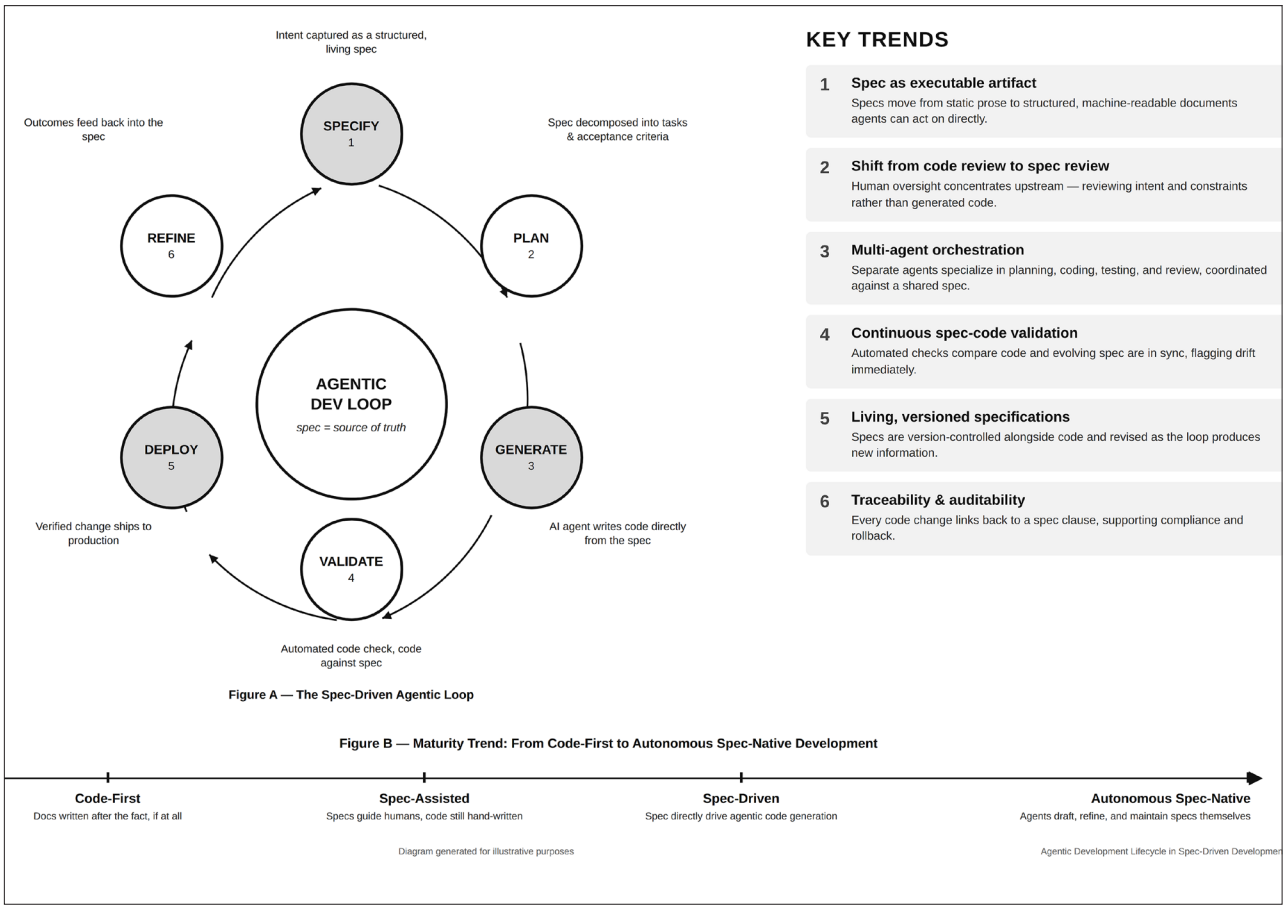


Figure 1: Trends in spec-driven development

Trends in spec-driven development in the agentic development lifecycle

The agentic development lifecycle is changing the centre of gravity in software engineering. Earlier generations of AI tools helped developers complete lines of code or generate small functions. Newer agentic systems can analyse requirements, inspect repositories, propose designs, generate tests, modify several files, run builds, explain failures, and prepare pull requests. Industry reports describe this as a movement from isolated coding assistance towards coordinated agent teams, long-running workflows, and human oversight at scale. As agents take on larger portions of execution, the need for a durable expression of intent becomes stronger. Spec-driven development is therefore emerging as a coordination layer between product thinking, architecture, engineering, testing, security, and operations.

One visible trend is the movement from chat-led development to artifact-led execution. Instead of relying on long conversational histories, teams are creating structured specifications, implementation plans, task lists, acceptance criteria, invariants, and testable contracts. These artifacts can be stored in version control, reviewed like code, and reused by multiple agents or developers. This matters especially in brownfield modernisation, where agents must respect existing architecture, hidden dependencies, security controls, and enterprise integration standards.

A second trend is the growth of multi-agent SDLC patterns. Requirement agents can transform business inputs into stories and acceptance criteria. Design agents can propose architecture alternatives. Build agents can implement bounded tasks. Test agents can generate unit, integration, regression, and security tests. Review agents can check coding standards, vulnerabilities, dependency risks, and architectural alignment. Release agents can assemble evidence for deployment readiness. In this model, the specification becomes the shared operating contract that aligns these agents and prevents each one from interpreting the goal differently.

A third trend is the rise of governed autonomy. Enterprises are beginning to distinguish between assisted coding, automated task execution, bounded agentic workflows, and trusted autonomous delivery. Spec-driven development supports this maturity journey because it makes intent,

Vibe coding vs spec-driven development

Dimension	Vibe coding	Spec-driven development
Primary intent	Rapid exploration and quick generation	Controlled implementation from defined intent
Source of truth	Prompt, chat history, and developer memory	Versioned specification, plan, tests, and acceptance criteria
Best fit	Prototype, demo, learning exercise, internal experiment	Production system, regulated workload, multi-team delivery, long-lived product
Risk profile	Higher risk of drift, hidden assumptions, technical debt, and weak traceability	Lower risk through explicit constraints, reviews, validation, and auditability
Human role	Prompt, observe, correct, and iterate	Define intent, review specification, govern decisions, verify outputs

scope, assumptions, approval gates, and verification evidence explicit. It also allows work to be decomposed into smaller, auditable tasks, which is essential when agents are allowed to act across repositories, pipelines, test environments, issue trackers, and cloud platforms.

Spec-driven development versus vibe coding

Vibe coding and spec-driven development are not merely two tool preferences; they represent two different philosophies of working with AI. Vibe coding is exploratory, conversational, and fast. A developer describes what they want, receives generated code, tests or observes the outcome, adjusts the prompt, and iterates. It is useful when the goal is discovery, prototyping, learning a new framework, creating a throwaway proof of concept, or testing whether an idea has value. It lowers the entry barrier and gives teams a quick way to move from thought to working sample.

Vibe coding appears weak when the work must survive beyond the prototype. Chat history is not a reliable architecture record. Generated code may work locally but miss edge cases, non-functional requirements, security controls, naming conventions, observability requirements, or integration contracts. Different prompts can produce different implementations for the same feature, and later contributors may not understand why specific choices were made. As the system grows, the early speed of vibe coding can be offset by rework, duplicated logic, inconsistent patterns, and fragile maintenance.

Spec-driven development takes a different route. It begins by making intent explicit before implementation begins. It asks teams to define the expected behaviour, constraints, data ownership, quality requirements, non-goals, compliance expectations, and verification approach. AI agents still accelerate implementation, but they operate inside a more carefully described boundary. The trade-off is clear: SDD requires more upfront thinking, but it improves maintainability, traceability, quality, and enterprise confidence. A practical enterprise strategy is not to reject vibe coding entirely. Teams can use vibe coding for early exploration, then convert the learning into a living specification before production-grade development begins.

Advantages of spec-driven development

The first advantage of SDD is clarity. A specification removes much of the guesswork that typically sits between a business idea and implementation. It defines the expected result in a way that product owners, architects, developers, testers, security teams, and AI agents can all interpret. This shared clarity reduces repeated clarification cycles and helps teams avoid building features that are technically impressive but misaligned with business intent.

The second advantage is traceability. In enterprise environments, teams must often prove why a change was made, which requirement it satisfies, what tests validate it, and who approved the release. SDD creates a natural chain from requirement to design, from design to task, from task to code, and from code to evidence. This is valuable not only for audits, but also for daily engineering discipline. When a defect appears, the team can examine whether the specification was incomplete, the implementation violated the specification, or the tests failed to cover the intended behaviour.

The third advantage is better governance of AI-generated code. AI coding agents are powerful but not inherently accountable. They can produce plausible code that compiles while still violating security principles, architectural standards, licensing expectations, privacy rules, or performance constraints. A strong specification acts as a guardrail. It states what the AI system may assume, what it must not change, what approval is required, what data may be accessed, and what evidence is needed before the output is accepted.

The fourth advantage is maintainability. A codebase generated from vague prompts may function in the short term, but future teams will struggle to understand its assumptions. SDD keeps the rationale visible. It supports onboarding, refactoring, modernisation, and impact analysis because the design intent is available

alongside the implementation. This is especially useful when enterprises use AI to modernise legacy platforms, upgrade libraries, migrate cloud services, or refactor monolithic systems.

Developing skills for SDD

Spec-driven development requires a different skill profile from traditional coding or casual AI prompting. Developers need engineering depth, but they also need the ability to express intent precisely. Product owners must learn to write acceptance criteria that are testable rather than aspirational. Architects must translate quality attributes into constraints that agents can respect. Testers must think in terms of executable evidence. Security specialists must define misuse cases, policy checks, and restricted actions early enough to influence implementation rather than waiting for end-stage review. Here is an example skill code snippet to understand this better:

```
---
name: spec-driven-development
description: >
  Use this skill whenever a user asks to
build, add, or change a feature
  in a codebase. Enforces a spec-first
workflow: write and confirm a
  structured spec before generating any
code, then validate generated
  code against that spec. Trigger on
requests like "build a feature",
  "implement X", "add support for Y", or
"refactor Z" in a code project.
---

# Spec-Driven Development

## Workflow

1. **Specify** — Before writing code,
draft a spec at
   `specs/<feature-slug>.md` with these
sections:
   - `## Intent` — one paragraph, the
"why"
   - `## Requirements` — numbered,
testable statements
   - `## Non-goals` — explicitly out of
scope
   - `## Acceptance Criteria` —
checklist, each mapped to a test

2. **Confirm** — Show the spec to the
user. Do not generate code until
   they approve it or explicitly say to
proceed.

3. **Plan** — Break the spec into an
ordered task list. Each task must
   cite the requirement number(s) it
satisfies.

4. **Generate** — Write code task-by-
task. Reference the spec file
   in comments where non-obvious (`#
satisfies REQ-3`).

5. **Validate** — After generating code,
re-read the spec and check
   each Acceptance Criterion explicitly.
Report PASS/FAIL per item —
   do not just say "looks good."

6. **Refine** — If a criterion fails or
the user requests a change,
   update the spec FIRST, then
regenerate the affected code. Never
   let code and spec drift silently.

## Rules
- Never skip straight to code generation
for non-trivial features.
- Never mark an acceptance criterion as
passed without checking it.
- Keep the spec file updated — it should
always describe current
  intended behavior, not just the
original request.
```

A useful SDD skill path begins with requirement articulation. Practitioners should learn to separate business goals, user journeys, functional behaviour, constraints, and assumptions. They should also become comfortable writing non-goals, because stating what the system will not do prevents agents from adding unnecessary features. The next skill is decomposition. A good specification must be broken into small, bounded tasks that can be implemented, reviewed, and tested independently. This reduces the risk of large AI-generated changes that are difficult to inspect.

Another important skill is evaluation design. Teams must define how they will know that the implementation is correct. This includes unit tests, integration tests, contract tests, performance checks, accessibility checks, security scans, policy validations, and manual review criteria. In agentic development, evaluation is not a final stage; it is the steering mechanism that keeps the agent aligned. Finally, practitioners need AI collaboration literacy: how to provide context, how to bound tasks, how to inspect generated diffs, how to challenge assumptions, how to preserve institutional knowledge, and how to decide when human engineering judgment must override AI output.

Governance model in spec-driven development

Governance in SDD should not be designed as a bureaucratic checkpoint that slows every team. It should function as a lightweight control system that makes safe delivery easier. A practical governance model begins with specification ownership. Every specification should have named business, architecture, engineering, quality, and security owners depending on the risk level of the work. Their role is not to write every detail, but to ensure that the specification is complete enough for reliable implementation and review.

The second governance layer is risk classification. Not every feature needs the same level of formality. A temporary prototype may need a short intent brief. A customer-facing financial transaction, healthcare workflow, identity service, or production infrastructure change requires a stronger specification with security, privacy, audit, rollback, and approval

expectations. Risk classification allows teams to scale governance according to consequence rather than treating all work identically.

The third layer is human-in-the-loop approval. AI agents may generate implementation plans, code, tests, and documentation, but high-impact decisions should remain under accountable human review. Approval gates may be required for architecture changes, data access, external API calls, security-sensitive code, deployment scripts, infrastructure changes, and irreversible business actions. The fourth layer is observability. Teams should retain evidence of specification versions, prompts or agent instructions, generated plans, test runs, tool calls, approvals, exceptions, and deployment outcomes. This evidence is essential for debugging, compliance, incident response, and continuous improvement.

Best practices for enterprise adoption

Enterprises should start with a pragmatic adoption model rather than a grand methodology rollout. The first step is to select use cases where specifications naturally create value: API development, platform modernisation, regulatory workflows, data pipeline changes, security remediation, test automation, release readiness, and multi-team integration. These scenarios benefit from clarity, traceability, and reusable patterns. Teams should avoid forcing SDD onto short-lived experiments where the cost of specification is greater than the value of the output.

Another best practice is to create reusable specification templates. Templates should include sections for problem statement, users, scope, non-goals, functional requirements, non-functional requirements, data considerations, security constraints, accessibility expectations, observability requirements, acceptance criteria, test strategy, rollout plan, and rollback approach. Templates prevent blank-page anxiety and help teams converge on a common language. They also make it easier for AI agents to consume specifications consistently.

A third best practice is to integrate SDD into existing DevSecOps workflows. Specifications should be stored with the code or linked to the work item system. Pull requests should reference the relevant specification and tests should map back to acceptance criteria. Security and compliance checks should be automated wherever possible. This keeps SDD close to delivery instead of turning it into a separate documentation exercise.

A fourth best practice is to use a hybrid operating pattern: vibe-explore, spec-produce, agent-implement, human-verify, and continuously-learn. Teams can use AI freely during discovery to understand options and prototype ideas. Once the direction is clear, they convert the learning into a living specification. Agents then implement bounded tasks, while humans review outputs and maintain accountability. After deployment, feedback from users, defects, incidents, cost data, and operational logs should update the specification so it remains a living artifact rather than a forgotten document.

Reusability and extensibility in SDD

Reusability is one of the most important enterprise advantages of spec-driven development. When specifications are treated as living engineering assets rather than one-time documents, they can become reusable patterns that improve consistency across products, teams, and AI-assisted delivery workflows. In a large organisation, many requirements repeat in different forms: authentication, authorisation, audit logging, error handling, API versioning, accessibility, observability, privacy controls, deployment approvals, and rollback expectations. SDD allows these recurring expectations to be captured once as reusable specification fragments, templates, checklists, policies, and test contracts that can be applied across multiple initiatives.

A mature SDD practice can maintain a specification library similar to a code library. This library may include reusable API specification templates, standard non-functional requirement patterns, domain-specific acceptance criteria, security requirement blocks, data classification rules, observability expectations, and release readiness checklists. For example, every customer-facing API may reuse a standard OpenAPI template that defines request validation, error response formats, authentication requirements, rate-limiting expectations, logging rules, and backward compatibility constraints. Similarly, all user interface features may reuse accessibility acceptance criteria aligned with WCAG expectations, while all production services may inherit common requirements for tracing, metrics, alerts, and incident response evidence.

Reusability becomes even more valuable in agentic AI development because agents need consistent instructions and boundaries. If every team writes requirements differently, AI coding agents may interpret similar problems in inconsistent ways. Reusable specifications reduce ambiguity by giving agents repeatable structures, standard vocabulary, known constraints, and validated examples. Requirement patterns, policy-as-code rules, test-as-contract definitions, and reference architectures help agents generate outputs that align with enterprise engineering standards. A reusable agent workflow may define how a build agent decomposes tasks, how a test agent maps acceptance criteria to test cases, how a review agent checks architectural compliance, and how a release agent assembles deployment evidence.

Extensibility is the companion principle to reusability. A reusable specification must not be rigid. It should evolve as products grow, regulations change, architectures modernise, and AI tools become more capable. Extensible SDD depends on modular specifications, version control, traceability, and structured change impact analysis. A modular specification separates stable intent from variable implementation details. For example, a payment workflow specification may keep the core business rules stable while allowing extensions for new payment providers, fraud checks, reporting obligations, or regional compliance requirements.

Versioning is essential for this evolution. Teams should know which version of a specification produced which implementation, which acceptance criteria were valid at the time of release, and what changed when a new requirement was introduced. Traceability links requirements to design decisions, code changes, tests, approvals, and operational evidence. This makes it possible to assess the impact of change before agents modify code.

Reusable and extensible SDD reduces duplication, improves governance, accelerates delivery, and helps enterprises scale AI-assisted software development without losing control over quality or intent.

Developing framework-agnostic SDD

Framework-agnostic SDD means designing specifications so that the expression of intent is independent of a specific programming language, software framework, cloud provider, AI coding assistant, issue tracker, or DevOps platform. The central idea is to separate what the system must achieve from how a particular team chooses to implement it. This distinction is critical in modern enterprises because the same business capability may be implemented across different technology stacks such as Java and Spring Boot, .NET, Python, Node.js, React, native mobile applications, microservices, data pipelines, Kubernetes workloads, serverless functions, or legacy platforms. If the specification is tightly coupled to one framework, it loses value when the implementation environment changes.

A framework-agnostic SDD approach begins with stable engineering artifacts. These include intent, scope, requirements, non-goals, assumptions, constraints, interfaces, data contracts, quality attributes, acceptance criteria, verification strategy, and governance gates. These elements should remain understandable even if the implementation moves from a monolith to microservices, from on-premises infrastructure to cloud-native deployment, or from one AI coding platform to another. For example, a requirement such as "the system must validate customer identity before initiating a funds transfer" is framework-agnostic. The implementation may differ across Java, .NET, or Python, but the business rule, verification expectation, audit requirement, and security constraint remain the same.

Modern technical standards make framework-agnostic SDD more practical. OpenAPI can describe REST interfaces independently of implementation language. AsyncAPI can define event-driven contracts across messaging systems. JSON Schema can formalise data structures. C4 model diagrams can describe architecture at different levels of abstraction. Architecture Decision Records can capture important design choices without embedding them only in code. Policy-as-code tools can express security, compliance, and infrastructure rules in a repeatable way. Test contracts can define expected behaviour that must pass regardless of the underlying stack. CI/CD evidence can then prove that the implementation satisfies the specification in a specific environment.

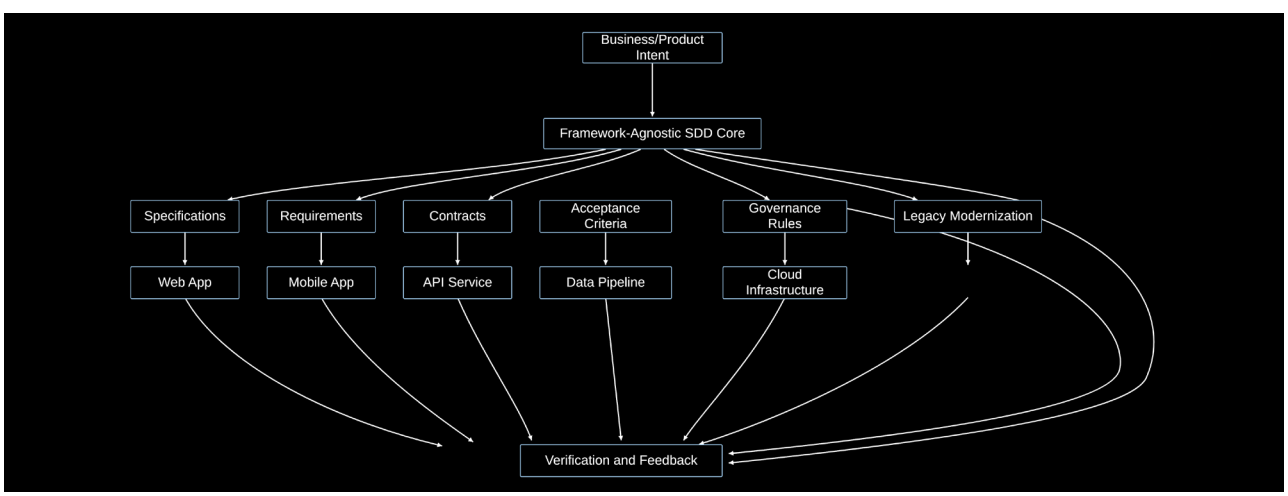


Figure 2: Developing framework-agnostic SDD

This approach is also important for agentic AI development. Different teams may use different coding assistants or autonomous engineering agents, but those agents should consume the same intent. A framework-agnostic specification provides model-context-friendly documentation that can be passed to requirement agents, design agents, build agents, test agents, and review agents without locking the organisation into one vendor or toolchain. It also reduces the risk that an AI agent will overfit the solution to a preferred framework rather than the real business requirement.

Framework-agnostic SDD improves long-term maintainability and modernisation. When specifications are independent of implementation choices, teams can migrate systems, replace frameworks, refactor services, or modernise legacy platforms with better confidence. The specification remains the stable contract, while the implementation can evolve. This is especially valuable for regulated workloads and multi-team delivery because compliance obligations, acceptance criteria, and approval gates remain traceable even when the technology landscape changes. In the agentic AI era, framework-agnostic SDD helps organisations preserve intent, enable portability, and maintain governance across diverse and continuously evolving engineering ecosystems.

Spec-driven development does not remove creativity, speed, or experimentation from development. Instead, it channels them through a clearer expression of intent. Vibe coding will continue to be valuable for discovery and rapid prototyping, but enterprise delivery needs more than an impressive first draft. It needs shared understanding, reviewable decisions, testable outcomes, secure implementation, and accountable governance. SDD provides that foundation. END

References

- *https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html*
- *https://www.ibm.com/think/topics/spec-driven-development*
- *https://www.augmentcode.com/guides/what-is-spec-driven-development*
- *https://www.archi-lab.io/infopages/cex/ss26/cex26-01-spec-driven-dev.html*
- *https://piccalil.li/blog/framework-agnostic-design-systems-part-1/*
- *https://www.thebcms.com/blog/spec-driven-development/*

By: Dr Magesh Kasthuri and Dr Anand Nayyar

Dr Magesh Kasthuri is a PhD in artificial intelligence and the genetic algorithm. He currently works as a distinguished member of the technical staff (master) and chief architect at Wipro Ltd. This article expresses his views and not of the organisation he works in.

Dr Anand Nayyar is a PhD in wireless sensor networks and swarm intelligence. He works at Duy Tan University, Vietnam, and loves to explore open source technologies, IoT, cloud computing, deep learning, and cybersecurity.

OSFY Magazine Attractions During 2026-27

Month	Theme
March 2026	Gen AI, Deep Learning and Machine Learning
April 2026	Open Source Cloud & Infrastructure
May 2026	Blockchain and Open Source
June 2026	Modern Software Development with Open Source | Programming Languages
July 2026	Observability & Performance | Security & Privacy in Open Source
August 2026	All About Data Management (Database management, Optimisation, Big Data, Data analysis, etc)
September 2026	DevOps, SRE & Platform Engineering
October 2026	Open Source and Quantum Computing
November 2026	Open Source for Edge, IoT & Embedded
December 2026	Cloud Special: Everything from Management to Implementation
January 2027	Python, Data Engineering and Open Data Stacks
February 2027	Open Source Communities & Careers | The Future Of Open Source

The Internet and AI: Multipliers of National Progress

The next global superpowers may be defined less by oil, geography, or military strength, and more by intelligence, connectivity, and speed. In earlier industrial eras, economic strength depended heavily on physical infrastructure. The new economy increasingly depends on software, semiconductors, algorithms, artificial intelligence, and digital reach. This transition creates a historic opportunity for India. But technology waves reward speed, and once ecosystems mature elsewhere, catching up becomes significantly harder.



Education builds capability. Innovation converts capability into value. The internet and AI multiply both at an unprecedented scale. This is where the pace of national progress accelerates.

For most of human history, progress moved slowly. Distance limited collaboration. Infrastructure limited access. Knowledge spread gradually. Entire generations were needed to transform industries and economies. That cycle is collapsing. The internet removed distance. AI is removing effort. Together, they are compressing decades of progress into years. They are not optional upgrades. They are force multipliers.

When years collapse into weeks

A student in a small town now has access to capabilities once limited to elite institutions. A startup with five people can build products that once required hundreds or even thousands of engineers.

Designers can prototype faster. Programmers can write and debug software more quickly. Researchers can analyse enormous datasets in minutes. What once took years can now happen in weeks. This is not an incremental improvement. It is a change in the speed of progress itself. The countries that understand this shift early will move ahead rapidly. Those who move slowly will struggle to keep pace.

India entered previous industrial revolutions late, while manufacturing leadership, large-scale R&D ecosystems, and deep technology ownership emerged elsewhere. The internet and AI present another historic opening, and India cannot afford to miss it.

The internet became infrastructure

The internet is no longer merely a communication tool. It has become infrastructure. In the industrial age, governments built roads, dams, and power lines. In the digital age, connectivity is the new lifeline.

Commerce, education, healthcare, banking, logistics, governance, manufacturing, and entertainment now depend on digital connectivity. Once connectivity reaches a national scale, opportunity scales with it. India has already demonstrated this capability.

UPI transformed digital payments. Aadhaar enabled identity at the population scale. CoWIN coordinated vaccination delivery for billions of doses. Pradhan Mantri Bharatiya Janaushadhi Pariyojana (PMBJP), or the Prime Minister's Indian Public Medicine Scheme, is providing quality generic medicines at affordable prices across India at massive scale. India Stack created digital infrastructure that enabled large-scale innovation across sectors. Scale changes economics.

A capable individual no longer needs to live in a major city to participate in the economy. Small businesses can access national markets. Students can learn from anywhere. Services can reach remote regions instantly. Connectivity changes access. Access changes outcomes.

AI: The new electricity

Artificial intelligence is not just another technology wave. It is becoming as foundational as electricity itself. Just as

electricity transformed every industry in the twentieth century, AI will transform every sector in the twenty-first.

AI can assist in coding, design, language translation, research, diagnostics, manufacturing optimisation, legal analysis, and scientific discovery. It reduces repetitive work and accelerates exploration. Importantly, AI not only helps experts. It lowers entry barriers. A student with curiosity, internet access, and AI tools can now perform tasks that previously required large teams, expensive infrastructure, or years of specialised training. This changes who gets to participate in building the future.

Throughout history, advanced capability remained concentrated within a small number of people, institutions, corporations, universities, or countries. The internet and AI are redistributing parts of that capability to individuals at a global level.

Small teams. Massive impact

The modern economy is entering an era where small teams can create disproportionate impact. An AI-driven company can now compete with much larger organisations. A creator can directly reach millions. A startup can serve global markets from day one. This changes the economics of growth. This creates a historic opportunity for India.

With a young population, strong software capability, improving digital infrastructure, and widespread familiarity with English, India is positioned to benefit from this transition on an unprecedented scale. The advantage is no longer just low-cost talent, but scale combined with intelligence.

The first time India can leap ahead

Many nations built their strength during earlier industrial revolutions. India entered several of those transitions late. The internet and AI era is different. The cost of participation has fallen sharply. Knowledge is no longer restricted to elite institutions. Powerful software tools are globally accessible. Distribution is digital. Learning is continuous.

For the first time in modern history, India has a realistic opportunity not just to catch up but to leap ahead. But technology waves reward speed. Once ecosystems mature elsewhere, catching up becomes significantly harder. Hesitation becomes expensive. Time itself becomes a competitive advantage.

The risks are real. So is the opportunity

Powerful technologies also introduce real risks. Misinformation can spread rapidly. Privacy can weaken. AI systems can inherit bias. Repetitive jobs may face disruption. Excessive dependence on external platforms can create strategic vulnerability. These concerns are real.

But history shows that societies progress not by avoiding technology but by learning to govern and use it wisely.

India must build its own talent, infrastructure, AI ecosystems, platforms, semiconductor capability, and research systems at scale. Nations that build capability control their future. Nations that depend entirely on external systems eventually lose strategic flexibility. AI ultimately runs on compute, chips, data centres, and electricity. In the AI era, energy policy increasingly becomes technology policy.

When the internet and AI multiply everything

The internet and AI do not replace education or innovation. They amplify both. Education becomes more accessible, personalised, and scalable. Innovation cycles become faster. Collaboration becomes easier. Ideas move from concept to execution with unprecedented speed.

Capability compounds faster in connected systems. A strong student becomes dramatically more capable with AI tools. A strong innovator reaches markets faster through digital infrastructure. A strong ecosystem compounds continuously. Education builds capability. Innovation converts capability into value. The internet and AI multiply both.

The execution imperative

India's potential is no longer the question; the challenge is now execution at the national level. To translate ambition into reality, India must move beyond slogans towards a deliberate national strategy.

- ***Domestic infrastructure:*** India must build its own talent, AI ecosystems, platforms, and semiconductor capabilities at scale.
- ***Strategic autonomy:*** Nations that depend entirely on external systems eventually lose flexibility; India must prioritise internal research systems to control its future.
- ***Energy policy:*** Because AI runs on compute and data centres, energy policy must evolve into a core pillar of technology policy.

In the AI era, speed becomes power

The next global leaders will not necessarily be the countries with the oldest institutions or the largest landmass. Increasingly, they will be the countries that learn, adapt, build, and scale the fastest. The internet and AI are changing the speed limit of progress itself. India already has the population, talent, and digital foundation required to participate at the highest level. The challenge now is execution at the national level.

Without a serious national strategy for education and innovation, economic ambition remains limited to speeches and slogans. The opportunity is historic. Execution will determine whether India leads the future or spends decades trying to catch up. END

By: Janardhana Swamy

The author is is an electronics engineer, inventor with patents in India and the USA, and a former Member of Parliament from Chitradurga. This article draws on his rare blend of technical depth, global experience, and lifelong passion for India's leadership.

This article was originally published in the June 2026 issue of Electronics For You.

FOCUS

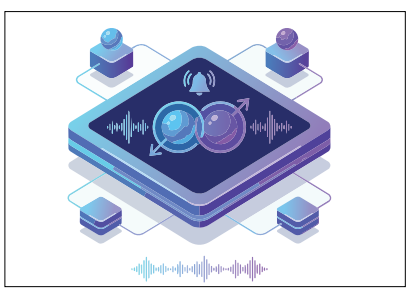
Cracking the Quantum Code

Quantum Computing: Architecture, Access, and Open Source Tools



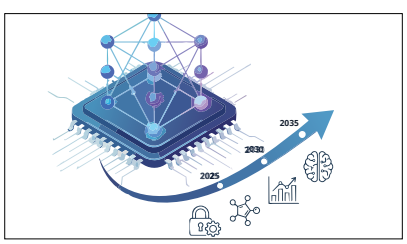
Thinking of investing in quantum computing? Begin by training a couple of engineers on it at zero cost with open source tools. That's all you may need to do right now.. *Read more on page.....36*

Quantum Computing: Ringing the Bell States



In this fifth article in the series on quantum computing, we take a closer look at entanglement, meet the four Bell states, and examine the many ways in which a real superconducting quantum computer can go wrong.
Read more on page.....40

The Road Ahead: Quantum Computing in the Next Decade



The emergence of quantum computing is bound to be disruptive, but organisations that learn to complement it with existing digital infrastructure will benefit. *Read more on page.....51*

Quantum Cryptography: Ensuring Secure Digital Communication

The digital world is entering an era in which the security of information can no longer depend solely on traditional cryptographic systems. Quantum cryptography is poised to become a significant component of future digital communication.
Read more on page.....49

NVIDIA CUDA-Q: Accelerating the Development of Quantum Computing

Nvidia is among the front-runners in quantum computing…its open source platform NVIDIA CUDA-Q has a lot to offer in this field.
Read more on page.....56

PennyLane: Building a Bridge Between Machine Learning and Qubits

Quantum machine learning can appear intimidating because it combines two complex subjects. PennyLane lowers that barrier by allowing developers to write a quantum circuit as a trainable component inside a familiar Python machine-learning workflow.
Read more on page.....59

Quantum Computing: Architecture, Access, and Open Source Tools



Thinking of investing in quantum computing? Begin by training a couple of engineers on it at zero cost with open source tools. That's all you may need to do right now.

A client asked me in June whether they should buy a quantum computer. Genuine question, asked seriously, by a company that makes agricultural equipment. Someone on their board had read an article.

I told them 'No'. And then spent the next hour on the more useful question: What does quantum computing look like from a developer's chair in 2026? That conversation is in this article. Not physics. There are better people than me for physics, and honestly, physics is not what stops teams from getting started. What stops them is that the field is buried under two layers of noise. Vendor marketing on one side, promising commercial advantage that has not arrived. Academic material on the other, written for people who already know what a Hamiltonian is. Very little in between for a working engineer who wants to know how quantum computing is accessed, what it costs, and which tools are worth learning.

So: architecture, access, tooling. In that order.

What you are connecting to

Let's start with the physical reality, because it explains every constraint that follows.

A gate-model quantum processor is a small chip that must be kept at roughly fifteen millikelvin, colder than deep space, inside a dilution refrigerator. If you have seen the photographs of a gold chandelier hanging in a lab, that is the refrigerator, not the computer. The actual processor is a fingernail-sized square at the bottom.

Superconducting machines from IBM, Rigetti and IQM work this way. Trapped-ion systems from IonQ, Quantinuum and AQT hold charged atoms in electromagnetic fields inside a vacuum chamber and manipulate them with lasers, which runs warmer but is no less fussy. Neutral-atom machines from QuEra and Pasqal arrange atoms in optical tweezers. Photonic systems from Xanadu and Quandela use light.

Every one of these needs a controlled room, specialist staff and constant recalibration. Machines drift. A processor that behaved one way this morning behaves slightly differently in the afternoon, which is why providers publish calibration data per device and update it daily.

That is why the cloud is not one delivery option among several for quantum computing. It is essentially the only one. Nobody is putting a dilution refrigerator in their office in Pune, and the handful of organisations buying on-premises systems are national labs and a few very large banks doing long-horizon research.

The architecture is a queue, not a socket

Here is the part that surprises people coming from ordinary cloud work.

You do not open a connection to a QPU and issue instructions interactively. You submit a job. The job goes into a queue. At some point, minutes or hours later depending on demand, your circuit is loaded onto the hardware, executed a few thousand times, and the aggregated measurement results come back.

The path looks roughly like this. You write a circuit in Python on your laptop. A transpiler rewrites it to fit the target device, mapping your abstract qubits onto physical ones and decomposing your gates into the specific gate set that machine supports. The rewritten circuit is serialised, usually into OpenQASM, and posted to the provider's API. It waits in a queue. Control electronics eventually translates it into microwave or laser pulses. The results, which are just counts of measured bit strings, travel back.

This has two consequences.

Latency is not a network problem. Your round trip is dominated by queue time, which no amount of colocation fixes. If somebody tells you they need a QPU in a specific region for latency reasons, they have misunderstood the model.

And because results are statistical, you run the same circuit repeatedly. A thousand shots, ten thousand, more. Each run is a sample from a probability distribution, and you need enough samples to see the shape of it. This is why quantum jobs are billed per shot or per task rather than per second.

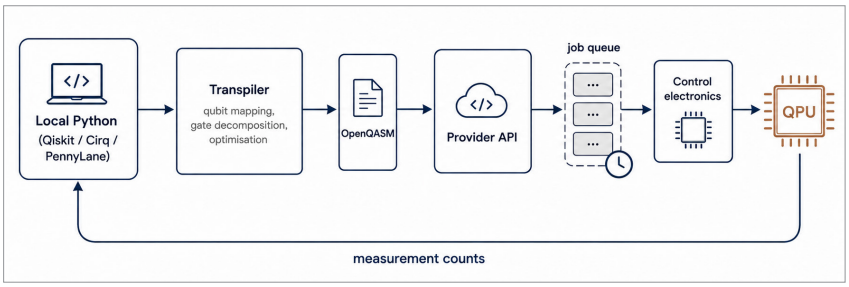


Figure 1: Measurement counts

Where the noise comes from

Skip this section if you already know it, but most explanations bury it and it matters for everything practical.

Qubits lose their quantum state. Fast. Coherence times on current hardware sit in the range of tens to hundreds of microseconds depending on the technology, and every gate operation you apply has an error rate attached, typically somewhere between one in a hundred and one in a thousand for two-qubit gates on good machines.

Errors compound. A circuit with a few hundred gates will accumulate enough errors that the output becomes indistinguishable from noise. That is the real limit on what you can run today, and it is why raw qubit counts are a poor way to compare machines. A five hundred qubit processor with mediocre gate fidelity may be less useful than a twenty qubit processor with excellent fidelity, because circuit depth is what you actually spend.

This is the NISQ (noisy intermediate-scale quantum) era, and the honest summary is that we are still in it. Error correction is progressing and the published roadmaps point at fault-tolerant machines around the end of this decade, but roadmaps are roadmaps. Plan for what exists.

Which brings me to the thing I say to every client who asks about this.

There is no verified case of a quantum computer beating a good classical algorithm on a real business problem today. Not one that has survived scrutiny. Several claimed results were later matched or beaten by improved classical methods running on ordinary hardware. If a vendor presents you with a quantum advantage claim for your industry, ask what the best classical approach achieves on the same problem, and ask who checked.

This is not a reason to ignore the field. It is a reason to be precise about why you are engaging with it, which for almost everybody right now is capability building rather than production work.

Four ways of access

Access is consolidated into a small number of routes, and they split cleanly by business model.

IBM Quantum builds its own superconducting hardware and offers it directly. Open access to real devices, deep free tutorial material, and Qiskit as the native SDK. If you want a first account, this is where I send people, because the learning path is the most complete.

Google Quantum AI also builds its own hardware, around superconducting processors, but access is oriented towards research partners rather than general developers. Cirq is the SDK.

Amazon Braket is an aggregator. AWS builds little general-purpose QPU hardware itself and resells a marketplace of partner machines, currently spanning superconducting devices from IQM and Rigetti, trapped-ion systems from IonQ and AQT, and neutral-atom hardware from QuEra. If you already live in AWS, the IAM and S3 integration will feel familiar, and Braket Hybrid Jobs handles the classical-quantum loop.

Azure Quantum is the other aggregator, with IonQ, Quantinuum, Rigetti and Atom Computing among its backends, plus Microsoft's own Q# and the QDK. Sensible if you are already an Azure shop.

Then there are the independents worth knowing. qBraid and Strangeworks sit above the hyperscalers and give you one interface across several of them. D-Wave Leap is a different animal entirely and I will come back to it.

There is no Indian public cloud offering QPU access at the time of writing, though there is domestic hardware work underway under the National Quantum Mission. For now, expect your jobs to execute on machines in the US or Europe. If your organisation has data residency rules, raise this early with legal rather than late, because it has stopped more than one pilot I have been involved with.

Annealing is not the same thing

This is worth being blunt about, because the confusion is common and expensive.

D-Wave's machines are quantum annealers. They do not run gate-model circuits. They find low-energy configurations of a problem you have expressed as a QUBO or Ising model, which makes them a fit for certain optimisation shapes and useless for Shor's algorithm or quantum chemistry.

The practical difference for a developer: you write completely different code, using the Ocean SDK rather than Qiskit, and you cannot port between them. D-Wave has offered free metered access for years and their machines have far more qubits than any gate-model device, which is exactly why the qubit-count comparison misleads people. Different technology, different problem class.

If your problem is genuinely combinatorial optimisation, an annealer is worth an afternoon. Spend some time on running the same problem through a good classical solver like OR-Tools first, so you have something to compare against.

The open source layer

This is where the field is healthiest, and it is genuinely the most encouraging part of the whole picture. Almost every serious tool is open source and permissively licensed.

Qiskit is the centre of gravity. Apache 2.0, Python interface over a Rust core, and since version 2.0 there is a C API as well for people writing extensions or embedding it. Version 2.3 landed in January this year. The programming model is built around circuits, operators and two primitives: Sampler, which gives you measurement outcomes, and Estimator, which gives you expectation values of observables. Most tutorials you find will use one or the other. It also carries the most capable transpiler in the ecosystem, which matters more than beginners expect.

Cirq is Google's framework, Apache 2.0, built with superconducting device constraints close to the surface. Strong for research and algorithm prototyping.

PennyLane from Xanadu takes a different angle. It treats a quantum circuit as a differentiable function, so you can compute gradients through it and plug the whole thing into PyTorch or TensorFlow. If your interest is quantum machine learning or variational algorithms, start here rather than with Qiskit.

pytket from Quantinuum is a compiler-first toolkit with a reputation for aggressive circuit optimisation. If you are trying to squeeze a circuit under the depth limit of a real device, it is worth benchmarking against Qiskit's transpiler.

Ocean is D-Wave's suite for annealing work. **CUDA-Q** from Nvidia targets GPU-accelerated simulation and hybrid workloads. **Qibo** covers simulation and self-hosted hardware control. **Perceval** is Quandela's photonic framework.

And **OpenQASM** works underneath all of it as the interchange format, which is why cross-framework transpilation is possible at all.

A word on simulators, since this is where you will spend most of your time. Classical simulation of a quantum circuit costs memory that doubles with each qubit added. Around thirty qubits is comfortable on a decent workstation. Beyond forty you need serious hardware. Past roughly fifty, full state-

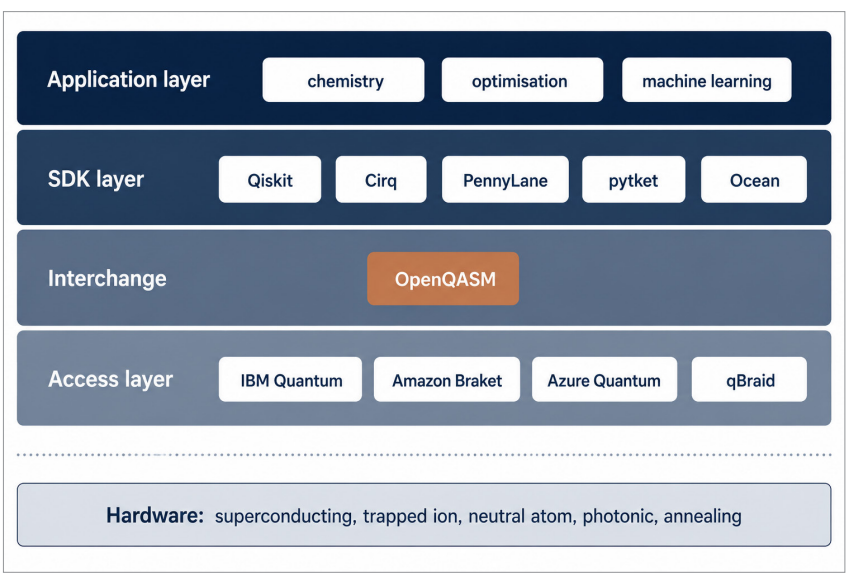


Figure 2: The architecture

vector simulation becomes impractical on anything. That ceiling is precisely why quantum hardware is interesting, and in the meantime, it means your development loop runs locally and free.

Actually running something

Enough of architecture. Here is the shortest useful path to run a quantum program.

Install Qiskit and the Aer simulator, build a Bell state, and run it locally. Two entangled qubits, the traditional first program.

```
python
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit import transpile

qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])

sim = AerSimulator()
result = sim.run(transpile(qc, sim),
shots=1024).result()
print(result.get_counts())
```

You should see roughly 500 counts of 00 and 500 of 11, with nothing in between. That absence of 01 and 10 is the entanglement. Run it on real hardware later and you will see a few hundred stray counts in those buckets, which is the noise made visible.

Do not start on real hardware. I mean that as practical advice rather than gatekeeping. Simulator first, always, because a circuit that is wrong on a simulator will be wrong and slow, and possibly billed on a QPU.

When you are ready for hardware, the pattern is the same across providers: authenticate, pick a backend, transpile against that backend's real coupling map and gate set, submit, wait, collect. The transpile step against the actual device is the part people skip and then wonder why their results are garbage.

On cost, the model varies but the shape is consistent. Braket charges per task plus per shot. IBM offers open access with monthly time limits on real devices and paid plans beyond that. D-Wave gives free metered time. Simulators are cheap or free everywhere.

The failure mode to avoid is a loop that submits jobs to hardware while you debug. Rate limits and budget alarms exist for the same reason they exist in any cloud service, and quantum jobs are billed in units that are unfamiliar enough that people misjudge them. Set a spending cap before your first hardware submission, not after.

What to actually do with quantum computing

My honest read, and clients do not always enjoy hearing it.

For the overwhelming majority of organisations, the correct engagement level right now is one or two engineers spending a few hours a month learning, plus standing watch on developments in your specific industry.

The exceptions are real but narrow — pharmaceutical and materials work, where molecular simulation is the most credible near-term application, and certain logistics and portfolio optimisation problems, though check the classical baseline first and expect to be disappointed. Cryptography, which is a different conversation entirely and the one that actually has a deadline.

That last one deserves a sentence on its own. Post-quantum cryptography is not speculative. NIST standards are published, migration guidance exists, and the harvest-now-decrypt-later risk means data you transmit today with classical encryption may be readable later. If your organisation handles anything with a long confidentiality horizon, post-quantum cryptography migration is a present-tense infrastructure project, and it has nothing to do with whether you ever run a quantum circuit.

I find that distinction gets lost constantly. Quantum computing is a research capability you are watching. Post-quantum cryptography is work you should have already started.

Where this leaves us

The agricultural equipment company did not buy a quantum computer. Two of their engineers now have IBM Quantum accounts and one of them has become genuinely good at this, which is a far better outcome for roughly zero rupees.

That is the shape of the opportunity in 2026. The hardware is remarkable, the results are not yet commercially useful, and the barrier to learning has collapsed to nearly nothing because the entire software layer is open source and the simulators run on your laptop. You can spend a weekend getting properly oriented for free.

The organisations that will benefit when the hardware matures are not the ones buying access now. They are the ones that will have people who understand the problem shapes, know what a variational algorithm is, and can tell a real advantage claim from a marketing one. That is a training question, and training takes years, which is the honest argument for starting now rather than waiting.

Install Qiskit. Build the Bell state. Read the calibration data for a real device and sit with what the error rates mean. That is a genuinely useful afternoon, and it costs you nothing. END

By: Vishal Tanna

The author is the CEO of TechCrave, a technology services company that helps enterprises adopt open source, cloud native, and AI-driven solutions. He has over 15 years of experience in software development and technology leadership, and is particularly interested in scalable architectures and digital transformation.

Quantum Computing: Ringing the Bell States

In this fifth article in the series on quantum computing, we take a closer look at entanglement, meet the four Bell states, and examine the many ways in which a real superconducting quantum computer can go wrong.



In the previous article in this series (*OSFY*, September 2026), we finally left the comfortable world of the single qubit behind. We met the Controlled-NOT (CNOT) gate, our first multi-qubit gate, and used it to create a pair of entangled qubits. We also learned how to make Qiskit's simulator misbehave on purpose by attaching a noise model to it, and we executed a quantum program on an actual superconducting quantum computer through the IBM Quantum Platform. At the end of that article, three promises were made: to look more deeply into entanglement, to introduce the Bell states, and to take a closer look at the various sources of error that affect today's superconducting quantum computers. This article keeps all three promises.

The central theme of this article is quantum entanglement and, in particular, a family of four two-qubit states known as the Bell states. If superposition is what makes a single qubit interesting, entanglement is what makes a collection of qubits powerful. Erwin Schrödinger, who coined the term entanglement in 1935, went considerably further than that. He described it not as one interesting feature of quantum mechanics among many, but as the one feature that forces quantum mechanics to depart completely from classical ways of thinking. In the previous article we produced an entangled pair almost casually, with two lines of Qiskit code. In this article we shall find out what we actually produced.

As in the previous articles, we shall remain faithful to the three-level approach of this series and explore entanglement through code, mathematics and hardware. But before meeting the Bell states, let us honour a small tradition of this series: we shall first send our code to a real quantum computer.

Entanglement on real hardware

In the previous article, we executed the program *qiskit8.py*, a single-qubit Pauli-X circuit, on the IBM quantum processor *ibm_marrakesh*. That circuit involved just one qubit and one gate, and it was about as gentle a request as one can make of a quantum computer. Let us now raise the stakes. The program *qiskit10.py*, shown below, takes the two-qubit entangling circuit of *qiskit9.py* from the previous article—a Hadamard gate followed by a CNOT gate—and executes it on real superconducting hardware instead of the *AerSimulator*.

```
1.  from qiskit import QuantumCircuit, transpile
2.  from qiskit_ibm_runtime import QiskitRuntimeService,
    SamplerV2 as Sampler

3.  API_KEY = "PasteYourAPIKeyHere"

4.  QiskitRuntimeService.save_account(
      channel="ibm_quantum_platform",
      token=API_KEY,
      set_as_default=True,
      overwrite=True)
5.  service = QiskitRuntimeService( )
6.  backend = service.least_busy(operational=True,
```

```
     simulator=False)
7.  print(f"Running on backend: {backend.name}")

8.  qc = QuantumCircuit(2, 2)
9.  qc.h(0)
10. qc.cx(0, 1)
11. qc.measure([0, 1], [0, 1])

12. transpiled_circuit = transpile(qc, backend)

13. sampler = Sampler(mode=backend)
14. job = sampler.run([transpiled_circuit], shots=1000)
15. print("Job ID:", job.job_id( ))

16. result = job.result( )
17. counts = result[0].data.c.get_counts( )
18. print(counts)
```

The program *qiskit10.py* is so close to *qiskit8.py* that a line-by-line explanation would waste space. Only three things are worth pointing out. First, the quantum part of the program—the four lines (8-11) that create two qubits, apply a Hadamard gate, apply a CNOT gate and measure both qubits—is character for character identical to *qiskit9.py*. Not a single line had to be changed to move from a simulator to a machine cooled to 15 millikelvin. That is a remarkable achievement of the Qiskit software stack, and it is worth pausing to admire it. Second, instead of the preset pass manager used in *qiskit8.py*, we simply call *transpile( )* with the chosen backend as an argument. Both rewrite our circuit so that it uses only the operations the target processor physically supports—*transpile( )* builds a preset pass manager internally—though it optimises a little harder, defaulting to level 2 where *qiskit8.py* asked for level 1. Third, *save_account( )* writes the credentials to the file *~/.qiskit/qiskit-ibm.json*, and *set_as_default=True* marks this account as the one to use when more than one has been saved. This is a one-time step: later programs can create a *QiskitRuntimeService( )* object with no arguments and simply find the account.

A word of caution. Your API key is a password, not a decorative string. Never publish it, never commit it to a public repository, and never leave it in a screenshot. If one is ever exposed, revoke it on the IBM Quantum Platform and generate a new one.

Before executing the program, activate the virtual environment with the command *conda activate qiskit_env* and make sure that the IBM Runtime package is installed, using *pip install qiskit-ibm-runtime*, exactly as we did in the previous article. Then replace the placeholder *"PasteYourAPIKeyHere"* with your own API key and run the command *python qiskit10.py*. The output obtained is shown in Figure 1.

```
Using instance: open-instance, plan: open
Running on backend: ibm_kingston
Job ID: d9s37copdb6s73e5im4g
{'11': 461, '00': 526, '01': 5, '10': 8}
```

Figure 1: Terminal output obtained after executing *qiskit10.py* on an IBM quantum processor

This time the least busy processor turned out to be *ibm_kingston*, a 156-qubit machine belonging to IBM's Heron R2 family. The measurement statistics returned by the hardware are *{'11': 461, '00': 526, '01': 5, '10': 8}*. Compare this with the ideal simulator output of the previous article, which contained nothing but 00 and 11. Here, 987 of the 1000 shots behaved themselves and produced the perfectly correlated outcomes we expect from the entangled state $(|00\rangle+|11\rangle)/\sqrt{2}$. The remaining 13 shots produced 01 or 10—outcomes that a perfectly entangled pair is simply not allowed to produce. Those 13 stray counts are the most interesting numbers in this article. On an ideal simulator, entanglement is a promise that is always kept; in this particular run, it was kept 98.7 per cent of the time. Let us now find out who broke the promise.

Where do the errors come from?

In the previous article we introduced the term Noisy Intermediate-Scale Quantum (NISQ) and noted that today's processors are noisy. That is a little like saying that the monsoon is wet: true, but not very informative. A superconducting quantum computer fails in several distinct ways, each with its own physical origin, its own characteristic magnitude, and its own set of countermeasures. Let us look at the most important ones.

Energy relaxation, usually denoted T1, is the tendency of an excited qubit to give up its energy to the environment and fall back from $|1\rangle$ to $|0\rangle$. A qubit does not stay excited out of politeness; it stays excited only until the surrounding world persuades it otherwise. Both processors we have used—*ibm_marrakesh* in the previous article and *ibm_kingston* in this one—belong to IBM's Heron family, where T1 is typically 100 to 300 microseconds, though the figure varies from qubit to qubit and from one calibration to the next. Notice that this error is not symmetric: a qubit decays from $|1\rangle$ to $|0\rangle$, but essentially never the other way round, because the refrigerator is far colder than the energy gap between the two levels.

Dephasing, denoted T2, is more subtle and, for readers of the third article in this series, more interesting. Here the qubit does not lose its energy; it loses its phase. Recall from the third article that a superposition carries a relative phase between $|0\rangle$ and $|1\rangle$, and that this phase, though invisible to a direct measurement, decides the outcome of every subsequent operation. Random fluctuations in the qubit's frequency cause that phase to drift

unpredictably, and once it has been randomised, the superposition has quietly degraded into an ordinary classical probability distribution. T2 is also typically in the range of 100 to 300 microseconds on Heron-class devices. Since a single-qubit gate takes only a few tens of nanoseconds, we have room for a few thousand operations before decoherence destroys the computation—which sounds generous until you recall that a useful quantum algorithm may need millions of operations.

Gate errors arise because the microwave pulses that implement our gates are not perfect. The pulse may be slightly too long, slightly off frequency, or slightly the wrong shape, so the state vector does not rotate through exactly the intended angle. On present-day Heron processors, single-qubit gates are extraordinarily good, with error rates of the order of a few parts in ten thousand. Two-qubit gates are roughly ten times worse, with error rates of a few parts in a thousand. That ten-to-one ratio shapes the whole of practical quantum programming. In the NISQ era, the number of two-qubit gates is the currency in which we pay for our algorithms, and circuit designers count them the way a jeweller counts grams. Our Bell circuit spends exactly one, which is why it came back so nearly intact.

Readout errors, also called measurement errors, occur at the very last step. The state of a superconducting qubit is read by bouncing a microwave signal off a resonator coupled to it and inspecting how the signal comes back. Occasionally the classification is simply wrong, and a qubit in $|0\rangle$ is reported as 1. Readout is usually the noisiest single operation on the chip, with error rates around one percent or a little below. Together with imperfect initialisation, these are known as SPAM errors—state preparation and measurement errors.

Crosstalk is the failure of qubits to mind their own business. Driving one qubit can disturb its neighbours, and two qubits that are physically adjacent can retain a small unwanted interaction even when we are not using it. This is precisely the problem that the tunable couplers of the Heron architecture were designed to solve: when a coupler is not needed, it is detuned so that the residual interaction is suppressed.

Leakage is a reminder that our tidy two-level picture is a convenient fiction. A transmon qubit (a type of superconducting charge qubit) is really an oscillator with many energy levels, of which we have borrowed the lowest two and named them $|0\rangle$ and $|1\rangle$. A badly shaped pulse can push the device into the second excited state, outside the computational space altogether. Such a qubit has not merely made an error—it has left the game.

Finally, there are the material and environmental villains. Microscopic defects in the oxide layers of the chip, known as two-level systems or TLS defects, drift in and out of resonance with the qubits and steal energy from them; the Heron R2 generation includes dedicated TLS mitigation for exactly this reason. And, spectacularly, cosmic rays occasionally strike the substrate and produce bursts of correlated errors across many qubits at once. It is a humbling thought that a quantum processor in a laboratory in the United States can be disturbed by a particle that began its journey outside the solar system

At this point, it is worth revealing something that the three-level approach of this series obliges us to confront. The CNOT gate we wrote in *qiskit10.py* does not exist on the chip. The Heron processor implements only a small set of native operations: the single-qubit gates X, SX (the square root of X) and RZ, and the two-qubit CZ (controlled-Z) gate. When *transpile( )* prepares our circuit for the backend, our elegant Hadamard-plus-CNOT pair is rewritten as a sequence of RZ and SX rotations surrounding a single CZ gate. Try adding the statement *print(transpiled_circuit.draw( ))* to the program and see for yourself. The result is shown in Figure 2, and every gate in it is drawn from that short native list. Every physical error we have just discussed attaches itself to those operations, not to the idealised gates we wrote. Figure 2 also shows something the drawing of the original circuit could not: the labels *q_0 -> 22* and *q_1 -> 23* record which physical qubits on the 156-qubit chip the transpiler chose for us. Here we were fortunate, because qubits 22 and 23 are already neighbours on the lattice. Had they not been, the transpiler would have had to insert SWAP operations to bring them together, and each SWAP costs three more CZ gates.

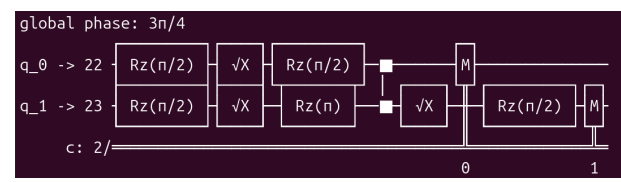


Figure 2: The Bell circuit of *qiskit10.py* after transpilation for *ibm_kingston*

The four Bell states

We are now ready for the central idea of this article. In 1935, Albert Einstein, Boris Podolsky and Nathan Rosen argued that quantum mechanics might not provide a complete description of reality. Their argument concerned entangled correlations: measuring one particle could reveal something about a distant partner. Surely, they reasoned, the result must already have been encoded in the pair through hidden variables that quantum mechanics had omitted. For nearly thirty years, the Einstein-Podolsky-Rosen (EPR) paradox remained largely a conceptual and philosophical dispute. That changed in 1964, when the Northern Irish physicist John Stewart Bell showed that local hidden-variable theories and quantum mechanics make measurably different predictions. A philosophical argument had become an experiment. Alain Aspect, John Clauser, and Anton Zeilinger conducted increasingly sophisticated experiments that supported the quantum predictions, and in 2022 they shared the Nobel Prize in Physics. Regular readers of this series may by now expect

a Nobel laureate in every paragraph. Bell, however, is the exception that proves the rule. He died in 1990, and since Nobel Prizes are not awarded posthumously, he never received the honour. Although Bell never received a Nobel Prize, his theorem paved the way for others to receive science's highest honour and helped settle one of the great philosophical disputes of modern physics.

The four maximally entangled two-qubit states are named the Bell states in his honour. They are shown in Figure 3, written in the two notations this series has used since the first article: Dirac notation, the bra–ket notation named after the physicist and Nobel laureate Paul Dirac, and the vector form, which lists the four amplitudes in the order $|00\rangle$, $|01\rangle$, $|10\rangle$, $|11\rangle$, the superscript T marking the printed row as a column vector. Notice their structure: two of them, conventionally written $|\Phi+\rangle$ and $|\Phi-\rangle$, are superpositions of $|00\rangle$ and $|11\rangle$, and the other two, $|\Psi+\rangle$ and $|\Psi-\rangle$, are superpositions of $|01\rangle$ and $|10\rangle$. Within each pair, the two states differ only by a minus sign—which is to say, only by a phase. The state we have been producing since the previous article is $|\Phi+\rangle$, obtained from $|00\rangle$ by a Hadamard gate followed by a CNOT gate.

Name	Dirac notation	Vector form
$\|\Phi^+\rangle$	$\frac{\|00\rangle+\|11\rangle}{\sqrt{2}}$	$\frac{1}{\sqrt{2}}[1\ 0\ 0\ 1]^T$
$\|\Phi^-\rangle$	$\frac{\|00\rangle-\|11\rangle}{\sqrt{2}}$	$\frac{1}{\sqrt{2}}[1\ 0\ 0\ -1]^T$
$\|\Psi^+\rangle$	$\frac{\|01\rangle+\|10\rangle}{\sqrt{2}}$	$\frac{1}{\sqrt{2}}[0\ 1\ 1\ 0]^T$
$\|\Psi^-\rangle$	$\frac{\|01\rangle-\|10\rangle}{\sqrt{2}}$	$\frac{1}{\sqrt{2}}[0\ 1\ -1\ 0]^T$

Figure 3: The four Bell states

Three properties make these four states special. First, they are maximally entangled: measuring either qubit alone yields 0 or 1 with equal probability, so an individual qubit of a Bell pair carries no information whatsoever, while the pair together is perfectly correlated. All the information lives in the relationship, not in the parts. Second, they are mutually orthogonal and form a complete basis for two qubits, called the Bell basis. Any two-qubit state can be written as a combination of these four, just as any single-qubit state can be written in terms of $|0\rangle$ and $|1\rangle$. Third—and this is the property we shall exploit shortly—all four can be reached from $|\Phi+\rangle$ by operating on just one of the two qubits, leaving the other untouched.

Before we write any more code, a small but important warning about notation. Qiskit prints its bit strings with the highest-numbered qubit on the left. In a result such as '01', the right-hand character is the outcome of qubit 0 (1 in this case) and the left-hand character is the outcome of qubit 1 (0 in this case). Until now nothing has depended on this. Our entangled results were 00 and 11, which read the same in either direction, and the stray 01 and 10 counts in Figure 1 were simply counted together as errors. From this point onwards, 01 and 10 are different answers and the ordering genuinely matters. This convention has confused more beginners than any quantum-mechanical subtlety, so it is worth committing to memory.

Now let us understand, at the mathematical level, what entanglement means. A two-qubit state is called separable, or unentangled, if it can be written as the product of two independent single-qubit states. Suppose $|\Phi+\rangle = (|00\rangle+|11\rangle)/\sqrt{2}$ were separable. Then we could write it as $(\alpha|0\rangle+\beta|1\rangle) \otimes (\gamma|0\rangle+\delta|1\rangle)$, which expands to $\alpha\gamma|00\rangle+\alpha\delta|01\rangle+\beta\gamma|10\rangle+\beta\delta|11\rangle$. Comparing coefficients, we need $\alpha\gamma = 1/\sqrt{2}$ and $\beta\delta = 1/\sqrt{2}$, so none of the four numbers may be zero. But we also need $\alpha\delta = 0$ and $\beta\gamma = 0$, which forces at least one of them to be zero. The two requirements contradict each other, so no such factorisation exists. That short argument is the whole definition: an entangled state is one that cannot be described by describing its parts separately. There is no mysterious influence travelling between the qubits. There is simply no longer any such thing as 'the state of qubit 0'.

Figure 3 tells us what the four states are, but not how to make them. All four begin with the same Hadamard and CNOT pair that produced $|\Phi+\rangle$ in the previous article; what differs is the operation applied afterwards, and it acts on qubit 1 alone. Z turns $|\Phi+\rangle$ into $|\Phi-\rangle$, X turns it into $|\Psi+\rangle$, and X followed by Z turns it into $|\Psi-\rangle$. That is the third property above, expressed as a recipe: the four Bell states differ only by an operation on a single member of the pair.

Now look at what separates $|\Phi+\rangle$ from $|\Phi-\rangle$. It is a single minus sign, and by now that should be setting off an alarm bell. In the third article of this series we met exactly this situation with a single qubit: the Pauli-Z gate appeared to do nothing at all, because a phase cannot be seen in the measurement statistics. Does the same disappointment await us here?

Telling the Bell states apart

Let us find out. The program *qiskit11.py*, shown below, prepares all four Bell states in turn and measures each of them in the computational basis.

```
1.  from qiskit import QuantumCircuit, transpile
2.  from qiskit_aer import AerSimulator

3.  bell_states = {"Phi+": "", "Phi-": "Z", "Psi+": "X",
    "Psi-": "XZ"}
4.  sim = AerSimulator( )
```

```
5.   for name, local_gates in bell_states.items( ):
6.       qc = QuantumCircuit(2, 2)
7.       qc.h(0)
8.       qc.cx(0, 1)
9.       if “X” in local_gates:
10.          qc.x(1)
11.      if “Z” in local_gates:
12.          qc.z(1)
13.      qc.measure([0, 1], [0, 1])
14.      compiled = transpile(qc, sim)
15.      counts = sim.run(compiled, shots=1000).result(
         ).get_counts( )
16.      print(name, counts)
```

Now let us understand how the program works. Line 3 defines a dictionary whose keys name the four Bell states and whose values record the extra gates each one needs. Line 5 loops over them. Lines 6 to 8 build the |Φ+⟩ circuit we already know, and Lines 9 to 12 apply the extra gates to qubit 1. Line 13 measures both qubits, and Lines 14 to 16 run the circuit for 1000 shots and print the counts. Remember to activate *qiskit_env* before running the program.

The results are shown in Figure 4, and the outcome is disappointing in a very instructive way. Both |Φ+⟩ and |Φ−⟩ produce roughly 500 counts of 00 and 500 of 11. Both |Ψ+⟩ and |Ψ−⟩ produce roughly 500 counts of 01 and 500 of 10. Four distinct, mutually orthogonal quantum states have collapsed into two indistinguishable pairs of statistics. The reason is arithmetic: a probability is the squared magnitude of an amplitude, and both +1/√2 and −1/√2 square to the same 1/2. The minus sign is still in the state; it simply does not survive the squaring.

```
(qiskit_env) # python qiskit11.py
Phi+ {'11': 514, '00': 486}
Phi- {'00': 483, '11': 517}
Psi+ {'01': 520, '10': 480}
Psi- {'10': 495, '01': 505}
```

Figure 4: Output of the program *qiskit11.py*

Two orthogonal quantum states—as different as two states can be, and distinguishable in principle with perfect certainty—have been made to look identical simply by asking the wrong question. The information is not missing. We are asking in the wrong language.

So let us ask properly. Recall from the second article that quantum gates are reversible. If the circuit *H* followed by CNOT turns |00⟩ into |Φ+⟩, then running that circuit backwards—CNOT followed by H—must turn |Φ+⟩ back into |00⟩. And since the four Bell states form a basis, the same reversed circuit should map each of the other three to a different computational basis state. This procedure is called a Bell measurement, and the program *‘qiskit12.py’*, shown below, implements it.

```
1.   from qiskit import QuantumCircuit, transpile
2.   from qiskit_aer import AerSimulator

3.   bell_states = {“Phi+”: “”, “Phi-”: “Z”, “Psi+”: “X”,
     “Psi-”: “XZ”}
4.   sim = AerSimulator( )

5.   for name, local_gates in bell_states.items( ):
6.       qc = QuantumCircuit(2, 2)
7.       qc.h(0)
8.       qc.cx(0, 1)
9.       if “X” in local_gates:
10.          qc.x(1)
11.      if “Z” in local_gates:
12.          qc.z(1)
13.      qc.cx(0, 1)
14.      qc.h(0)
15.      qc.measure([0, 1], [0, 1])
16.      compiled = transpile(qc, sim)
17.      counts = sim.run(compiled, shots=1000).result(
         ).get_counts( )
18.      print(name, counts)
```

Now let us understand how the program works. The only new statements are Lines 13 and 14, which apply a CNOT gate and then a Hadamard gate before the measurement in Line 15. That is the preparation circuit of Lines 7 and 8, run in reverse order. Everything else is unchanged.

The output is shown in Figure 5, and it is one of the most satisfying results we have obtained in this series. The state |Φ+⟩ yields ‘00’ in all 1000 shots. |Φ−⟩ yields ‘01’ in all 1000 shots. |Ψ+⟩ yields ‘10’ every time, and |Ψ−⟩ yields ‘11’ every time. Not approximately, not on average—every single shot. The randomness has vanished completely, and the four states that were indistinguishable a moment ago are now labelled with perfect certainty.

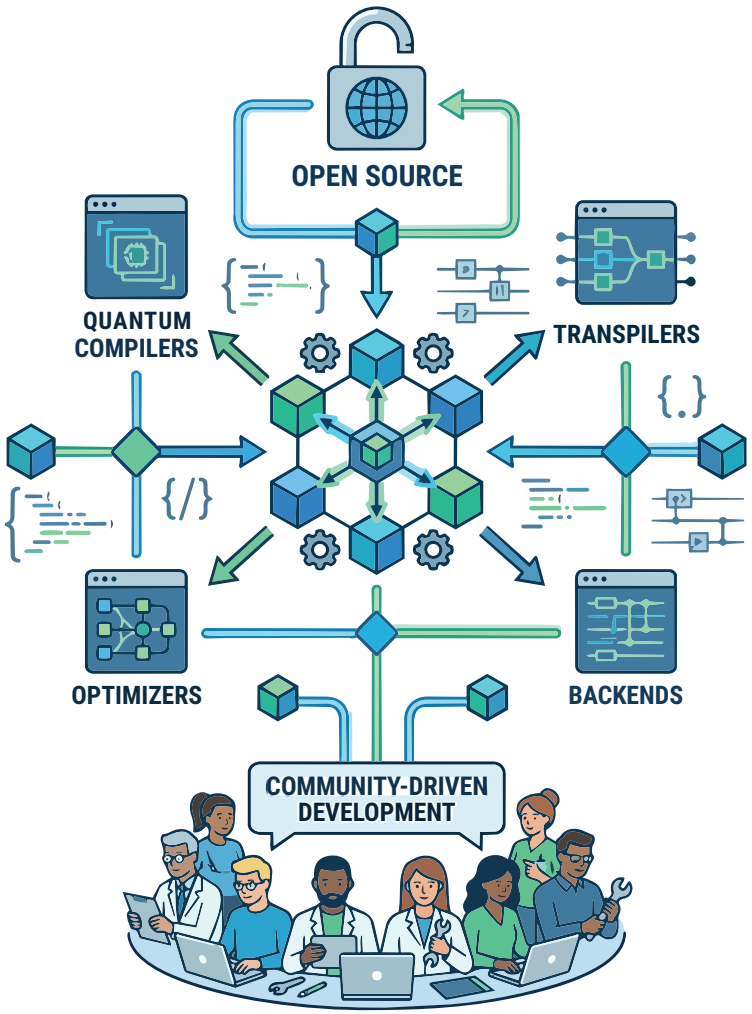
```
(qiskit_env) # python qiskit12.py
Phi+ {'00': 1000}
Phi- {'01': 1000}
Psi+ {'10': 1000}
Psi- {'11': 1000}
```

Figure 5: Output of the program *qiskit12.py*

Take a moment to appreciate what has happened, because it contains the seed of everything that follows in this series. The phase difference between |Φ+⟩ and |Φ−⟩ was invisible to measurement. By applying two more gates

■■ *Continue to page...48*

Open Source Quantum Compilers:
A Technical Overview of the Current Landscape



Quality quantum compilers are a necessity to get the best results from noisy quantum machines. Fortunately, there are quite a few open source compilers that are up to the task.

Every quantum program needs to transform abstract logic of qubits and gates into a sequence of control signals a machine can interpret. Software that carries out this function is called a quantum compiler. Constructing a compiler for early-stage quantum machines that have limited connectivity, coherence, and suffer from noise is a major challenge for quantum computing. For such machines, the quality of a compiler can make a considerable difference in the performance of an algorithm.

In classical computing, the main challenges are speed and code size, whereas in quantum computing, the main problems are decoherence, gate errors, and qubit connectivity. Despite these problems, hardware vendors have been sustaining an active open source community of quantum compiler builders. Vendor competition has made the quality of available quantum compilers a priority.

What a quantum compiler does

A quantum compiler lowers a circuit from a high level of abstraction to a level that a machine can execute. The stages in this process include:

- Circuit translation — Conversion of a circuit to an internal intermediate representation, which is typically a directed acyclic graph (DAG) of gates.
- Circuit optimisation — Combining, cancelling, and rewriting gate

A comparison of the major open source compilers

Compiler/ SDK	Backer	Language	Distinguishing focus
Qiskit (Transpiler)	IBM	Python	Broadest feature set: pulse control, dynamic circuits, noise-aware passes, largest hardware install base
Cirq	Google	Python	Fine-grained control for NISQ research; native path to Google superconducting hardware
tket/pytket	Quantinuum (CQC)	C++/Python	Retargetable optimiser prized for aggressive circuit-depth and gate-count reduction
PennyLane + Catalyst	Xanadu	Python	JIT compilation for differentiable, hybrid quantum-classical programs; strong QML fit
XACC	Oak Ridge Nat'l Lab	C++	Extensible, HPC-oriented compiler for integrating quantum kernels into classical supercomputing workflows
Staq/ ProjectQ	Academic/ Community	C++/Python	Lightweight, research-friendly toolchains for circuit synthesis and translation between formats

sequences to lower the circuit depth and the overall number of gates using algebraic identities.

- Qubit mappings and routing — Logical qubits are linked to physical qubits, and SWAP gates are placed to ensure that two-qubit gates only connect physically neighbouring qubits.
- Basis gate decomposition — Decomposing arbitrary gates in terms of the gate set that a particular machine supports.
- Noise-aware scheduling — This is about reducing error by using calibration data, which includes error rates, gate durations and crosstalk, to identify the least noisy qubits and the optimal gate order. The output is usually a hardware-executable circuit, which is often accompanied by detailed, pulse-level, instructions for the control electronics.

The major open source compilers

A handful of projects dominate current usage, each shaped by the hardware vendor or lab behind it.

Qiskit Transpiler (IBM): Qiskit is the most feature-rich open source library, covering the entirety of circuit construction, control, and dynamic circuits as well as modelling, noise, and simulation. It also has tight integration with IBM hardware.

Cirq (Google): Cirq is built for experimentation on near-term hardware and gives users flexibility in control, which is Google's superconducting processor. Until recently, access to Google's hardware was more limited than IBM's, which has kept its mindshare somewhat behind Qiskit.

tket/pytket (Quantinuum): Unlike Cirq and Qiskit, tket is a modular compiler that includes optimisation passes, which produce very thin circuits with low gate counts. It is easy to incorporate tket into other libraries.

PennyLane and Catalyst (Xanadu): Instead of focusing on optimisation, PennyLane has built tools for quantum machine learning. It's Catalyst just-in-time compiler now compiles directly to hardware control signals and is one of the first fully open source stacks spanning software through hardware.

XACC, Staq, ProjectQ, and more: Past the four main SDKs, a lengthier tail of community and lab-driven efforts targets unique use cases. XACC aims to facilitate integration of quantum kernels within classical high performance-computing frameworks, while Staq and ProjectQ provide circuit synthesis and format translation tools that are more appropriate for use in academia.

Core compilation techniques

Gate-level optimisation: Here, various peephole optimisation rewrite rules are applied to circuits. Gates are cancelled by their inverse and moved between circuits, while rotations are merged. To the best of my knowledge, no hardware mapping is employed at this level.

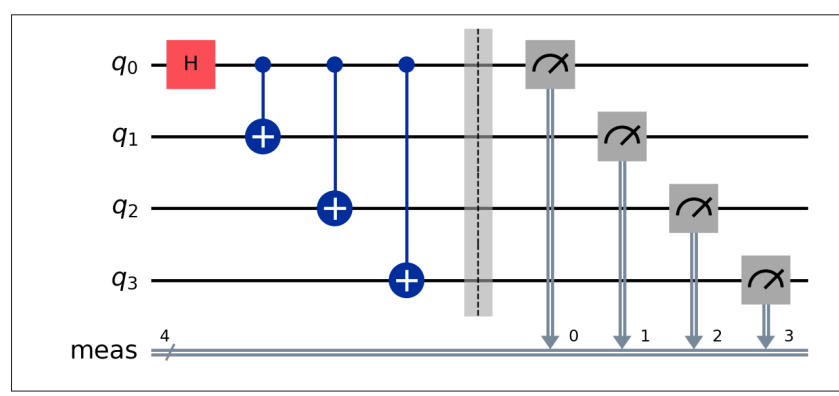


Figure 1: The logical circuit as written, before compilation

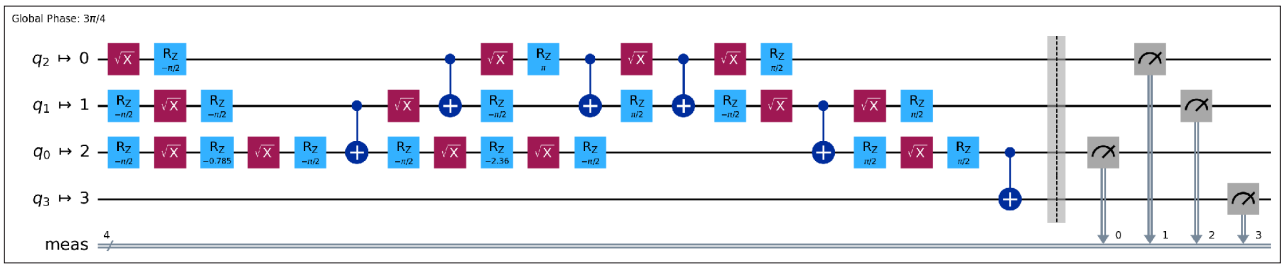


Figure 2: The same circuit as Figure 1, after transpilation: qubits remapped (q2↦0, q1↦1, q0↦2, q3↦3), non-adjacent CX gates routed through SWAPs, and every gate decomposed into the device's native basis

Qubit mapping and routing: Since most superconducting and many trapped-ion devices only connect a subset of qubit pairs, the compiler determines the placement of logical qubits and inserts SWAP networks to overcome connectivity constraints. This is usually implemented through heuristic or graph-based search algorithms..

Noise-aware and calibration-driven passes: Transpilers that support noise mitigation now integrate daily calibration data to perform the least error-prone routing and scheduling.

Pulse-level and hybrid compilation: This involves compiling beyond just mapping to gates and includes variational algorithms. In a hybrid approach, Catalyst extends this through classical control.

Choosing a compiler

The right choice highly depends on your target hardware and what you aim to achieve:

- If you want general-purpose learning and broad flexibility over the hardware you control, Qisket has the best resources and tutorials.
- If you want to maximise circuit depth for a noisy device, tket's optimiser is often used as the second-pass compiler, even with another SDK's frontend.
- If you want to pursue QML (quantum machine learning) or variational algorithms and to include gradients, PennyLane with Catalyst is the optimal choice.
- If you want to engage in research focused on Google's hardware or use gate-model prototyping, Cirq is a good choice.
- If you want to integrate quantum kernels into classical HPC, use XACC.

```
qiskit — transpile output

$ python demo_qiskit.py
=== Qiskit transpile: before vs after ===
Metric                    Logical    Transpiled
Depth                           5            21
Gate count                      9            41
2-qubit gates (cx)              3             6

Transpiled op counts: {'rz': 17, 'sx': 13, 'cx': 6, 'measure': 4, 'barrier': 1}
```

Figure 3: Actual console output: depth grew from 5 to 21 and gate count from 9 to 41 purely to satisfy hardware constraints, with no change to the circuit's logic

Implementation walkthrough

The next two examples were executed in Python to illustrate compilation in a more practical manner. They both use a 4-qubit GHZ-like entangling circuit to illustrate before and after images of what one pass of the compiler looks like.

- ***Qiskit: Transpiling for limited hardware connectivity***

This example builds a circuit in which logical CX gates connect qubit 0 to every other qubit, and then transpiles it for a device that only has linear nearest-neighbour connectivity (0–1–2–3) and a restricted native gate set.

1. Install the SDK: *pip install qiskit*
2. Build the logical circuit with QuantumCircuit(4), adding an H gate and three CX gates fanning out from qubit 0.
3. Define the target hardware's coupling map ([[0,1],[1,2],[2,3]]) and native basis gates (cx, rz, sx, x).
4. Call *transpile(circuit, coupling_map=..., basis_gates=..., optimization_level=3).*
5. Compare *circuit.depth()* and *count_ops()* before and after to see the routing and decomposition cost.

- ***pytket: Peephole optimisation***

This example deliberately adds redundant operations — a double Hadamard (H·H = identity) and a zero-angle rotation — to a similar GHZ-style circuit, then runs tket's FullPeepholeOptimise pass to show automatic simplification.

6. Install the SDK: *pip install pytket*
7. Build the circuit with Circuit(4), adding the entangling gates plus the two deliberately redundant operations.
8. Record *depth(), n_gates,* and the CX count before optimisation.
9. Apply the pass: *FullPeepholeOptimise(). apply(circuit).*
10. Re-check the same metrics and print the resulting gate list to confirm the redundant operations are gone.

Together, the two examples illustrate the two complementary jobs a quantum compiler does: Qiskit's transpiler shows the cost of adapting a circuit to real hardware constraints, while tket's

optimiser shows how much of that cost can be clawed back through algebraic simplification before or after routing.

All these projects aim at interoperability instead of settling on a single winner. Tket runs circuits in both Qiskit and Cirq. PennyLane offers plugins to interface with multiple backends. Cloud QPUs (IBM Quantum, Azure Quantum, Amazon Braket, etc) provide a choice of compilers as a pluggable layer, as opposed to a completely integrated and set stack. With improvements in hardware, focus in compiler research will likely shift to error correction compilation and resource estimation in conjunction with noise-aware, calibration driven optimisation.

Open source quantum compilation tools have advanced to the point where they can be considered serious competition. Qiskit, Cirq, tket and PennyLane, for instance, have a great deal of flexibility, each in their own ways. Most serious open source projects leverage multiple compilers. It is now essential to understand what each compiler optimises, and why, in order to get the best results from today's noisy quantum devices. END

```
pytket — FullPeepholeOptimise output

$ python demo_tket.py
=== pytket FullPeepholeOptimise: before vs after ===
Metric                    Before     After
Depth                          7         5
Gate count                    11         8
2-qubit gates (CX)             3         3

After (gate list):
  TK1(0.5, 0.5, 0.5) q[0];
  CX q[0], q[1];
  Measure q[1] --> c[1];
  CX q[0], q[2];
  Measure q[2] --> c[2];
  CX q[0], q[3];
  Measure q[0] --> c[0];
  Measure q[3] --> c[3];
```

Figure 4: Actual console output: the optimiser collapsed the redundant H·H and zero-angle Rz into a single combined TK1 rotation, cutting depth from 7 to 5 and gate count from 11 to 8 while leaving the three CX gates untouched

References

- The Quantum Insider, 'State of Quantum Computing: Programming Languages', June 2026.
- Adnan Masood, 'The Quantum Platforms Briefing — Day 5: Open Source Ecosystem', Medium, October 2025.
- QuantumZeitgeist, 'Top Quantum Software Companies 2026: The Definitive Stack Guide', May 2026.
- PostQuantum.com, 'Quantum Programming: An In-Depth Introduction and Framework Comparison', September 2025.
- RankRed, '13 Quantum Programming Languages (& Tools) To Learn In 2026'.

By: Dr R. Anand

The author is a professor in the Department of Computer Science and Engineering at Easwari Engineering College, Chennai, Tamil Nadu.

Continue from page...44

before measuring, we converted that invisible phase into a difference in outcomes that is not merely visible but absolutely certain. This is the same trick we used in the third article, when a second Hadamard gate exposed the hidden work of the Pauli-Z gate, and it has a name: quantum interference. Every quantum algorithm worth the name is, at bottom, an elaborate arrangement of this trick.

It is now time to conclude. In this article we executed an entangling circuit on a real 156-qubit superconducting processor and found 13 shots out of 1000 that had no business being there. Chasing those 13 counts took us through the whole zoo of errors that afflict present-day quantum hardware—relaxation, dephasing, gate errors, readout errors, crosstalk, leakage, material defects, and cosmic rays. We then met the four Bell states, proved what it means for a state to be entangled, discovered that measurement in the computational basis cannot tell all four apart, and repaired that failure with a Bell measurement.

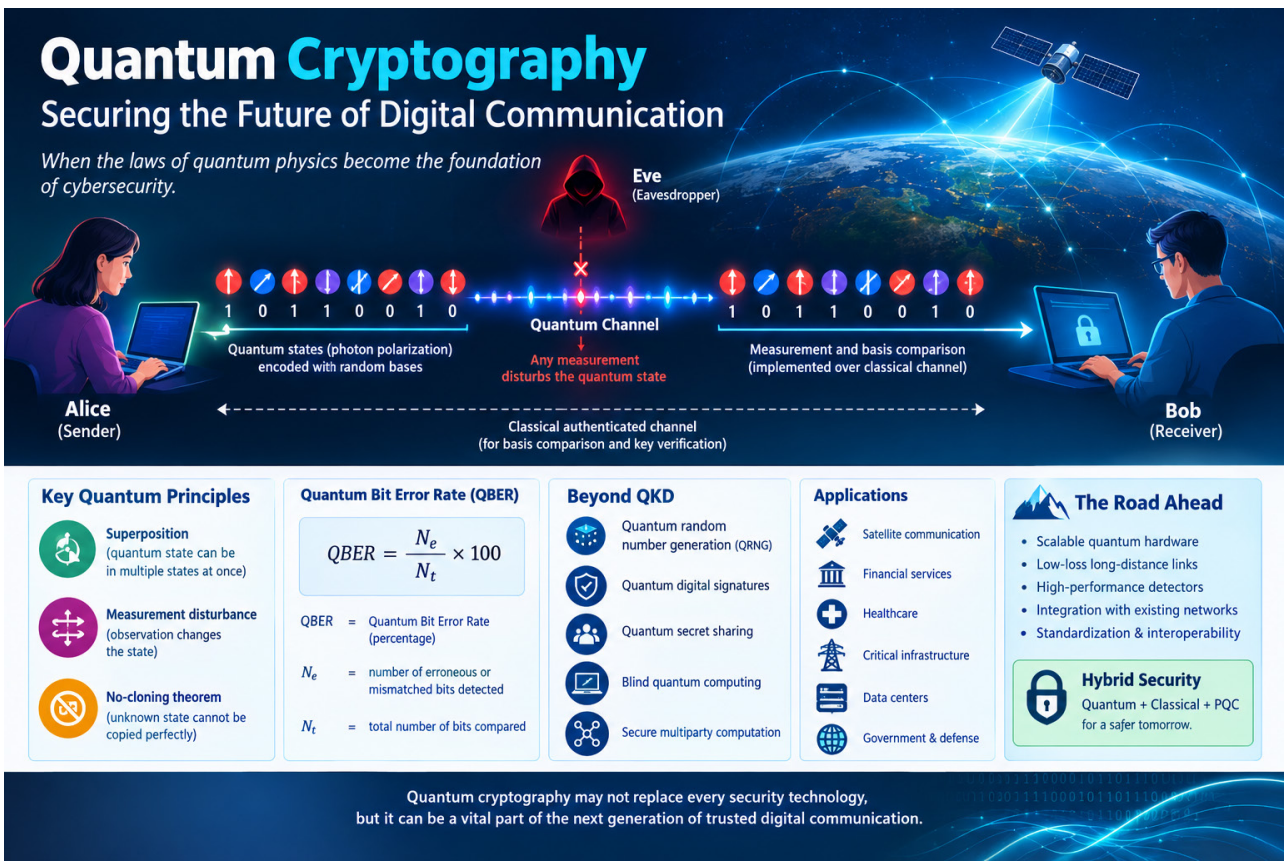
That last experiment left us with a loose thread, and it is deliberately loose. Twice now we have made a hidden phase visible by applying an extra gate before measuring: once with a single qubit in the third article, and once with a Bell pair in this one. Both times the revealing gate was the Hadamard—the first gate we ever met, and the one we have used most often without examining it closely. In the next article we shall give the Hadamard operator the attention it deserves and then turn to the phenomenon it has quietly been exploiting all along: quantum interference. We shall see how the cancellation of unwanted amplitudes—not some fantasy of trying every answer at once—is the real engine of quantum speedup. Entanglement, it turns out, was only half the story. END

By: Dr Deepu Benson

The author is a free software enthusiast with a keen interest in theoretical computer science. He received his PhD from IIT Palakkad and completed his postdoctoral research at IIT Gandhinagar. He maintains a technical blog.

Quantum Cryptography: Ensuring Secure Digital Communication

The digital world is entering an era in which the security of information can no longer depend solely on traditional cryptographic systems. Quantum cryptography is poised to become a significant component of future digital communication.



From online banking and digital healthcare to satellite communication and critical infrastructure, enormous volumes of sensitive information are transmitted every second. Traditional cryptographic systems have protected this information for decades, but the emergence of quantum computing is challenging many of the assumptions on which these systems are built. Quantum cryptography offers a fundamentally different approach: instead of relying only on mathematical problems that are difficult to solve, it uses the laws of quantum mechanics to detect eavesdropping and establish highly secure communication.

One of the most promising applications of quantum cryptography is Quantum Key Distribution (QKD). QKD allows two legitimate users to generate a shared secret cryptographic key while providing a mechanism to detect whether an attacker has attempted to intercept the communication.

From mathematical security to physical security

Most modern cryptographic systems use mathematical problems as the basis of security. For example, widely deployed public-key algorithms depend on the computational difficulty of problems such as integer factorization or discrete logarithms. A sufficiently powerful quantum computer, however, could use algorithms such as Shor's algorithm to solve some of these problems much more efficiently.

Quantum cryptography takes a different path. It exploits fundamental

quantum properties such as superposition, measurement disturbance, and the no-cloning theorem.

The no-cloning theorem states that an unknown quantum state cannot be perfectly copied. Consequently, an attacker cannot simply duplicate a quantum state, examine it, and forward an identical copy without potentially introducing detectable changes.

This leads to a powerful security principle: attempting to observe quantum information can disturb it.

How Quantum Key Distribution works

A widely studied QKD protocol is BB84, introduced by Charles Bennett and Gilles Brassard in 1984. In a simplified implementation, a sender, commonly called Alice, encodes information into quantum states of individual photons. The receiver, Bob, measures these states using randomly selected measurement bases.

For example, photon polarization can be used to represent quantum states. Alice may randomly select between two measurement bases and encode a sequence of bits. Bob independently chooses measurement bases when measuring the incoming photons.

After transmission, Alice and Bob communicate over a conventional authenticated classical channel. They publicly compare their selected bases but do not reveal the encoded bit values. Measurements made using compatible bases are retained, while incompatible measurements are discarded. The remaining sequence forms the sifted key.

If an eavesdropper, commonly called Eve, attempts to intercept and measure the photons, she generally does not know which measurement basis was used. Her measurements therefore introduce errors into the transmitted quantum states. Alice and Bob can estimate the resulting error rate and determine whether the communication channel has been compromised.

The role of Quantum Bit Error Rate

An important parameter in QKD is the Quantum Bit Error Rate (QBER). It can be expressed as:

$$QBER = \frac{Nt}{Ne} \times 100\,\%$$

…where:

- *Ne* = number of erroneous bits *detected*
- *Nt* = Tool number bits compared
- *QBER* = Quantum Bit Error Rate

A low QBER generally indicates that the quantum channel has relatively few errors, whereas a high QBER may indicate significant noise, hardware imperfections, or a potential eavesdropping attempt.

It is important to recognise that QKD does not automatically eliminate every cybersecurity risk. Practical systems must also address device imperfections, authentication, implementation vulnerabilities, detector attacks, and the security of the classical communication channel.

Quantum cryptography beyond QKD

Although QKD receives considerable attention, quantum cryptography is broader than quantum key distribution. Researchers are investigating several quantum-enabled approaches for secure communication, authentication, randomness generation, and secure computation.

Quantum Random Number Generation (QRNG) is another important application. Classical pseudo-random number generators rely on deterministic algorithms, whereas quantum processes can provide sources of fundamentally unpredictable randomness. High-quality random numbers are essential for cryptographic keys, authentication protocols, and security systems.

Quantum technologies are also being investigated for quantum digital signatures, quantum secret sharing, blind quantum computing, and secure multiparty computation. These developments could eventually create security architectures in which quantum and classical technologies operate together.

Quantum networks: The next frontier

The long-term vision extends beyond point-to-point QKD. Researchers are working towards quantum communication networks, potentially connected through quantum repeaters and quantum memories.

A future quantum network could enable the distribution of entangled quantum states across geographically separated locations. Such networks could form the foundation of a quantum internet, where quantum information is transmitted alongside conventional digital data.

Satellite-based quantum communication is particularly attractive for long-distance links. Terrestrial optical fibres introduce losses that make extremely long-distance quantum communication challenging. Satellites could potentially provide communication paths over much larger distances.

This technology could have significant implications for governments, financial institutions, defence organisations, healthcare networks, data centres, and critical infrastructure.

Challenges on the road to adoption

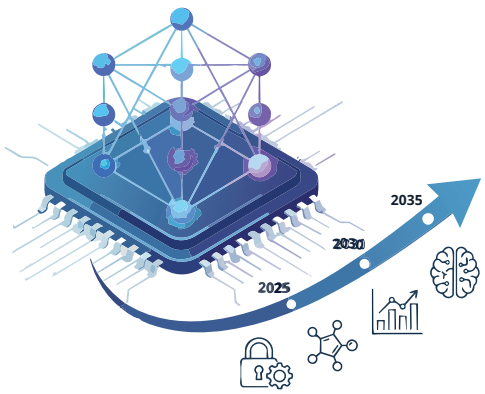
Despite its promise, quantum cryptography is not a simple replacement for existing encryption. One of the major challenges is hardware scalability. Quantum states are extremely sensitive to environmental disturbances, making reliable transmission and measurement technically demanding.

Other challenges include:

- Photon losses over long communication distances.
- High-performance quantum detectors and sources.
- Integration with existing communication infrastructure.

■■ *Continue to page...55*

The Road Ahead: Quantum Computing in the Next Decade



The emergence of quantum computing is bound to be disruptive, but organisations that learn to complement it with existing digital infrastructure will benefit.

Quantum computing has moved from the edges of theoretical physics into the centre of strategic technology planning. For decades, it was discussed mainly as a scientific possibility, admired for its elegance but viewed as distant from everyday use. That perception is changing. Governments, universities, cloud providers, startups, and large enterprises are now treating quantum technologies as part of a serious long-term innovation agenda. The next decade will not necessarily bring a universal quantum computer sitting on every desk, but it is very likely to reshape how organisations think about computation, security, materials discovery, optimisation, and scientific modelling.

The road ahead is both exciting and uneven. Quantum computing is advancing rapidly, yet it remains constrained by engineering realities such as qubit stability, error correction, cooling requirements, and the difficulty of converting laboratory breakthroughs into reliable industrial systems. The coming years will therefore be marked by a careful balance between ambition and practicality. The most successful organisations will be those that understand quantum computing not as a sudden replacement for classical computing, but as a specialised capability that will mature gradually and complement existing digital infrastructure.

Assessing progress: Where are we now?

At present, quantum computing is still largely in the noisy intermediate-scale quantum era, often described as the stage in which machines have enough qubits to demonstrate meaningful physical behaviour but not enough reliability to solve most commercial problems better than classical systems. Current processors can run experiments, support algorithm research, and help scientists test error-mitigation techniques, but their results are limited by noise and short circuit depths. This does not make today's systems insignificant; rather, it places them in the same category as many early technologies that were scientifically promising before they became operationally dependable.

The most important progress is no longer just the number of physical qubits on a chip. The central question is whether these can be organised into logical qubits that correct errors reliably enough to run deep computations. Recent industry roadmaps and market assessments point to a transition from isolated demonstrations towards early fault-tolerant components, with commercial interest also increasing as hundreds of organisations explore quantum use cases and capability building. This suggests that quantum computing is not yet broadly useful in production, but is becoming too mature for technology leaders to ignore.

Predictions: What's likely in five and ten years?

Over the next five years, the most credible progress is likely to occur in error correction, hybrid quantum-classical workflows, quantum software tooling, and cloud-based access models. Enterprises may not run large mission-critical workloads on quantum machines by default, but they will increasingly experiment with quantum algorithms through cloud platforms and specialised partnerships. Industries with complex optimisation, simulation, and risk-modelling needs will continue to test whether quantum-inspired and early quantum approaches can create advantage in narrow problem areas.

Within ten years, the picture may look more substantial, though still selective. If fault-tolerant systems scale as expected, quantum computers may begin to show practical value in chemistry, materials science, logistics, drug discovery, and certain forms of cryptanalysis. However, the decade ahead should not be imagined as a single dramatic leap. It will probably unfold as a sequence of domain-specific successes, where quantum systems solve carefully chosen problems alongside classical supercomputers,

artificial intelligence models, and high-performance data platforms.

Quantum's impact on society and workforce

Quantum computing could influence society in ways that extend beyond faster calculation. Its ability to model molecular structures, optimise complex systems, and accelerate certain classes of mathematical problems may affect healthcare, energy, finance, climate research, and national security. These benefits will not arrive evenly across the world. Countries and companies that invest early in talent, infrastructure, and responsible deployment may gain a significant advantage, while others may find themselves dependent on external platforms and expertise.

The workforce impact will be equally important. Quantum computing will create demand for physicists, mathematicians, computer scientists, control engineers, materials experts, cybersecurity professionals, and business translators who can connect quantum capabilities to real organisational problems. It will also change the profile of existing roles. Software engineers may need to understand quantum algorithms at a conceptual level, security architects will need to plan for post-quantum cryptography, and enterprise architects will need to evaluate when quantum services fit into broader technology portfolios.

Policy, regulation, and ethical considerations

Policy and regulation will become central as quantum computing matures. One of the most urgent concerns is the long-term security of public-key cryptography. Even before a cryptographically relevant quantum computer exists, sensitive data that is captured today could be decrypted later if it remains valuable. This 'harvest now, decrypt later' risk is why governments and standards bodies have been accelerating post-quantum cryptography guidance. Organisations should begin migration planning early because cryptographic change is rarely quick, especially in large enterprises with legacy systems, embedded devices, vendor dependencies, and long-lived data.

Ethical questions will also grow more complex. Quantum advantage could deepen inequality if only a small number of nations or corporations control the most powerful systems. There are also dual-use concerns, since capabilities that support scientific discovery may also be relevant to defence, surveillance, and cyber operations. Responsible quantum governance should therefore encourage innovation while setting expectations for transparency, security, access, and accountability. The goal should not be to slow scientific progress, but to ensure that progress is aligned with public trust and shared benefit.

Education and preparing for the quantum age

Preparing for the quantum age requires a broader educational approach than training only a small group of specialists. Universities will remain essential for deep research in quantum physics, mathematics, and engineering, but enterprises will also need practical learning pathways for technologists and decision-makers. A security leader does not need to design a dilution refrigerator, but they should understand why post-quantum cryptography matters. A business executive does not need to derive quantum algorithms from first principles, but they should know when quantum claims are credible and when they are exaggerated.

Organisations can begin now by building quantum literacy programs, supporting experimentation through cloud-based quantum platforms, and encouraging collaboration between domain experts and technical teams. The most valuable preparation may be the ability to identify suitable problems rather than to operate quantum hardware directly. Employees who understand optimisation, simulation, cryptography, data science, and systems architecture will be well positioned to translate quantum capabilities into practical enterprise value as the technology matures.

Potential disruptions across industries

The industries most likely to experience early disruption are those in which small improvements in prediction, simulation, or optimisation can produce large economic gains. Pharmaceutical companies may use quantum methods to explore molecular interactions more effectively, while materials firms may search for better batteries, catalysts, superconductors, and lightweight compounds. Financial institutions may investigate portfolio optimisation, risk analysis, and derivative pricing, although such applications will need careful validation because speed alone is not enough; accuracy, explainability, and regulatory confidence will matter just as much.

Logistics, energy, aviation, and manufacturing may also feel the effects as quantum techniques are applied to routing, scheduling, grid balancing, supply-chain design, and complex engineering simulations. Cybersecurity will face a different kind of disruption, driven less by commercial optimisation and more by the need to protect information against future quantum attacks. Taken together, these changes suggest that quantum computing will not transform every industry at the same speed. Its impact will be strongest where the problem is mathematically demanding, the reward is significant, and the organisation has enough technical maturity to integrate quantum results into real decision-making.

Investing in quantum solutions

The industrial opportunities described above create an immediate strategic question: how should capital be committed while hardware is improving but commercial advantage remains selective? A sensible answer is portfolio-based investment rather than a single large bet. Governments and universities can fund foundational physics, fabrication, cryogenics, control electronics, error correction, testbeds and open research, while enterprises concentrate first on quantum literacy, problem discovery, cloud access and integration skills. The European Union's 2025 Quantum Europe Strategy follows this broader logic by linking research, shared infrastructure, startup growth, resilient supply chains, security and skills instead of treating the processor as the entire market.

For most organisations, investment should progress through three horizons. The first is readiness: appoint a small cross-functional team, map high-value computational bottlenecks, inventory quantum-vulnerable cryptography and establish

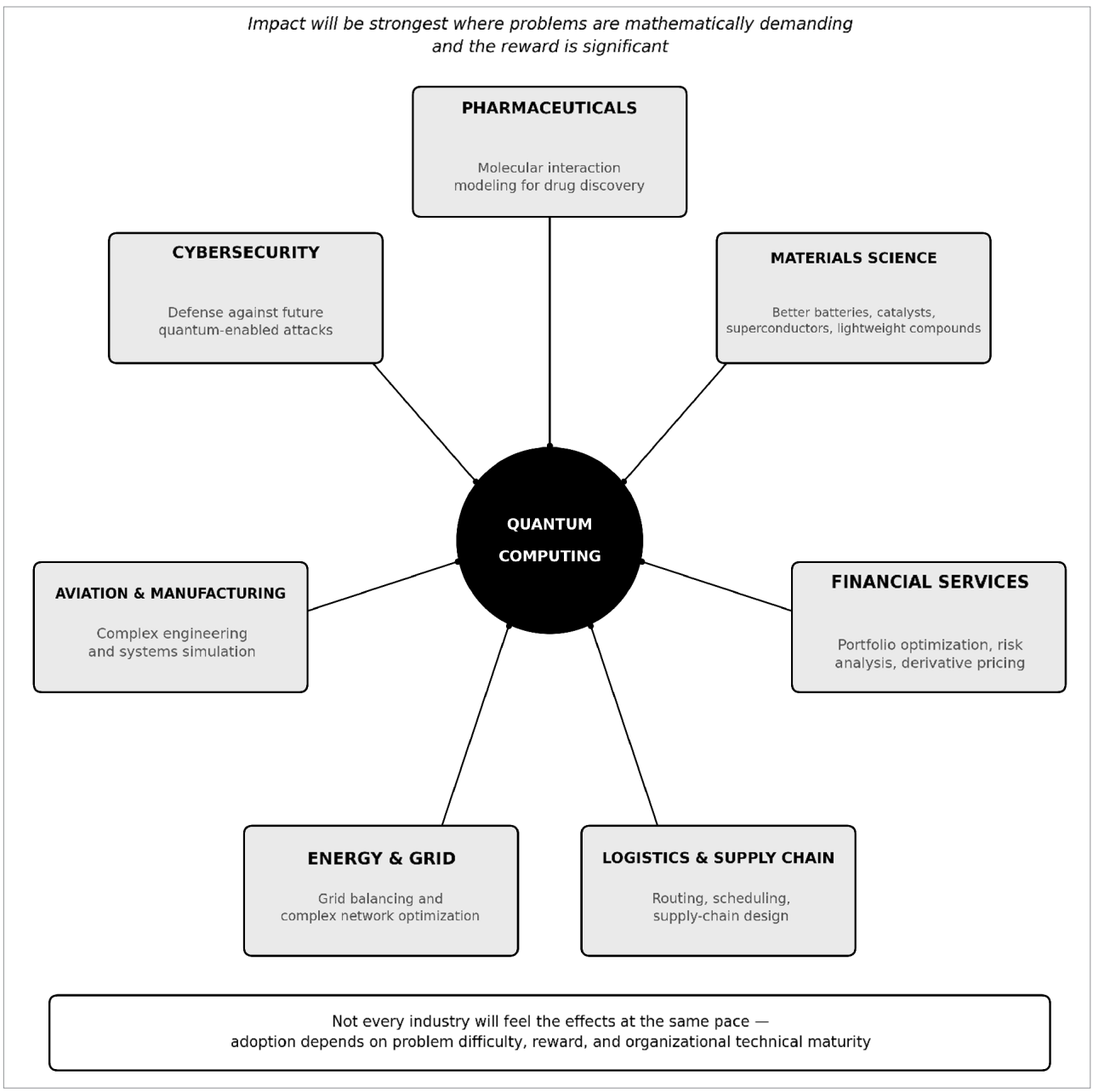


Figure 1: Potential disruptions across industries

relationships with cloud providers and research partners. The second is disciplined experimentation through Quantum-as-a-Service, which provides access to different hardware modalities without premature capital expenditure. Candidate problems in molecular simulation, materials, scheduling, portfolio construction or grid optimisation should be tested through proofs-of-concept. Every result must be compared with strong, classical, high-performance computing and quantum-inspired baselines using total runtime, solution quality, energy and access cost, data-loading overhead, reproducibility and operational relevance. Only validated cases should move into the third horizon of pilots and hybrid production integration.

Investment decisions must therefore look beyond physical-qubit counts. More useful indicators include logical error rates, executable circuit depth, gate fidelity, throughput, compiler performance, availability, interoperability, and the cost of connecting quantum output to an existing decision process. Current vendor roadmaps place ambitious fault-tolerant systems near the end of the decade, but these remain engineering targets rather than guaranteed capacity. Diversification across hardware approaches, software layers and partners can reduce modality risk, vendor lock-in and dependence on a single roadmap while preserving intellectual-property options.

One investment can begin immediately: cryptographic modernisation. NIST's finalised post-quantum standards are available for implementation, and its migration programme emphasises cryptographic inventories, risk prioritisation, interoperability and benchmarking. Organisations should build crypto-agility into procurement and architecture now, especially where data must remain confidential for many years. The strongest quantum portfolio will ultimately combine measured experimentation with practical security work and invest in the interfaces through which quantum processors will cooperate with AI, cloud and supercomputing platforms.

Quantum as part of convergent technologies

Those interfaces place quantum within a convergent architecture rather than a separate computing silo. Quantum computing, quantum communication and quantum sensing must first be distinguished. Computing uses controlled qubits to execute specialised algorithms; communication distributes quantum states or keys across networks; sensing exploits quantum effects for precise measurements of time, fields or motion. Their maturity, infrastructure and use cases differ, although all three may eventually interact with AI, cloud platforms, high-performance computing, cybersecurity and advanced engineering.

A practical converged workflow will be heterogeneous. Data from laboratories, enterprise systems, digital twins or IoT sensors will be prepared by classical software. An orchestration layer will schedule work across CPUs, GPUs, supercomputers and, only where justified, a quantum processing unit. The QPU may evaluate a molecular state, sample a circuit or handle a formulated optimisation subproblem; a classical optimiser or AI model will interpret the measurements, update parameters, quantify uncertainty and return the result to the application. This loop matters because quantum machines require classical control, error decoding, compilation and post-processing, while data-encoding costs, latency and repeated measurements can erase a theoretical advantage.

The infrastructure is already moving in this direction. EuroHPC's 2026 access programme allows European users to request several quantum technologies through the ecosystem used for supercomputers. Its LUMI-IQ initiative goes further by proposing an experimental platform combining HPC, AI and emerging quantum capabilities. Such developments make the quantum processor resemble a specialised accelerator within a shared computational fabric rather than a replacement for classical machines.

Convergence can also create compound capabilities. Peer-reviewed work on AlphaQubit showed that transformer-based machine learning can improve decoding of experimental quantum error-correction data, illustrating how AI may help stabilise future processors. In chemistry and materials discovery, quantum routines could estimate selected electronic properties, while HPC performs large-scale preprocessing and machine learning searches wider chemical spaces. In digital twins, a quantum optimiser might explore constrained alternatives while classical simulation verifies safety and feasibility. For 6G, robotics and distributed IoT, near-term quantum contributions may come more from sensing, timing and secure communications than from placing fragile QPUs at the edge.

Real value will depend on open source toolchains, common intermediate representations, portable APIs, workload schedulers, provenance and shared benchmarking. Post-quantum cryptography must protect the surrounding data and control plane, while quantum key distribution should be evaluated separately for specialised links. With these foundations, convergence becomes a disciplined method for assigning each technology the task it performs best.

The next decade is unlikely to deliver instant transformation across every sector, but it will steadily redefine what is computationally possible in areas where classical methods struggle to scale. Quantum's greatest value will emerge when organisations treat it as part of a wider technology ecosystem, working alongside artificial intelligence, cloud platforms, high-performance computing, and advanced security practices. To benefit from this shift, leaders will need to invest early in skills, governance, experimentation, and cryptographic readiness, while remaining realistic about technical limits. The future of quantum computing will belong not only to those who build powerful machines, but also to those who understand how to translate emerging capability into responsible, practical, and lasting impact. END

References

- *https://www.constellationr.com/insights/news/quantum-computing-where-we-stand-2026*
- *https://www.mckinsey.com/capabilities/mckinsey-technology/our-insights/mckinsey-quantum-technology-monitor-2026-a-commercial-tipping-point*
- *https://decentcybersecurity.eu/quantum-computing-societal-impact/*
- *https://www.qureca.com/quantum-ecosystem-future-workforce/*
- *https://quantumzeitgeist.com/the-ethical-dilemmas-of-the-quantum-age/*
- *https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-year-of-quantum-from-concept-to-reality-in-2025*
- *https://www.ubs.com/ch/en/services/guide/investments/articles/invest-in-quantum-computing.html*
- *https://www.techuk.org/resource/the-power-of-quantum-convergence-unlocking-the-uk-s-next-generation-compute-advantage.html*
- *https://www.aalto.fi/en/events/the-convergence-of-ai-and-quantum-technologies*

By: Dr Magesh Kasthuri and Dr Anand Nayyar

Dr Magesh Kasthuri is a PhD in artificial intelligence and the genetic algorithm. He currently works as a distinguished member of the technical staff (master) and chief architect at Wipro Ltd. This article expresses his views and not of the organisation he works in.

Dr Anand Nayyar is a PhD in wireless sensor networks and swarm intelligence. He works at Duy Tan University, Vietnam, and loves to explore open source technologies, IoT, cloud computing, deep learning, and cyber security.

■■ *Continue from page...50*

- Authentication of the classical communication channel.
- Cost and operational complexity.
- Vulnerabilities arising from imperfect real-world devices.
- Standardisation and interoperability.
- Need for specialised quantum networking infrastructure.

These limitations mean that practical quantum security will likely involve a hybrid architecture, combining quantum technologies with classical cryptography and emerging post-quantum cryptographic algorithms.

Quantum cryptography and post-quantum cryptography

Quantum cryptography and post-quantum cryptography (PQC) are related but fundamentally different concepts.

Quantum cryptography uses quantum phenomena to provide security, particularly for key distribution. PQC, in contrast, develops classical cryptographic algorithms designed to remain secure against both conventional and quantum computers.

This distinction is important. QKD requires specialised quantum hardware and communication infrastructure, whereas PQC can potentially be deployed through software and firmware updates to existing systems.

Therefore, the future of cybersecurity is unlikely to be purely quantum. Instead, organisations may adopt layered security strategies combining PQC, conventional cryptography, quantum key distribution, hardware security, zero-trust architectures, and AI-driven threat detection.

Quantum cryptography represents a major shift in the philosophy of information security. Instead of depending exclusively on the assumption that an attacker cannot solve a mathematical problem within a practical time, quantum communication can provide mechanisms for detecting interference based on the fundamental behaviour of quantum systems.

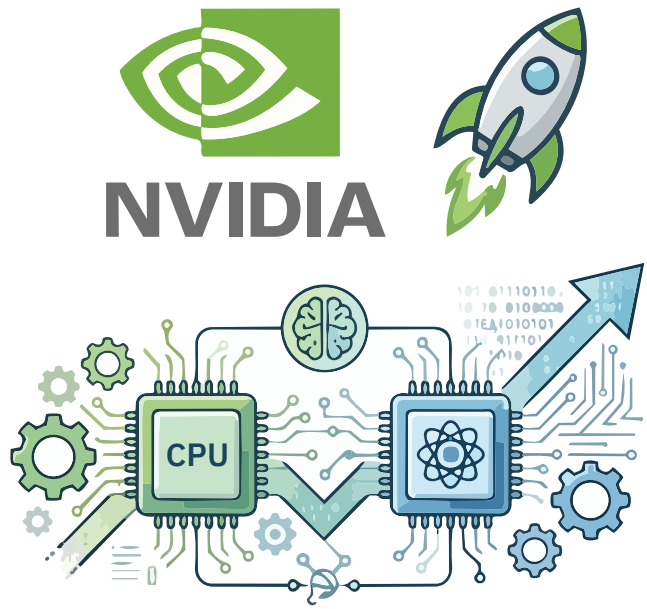
The technology is still evolving, and practical deployment remains challenging. Nevertheless, as quantum computers become increasingly capable, protecting long-lived sensitive information will become an increasingly urgent priority.

The coming decade may therefore witness a transition from classical cryptographic security towards quantum-aware cybersecurity architectures. Quantum cryptography may not replace every existing security technology, but it could become an important component of the next generation of trusted digital communication—helping secure the information infrastructure of an increasingly connected world. END

By: Dr Shubham Mahajan

The author is assistant professor at Amity University, Haryana. His research interests primarily lie in image processing, video compression, image segmentation, fuzzy entropy, data mining, machine learning, and robotics.

NVIDIA CUDA-Q: Accelerating the Development of Quantum Computing



Nvidia is among the front-runners in quantum computing...its open source platform NVIDIA CUDA-Q has a lot to offer in this field.

Quantum computing is an exciting prospect with its promise of solving complex problems at unbelievably fast speeds. Tech giants like IBM and Google are already working at developing quantum systems that make use of qubits rather than classical bits-bytes. Table 1 lists the work that some of these companies are

Table 1: Quantum computers and systems developed by corporates

Companies	Quantum technology and type
IBM, Google, Rigetti, OQC, IQM, Quantum Circuits	Superconducting
Quantinuum, Oxford Ionics, IonQ, AQT	Trapped ion
QuEra, Pasqal, Atom Computing, Infleqtion	Neutral atom
PsiQuantum, ORCA Computing, Xanadu, Quandela	Photonic
Intel, Diraq, Quantum Motion	Silicon spin
D-Wave	Quantum annealing
Microsoft	Topological
Quantum Brilliance, XeedQ	Diamond/NV centre
Alice & Bob	Cat qubits

Table 2: Global development on quantum computers by tech giants

Company	Type	Working pattern
Nvidia	Quantum software + GPU/HPC	CUDA-Q, GPU-accelerated quantum simulation, hybrid quantum-classical computing
IBM	Superconducting QPU	Qiskit and quantum processors
Google	Superconducting QPU	Cirq and quantum processors
IonQ	Trapped-ion QPU	Trapped-ion quantum computers
Quantinuum	Trapped-ion QPU	Trapped-ion quantum computers
QuEra	Neutral-atom QPU	Neutral-atom quantum computers
PsiQuantum	Photonic QPU	Photonic quantum computers
D-Wave	Quantum annealing	Quantum annealing systems
Xanadu	Photonic	Photonic quantum hardware + PennyLane
Intel	Silicon-spin	Silicon-spin quantum hardware
Microsoft	Topological	Topological/Majorana approach

doing at present in this domain.

NVIDIA CUDA-Q

Nvidia, a leader in the AI world, is now also speeding up the arrival of the quantum computing era with the release of the NVIDIA CUDA-Q *(https://developer.nvidia.com/cuda-q)*, a powerful open source platform for the development, simulation and implementation of quantum algorithms. The platform is popularly being referred to as the foundation for accelerated quantum-based supercomputing.

The key features of CUDA-Q are:

- CPU-based quantum simulation
- GPU-accelerated simulation
- HPC integration
- Hamiltonian simulation
- Hybrid quantum-classical computing
- Multi-GPU simulation
- Multiple backend support
- Python and C++ support
- QAOA and VQE
- Quantum algorithm development
- Quantum circuit programming
- Quantum hardware integration
- Quantum machine learning
- Quantum noise simulation

This open source portal can be

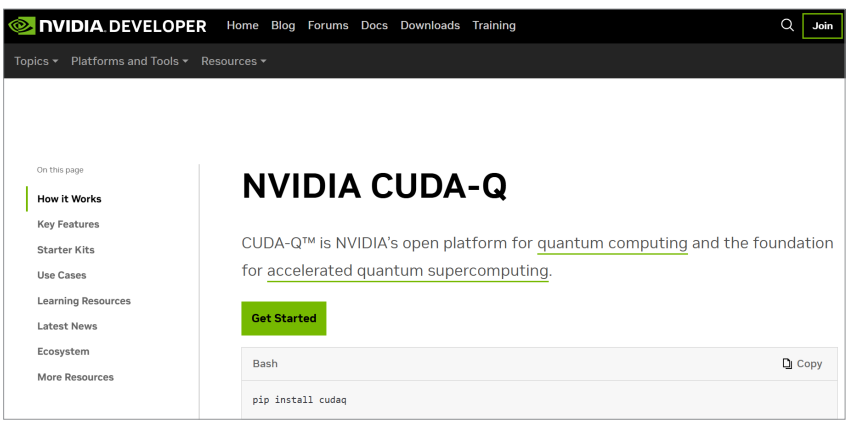


Figure 1: NVIDIA CUDA-Q framework

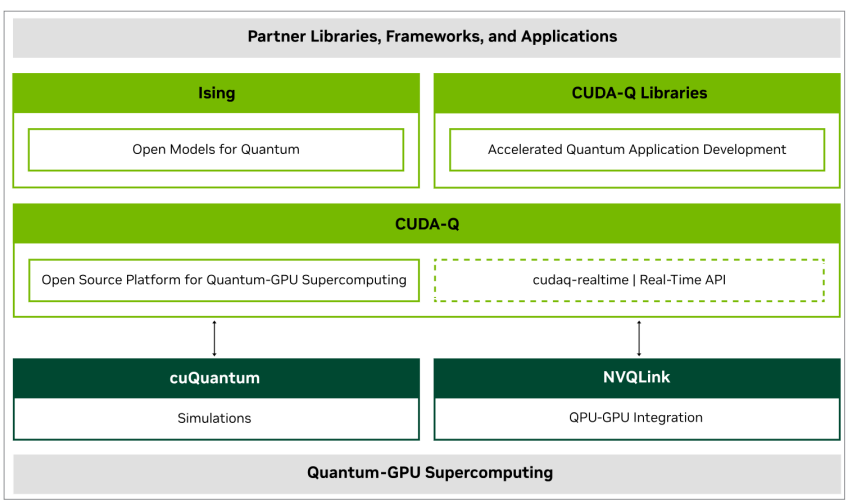


Figure 2: Technology ecosystem of NVIDIA CUDA-Q including partner libraries, frameworks and applications

used for real world, high-performance computing applications that include:

- Fault-tolerant qubits
- Solar energy prediction
- AI for algorithm design
- Divisive clustering
- Molecular generation
- Circuit synthesis

Working with CUDA-Q

To install CUDA-Q, use:

```
$ pip install cudaq
```

In addition to a local laptop, CUDA-Q can be used on Google Colab.

Here's an example of sentiment analysis using a pre-trained model and CUDA-Q package:

```
# Import Packages
import cudaq
from transformers import pipeline

# Pretrained model of sentiment for
integration with quantum
sentimentanalysis = pipeline(
    "sentiment-analysis",
    model="distilbert/distilbert-base-
uncased-finetuned-sst-2-english"
)

mytext = "I really enjoyed this course!"

myresult = sentimentanalysis(mytext)[0]

print("Text:", mytext)
print("Sentiment:", myresult)

# Conversion of sentiment confidence to a
quantum rotation
myscore = myresult["score"]

# CUDA-Q circuit
currentkernel = cudaq.make_kernel()
q = currentkernel.qalloc()


currentkernel.ry(score * 3.14159, q)
currentkernel.mz(q)

# Quantum simulation
analysiscounts = cudaq.sample(kernel,
shots_count=100)

print("Quantum result:", analysiscounts)
```

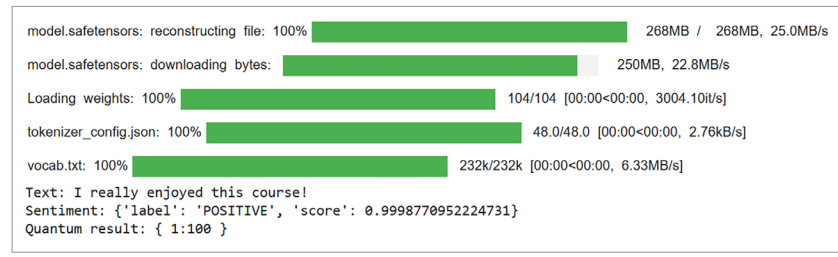


Figure 3: Output from Google Colab using CUDA-Q on sentiment analysis with pre-trained model

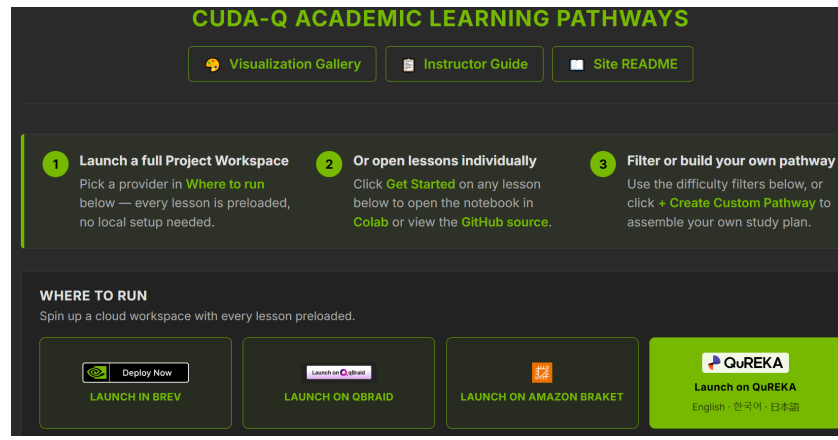


Figure 4: CUDA-Q Academic learning pathways

CUDA-Q Academic

Nvidia's CUDA-Q Academic (*https://github.com/NVIDIA/cuda-q-academic)* program is a collection of open source study materials and interactive resources for academics and researchers, which can be used freely for their research work and academic courseware. The portal offers easy to use and understand educational resources for academicians as well as researchers, with the integration of learning pathways.

Nvidia has given an excellent push to the development of quantum computing with its CUDA-Q platforms. By open sourcing them, the tech giant has made these platforms and programs highly available. END

References

- NVIDIA CUDA-Q, *https://developer.nvidia.com/cuda-q*
- NVIDIA CUDA-Q Quantum Documentation, *https://nvidia.github.io/cuda-quantum/latest/index.html*
- NVIDIA CUDA-Q Academic, *https://github.com/NVIDIA/cuda-q-academic*
- CUDA-Q Academic Learning Pathways, *https://nvidia.github.io/cuda-q-academic/learningpath.html*

By: Dr Gaurav Kumar

The author is associated with various academic and research institutes for delivering expert lectures and conducting technical workshops on the latest technologies and tools.

PennyLane: Building a Bridge Between Machine Learning and Qubits

Quantum machine learning can appear intimidating because it combines two complex subjects. PennyLane lowers that barrier by allowing developers to write a quantum circuit as a trainable component inside a familiar Python machine-learning workflow.



Quantum machine learning brings quantum circuits into the machine-learning workflow. In present-day experiments, a classical computer still prepares the data, controls training and interprets the result, while a quantum circuit acts as a specialised trainable processing component. This combination is called hybrid quantum-classical machine learning.

The goal is not to assume that quantum models are automatically better. The immediate value is the ability to investigate new ways of representing data, combining features and solving optimisation problems using open source tools.

What is PennyLane?

PennyLane is open source and developed by Xanadu. Its documentation describes it as a platform for quantum computing, quantum machine learning and quantum chemistry. The framework currently integrates with PyTorch, JAX and PennyLane NumPy, and its device abstraction allows the same high-level circuit to run on simulators or supported hardware backends. For new machine-learning projects, the current documentation recommends PyTorch or JAX; TensorFlow support in PennyLane is no longer maintained from version 0.44 onward.

Why PennyLane is needed

A beginner entering quantum computing quickly encounters Qiskit, Cirq, TensorFlow Quantum, Q#, pyQuil, and several hardware-specific tools. All of them can describe quantum programs, but they were not created around the same question. Some concentrate on building and compiling circuits for a particular hardware ecosystem. Others focus on hardware research, simulation, or a programming language. PennyLane places a different question at the centre: how can a quantum circuit be trained together with an ordinary machine learning model?

That distinction matters. In a conventional neural network, a framework such as PyTorch keeps track of operations, calculates gradients, and adjusts weights. PennyLane extends this style of differentiable programming to quantum circuits. A circuit can receive data, expose trainable gate parameters, return measurements and participate in the same optimisation loop as a classical layer.

How PennyLane works

PennyLane receives classical input values, uses them to control quantum gates, executes a circuit on a selected device, measures the resulting qubit states and returns ordinary numerical values. Those values can then be used by PyTorch or JAX to calculate a prediction and train the complete model.

A PennyLane program has four important parts:

- *Device:* The simulator or quantum processor on which the circuit executes.
- *Quantum function:* The sequence of gates applied to named qubits, called wires in PennyLane.
- *QNode:* A wrapper that connects the quantum function to a device and to an automatic-differentiation interface.
- *Optimizer or ML framework:* PyTorch, JAX or another supported interface that updates trainable parameters.

The QNode is the key bridge. From the machine learning side it behaves like a numerical function or layer. Internally, it prepares qubits, applies gates, executes the circuit, and returns measurement values. PennyLane then determines how those values change when the circuit parameters change.

The hybrid processing path

Consider two Iris measurements: petal length and petal width. After preprocessing, each value controls a rotation gate on one qubit. The rotations do not place decimal centimetre values into qubits as binary digits. Instead, each value tells the circuit how far to change a qubit's state. Additional rotations contain trainable parameters, and an entangling gate allows the two qubits to interact. Finally, the circuit measures the qubits and returns ordinary numbers. A small classical layer converts those numbers into a class probability as shown in Figure 1.

Training repeatedly follows the same loop: encode a flower, run the circuit, measure it, predict a class, compare the prediction with the known label, calculate the error, and update both the quantum and classical parameters. This is a hybrid model because the quantum circuit supplies a trainable transformation while the classical computer handles data preparation, optimisation and the final decision.

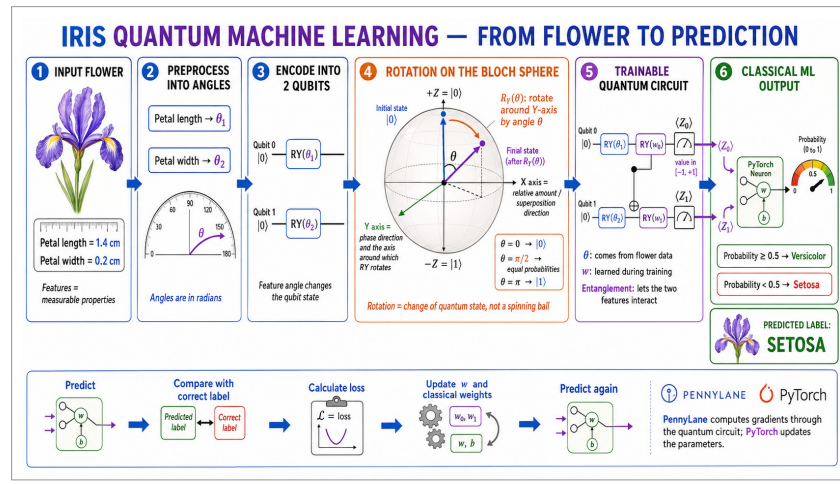


Figure 1: The complete hybrid QML path, from Iris measurements to qubit encoding, trainable circuit, measurement and classical prediction

Major strengths of PennyLane

Differentiable quantum programming: PennyLane's defining strength is gradient-based training across quantum and classical operations. Developers can place a circuit inside a PyTorch module and allow the optimiser to update circuit angles alongside ordinary neural-network weights. This makes small quantum neural networks and variational algorithms comparatively natural to prototype.

Backend flexibility: The device interface separates the circuit description from its execution backend. A learner can begin with the built-in default.qubit simulator and later experiment with a plugin for another simulator or supported provider. Hardware portability is never completely automatic—devices differ in gates, connectivity, noise and execution rules—but the abstraction reduces the amount of application code tied to one vendor.

Familiar Python and ML integration: PennyLane deliberately feels close to scientific Python. Quantum gates are written as Python operations, measurements are returned like numerical outputs, and QNodes can accept PyTorch tensors or JAX arrays. This is especially helpful for ML practitioners who want to learn quantum circuits without abandoning their

Framework	Primary emphasis	ML integration	Hardware orientation	Natural choice when...
PennyLane	Differentiable quantum and hybrid programming	Strong PyTorch/JAX integration	Multi-backend through devices and plugins	The quantum circuit must behave like a trainable ML layer
Qiskit	Broad quantum workflows, compilation and execution	QML is available within a wider ecosystem	Strong IBM Quantum integration	You need IBM hardware, transpilation or general quantum development
Cirq	Circuit construction, manipulation and NISQ hardware detail	Possible directly or through other libraries	Hardware-aware, especially Google-oriented research	You need fine circuit and device control
TensorFlow Quantum	Hybrid QML inside TensorFlow using Cirq	Native TensorFlow/Keras workflow	Cirq-based execution model	Your project is already centred on TensorFlow and Cirq
Q#	Quantum algorithm expression and resource reasoning	Not primarily an ML-layer framework	Microsoft/Azure quantum ecosystem	You want a dedicated quantum language or resource estimation

Table 1: Framework choice should follow the intended workflow rather than brand popularity

existing optimisation workflow.

A strong educational and research workflow: The framework can draw circuits, inspect operations, simulate ideal or noisy execution and calculate gradients without requiring access to physical hardware. It is therefore useful in classrooms, notebooks, and early-stage research, where understanding the model is more important than immediately deploying it on a quantum processor.

A comparison with other quantum frameworks

No framework is universally best. The useful comparison is not whether one can create a circuit—they all can—but which workflow receives first-class attention (Table 1).

Qiskit is broader than PennyLane and is the natural entry point to IBM's ecosystem. Cirq exposes circuit and device details suited to NISQ experiments. TensorFlow Quantum builds hybrid models by bringing Cirq circuits into TensorFlow. PennyLane is particularly attractive when the central requirement is end-to-end optimisation of a quantum circuit together with classical ML components.

Important use cases of PennyLane are:

- Quantum neural networks for classification and regression experiments.
- Variational quantum algorithms in which gate parameters are iteratively optimised.
- Quantum kernels and feature maps for exploring alternative representations of data.
- Quantum chemistry, including molecular Hamiltonians and variational energy estimation.
- Combinatorial optimisation prototypes using hybrid objective functions.
- Generative and probabilistic quantum models.
- Education, circuit visualisation, gradient demonstrations and backend comparison.

These are research and experimentation use cases. Present-day QML should not be presented as a guaranteed replacement for classical machine learning. Small demonstrations are valuable for learning the workflow and evaluating hypotheses, but performance, noise, data-loading cost, and fair classical baselines remain essential considerations.

```
C:\Users\Admin-SSD>mkdir pennylane-iris

C:\Users\Admin-SSD>cd pennylane-iris

C:\Users\Admin-SSD\pennylane-iris>python -m venv .venv-iris

C:\Users\Admin-SSD\pennylane-iris>dir
 Volume in drive C has no label.
 Volume Serial Number is 92EE-A284

 Directory of C:\Users\Admin-SSD\pennylane-iris

09/02/2026  06:51 PM    <DIR>          .
09/02/2026  06:51 PM    <DIR>          ..
09/02/2026  06:51 PM    <DIR>          .venv-iris
               0 File(s)              0 bytes
               3 Dir(s)   9,399,570,432 bytes free

C:\Users\Admin-SSD\pennylane-iris>.venv-iris\Scripts\activate
```

Figure 2: Python virtual environment creation

```
(.venv-iris) C:\Users\Admin-SSD\pennylane-iris>python -m pip install pennylane torch scikit-learn matplotlib
Collecting pennylane
  Downloading pennylane-0.45.1-py3-none-any.whl.metadata (10 kB)
Collecting torch
  Downloading torch-2.13.0-cp312-cp312-win_amd64.whl.metadata (39 kB)
Collecting scikit-learn
  Downloading scikit_learn-1.9.0-cp312-cp312-win_amd64.whl.metadata (11 kB)
Collecting matplotlib
  Downloading matplotlib-3.11.1-cp312-cp312-win_amd64.whl.metadata (80 kB)
Collecting scipy (from pennylane)
  Downloading scipy-1.18.1-cp312-cp312-win_amd64.whl.metadata (61 kB)
Collecting networkx (from pennylane)
  Using cached networkx-3.6.1-py3-none-any.whl.metadata (6.8 kB)
Collecting rustworkx>=0.14.0 (from pennylane)
  Downloading rustworkx-0.18.1-cp310-abi3-win_amd64.whl.metadata (10 kB)
Collecting autograd<1.9 (from pennylane)
  Downloading autograd-1.8.0-py3-none-any.whl.metadata (7.5 kB)
Collecting appdirs (from pennylane)
  Downloading appdirs-1.4.4-py2.py3-none-any.whl.metadata (9.0 kB)
Collecting autoray==0.8.4 (from pennylane)
```

Figure 3: PennyLane installation

The Iris dataset experiment

The Iris dataset contains 150 samples, four numerical measurements and three flower species. To keep the first experiment easy to inspect, we use 100 samples from two species—Setosa and Versicolor—and two features: petal length and petal width. Seventy-five samples are used for training and 25 for testing. One feature is angle-encoded on each of the two qubits.

The circuit uses feature rotations, two trainable entangling layers and measurements from both qubits. PennyLane converts the circuit into a PyTorch layer. A classical linear layer and sigmoid output then produce the Versicolor probability.

The setup begins by creating and activating a dedicated Python virtual environment, which keeps the experiment and its dependencies isolated from other projects as shown in Figure 2. PennyLane, PyTorch, Scikit-learn and Matplotlib are then installed using pip as shown in Figure 3. These libraries provide the tools required for quantum-circuit construction, model training, dataset preparation, and result visualisation.

Preparing the Iris dataset

The program loads Iris from Scikit-learn, keeps Setosa and Versicolor, selects petal length and petal width, and

divides the data into stratified training and test sets. Standardisation places both features on a comparable scale.

Encoding flower features into qubits

After preprocessing, petal length controls the rotation of Qubit 0 and petal width controls the rotation of Qubit 1. Different flowers therefore create different qubit states. The values are used as rotation instructions rather than being stored in the qubits as ordinary binary numbers.

Building the quantum circuit

The circuit first performs feature encoding, then applies trainable rotations and entanglement. Entanglement allows the circuit to process the relationship between the two flower measurements. Finally, both qubits are measured to produce two ordinary numerical outputs.

Connecting PennyLane with PyTorch

PennyLane's TorchLayer converts the QNode into a PyTorch-compatible layer. A classical linear layer receives the two quantum measurements, and a sigmoid output converts them into the probability of the flower being Versicolor.

Training the hybrid QML model

For every epoch, the model processes all training flowers, calculates prediction loss, and uses the Adam optimiser to update both the trainable quantum angles and the classical layer parameters. PennyLane supplies gradients through the circuit while PyTorch manages the optimisation loop.

Here's the complete Python Program:

```
import numpy as np
import pennylane as qml
import torch
from torch import nn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix

SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)

# 1. Load two classes and two petal features
iris = load_iris()
keep = iris.target < 2
X = iris.data[keep][:, [2, 3]]
y = iris.target[keep]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=SEED, stratify=y
)

# 2. Scale the data, then bound it into convenient angles
scaler = StandardScaler()
X_train = np.arctan(scaler.fit_transform(X_train))
X_test = np.arctan(scaler.transform(X_test))

X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)

# 3. Create a two-qubit simulator
dev = qml.device("default.qubit", wires=2)

# 4. Define the trainable quantum circuit
@qml.qnode(dev, interface="torch", diff_method="backprop")
def circuit(inputs, weights):
    qml.AngleEmbedding(inputs, wires=[0, 1], rotation="Y")
    qml.BasicEntanglerLayers(weights, wires=[0, 1])
    return [qml.expval(qml.PauliZ(w)) for w in [0, 1]]

# 5. Convert the QNode into a PyTorch layer
quantum_layer = qml.qnn.TorchLayer(
    circuit, {"weights": (2, 2)}
)

model = nn.Sequential(
    quantum_layer,
    nn.Linear(2, 1),
    nn.Sigmoid()
)

# 6. Train quantum and classical parameters together
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.05)

for epoch in range(1, 51):
    optimizer.zero_grad()
    probability = model(X_train)
    loss = loss_fn(probability, y_train)
    loss.backward()
    optimizer.step()

    if epoch == 1 or epoch % 10 == 0:
        pred = (probability >= 0.5).int().numpy().ravel()
        acc = accuracy_score(y_train.numpy().ravel(), pred)
        print(f"Epoch {epoch:02d}: loss={loss.item():.4f}, "
              f"training accuracy={acc:.3f}")

# 7. Evaluate unseen test flowers
model.eval()
with torch.no_grad():
    probability = model(X_test)

prediction = (probability >= 0.5).int().numpy().ravel()
truth = y_test.int().numpy().ravel()

print("Test accuracy:", accuracy_score(truth, prediction))
print("Confusion matrix:\n", confusion_matrix(truth, prediction))
```

Understanding the code without quantum mathematics

The data section selects only Setosa and Versicolor, and keeps petal length and width. Standardisation prevents the two measurements from being treated on incompatible scales. The arctangent bounds the transformed values so they remain convenient rotation inputs.

The device named default.qubit is a simulator, so no physical quantum computer or cloud account is required. AngleEmbedding rotates the two qubits according to the two flower features. BasicEntanglerLayers supplies trainable rotations and an interaction between qubits. The two measurements returned by the QNode become two ordinary numbers.

TorchLayer packages the QNode as a PyTorch module. From this point onwards, PyTorch can place the quantum layer inside a sequential model, calculate binary cross-entropy loss and update all trainable parameters with Adam. The developer does not need to create a separate optimisation routine for the quantum part.

Experimental results

With the fixed random seed and the stated 75/25 stratified split, the training loss fell from 0.7097 in the first epoch to 0.1058 after 50 epochs. Training accuracy reached 100 per cent by epoch 10. On the held-out test set, all 25 flowers were classified correctly: 13 Setosa and 12 Versicolor as shown in Figure 4.

Understanding the results

This trained model correctly classified all 25 test flowers in this split as shown in Figure 5. This does not mean that PennyLane always produces perfect models, that every Iris split will produce the same result, or that quantum computing has outperformed classical machine learning. Setosa and Versicolor are easily separated using petal measurements, and classical models can also solve this small task very well.

The value of this experiment is educational: it exposes every stage of a hybrid QML workflow in a problem that runs quickly on a laptop. A serious comparison would require repeated splits, multiple random seeds, classical baselines, noise experiments, runtime and resource measurements,

```
(.venv-iris) C:\Users\Admin-SSD\pennylane-iris>python iris_pennylane_qml.py
Experiment: Iris Setosa vs Versicolor
Training samples: 75 | Test samples: 25
Features: petal length and petal width | Qubits: 2
0: ─╭AngleEmbedding(M0)─╭BasicEntanglerLayers(M1)─┤  <Z>
1: ─╰AngleEmbedding(M0)─╰BasicEntanglerLayers(M1)─┤  <Z>

M0 =
tensor([0.7420, 0.9123])
M1 =
tensor([[0., 0.],
        [0., 0.]])
Epoch 01 | loss=0.7097 | train accuracy=0.493
Epoch 10 | loss=0.5473 | train accuracy=1.000
Epoch 20 | loss=0.3481 | train accuracy=1.000
Epoch 30 | loss=0.2166 | train accuracy=1.000
Epoch 40 | loss=0.1478 | train accuracy=1.000
Epoch 50 | loss=0.1058 | train accuracy=1.000

Final test accuracy: 1.000 (25/25)
Confusion matrix:
[[13  0]
 [ 0 12]]
Results saved in: C:\Users\Admin-SSD\pennylane-iris\iris_qml_results
```

Figure 4: Results of the experiment

Experiment setting	Observed value
Training/test samples	75 / 25
Qubits	2
Epochs	50
Final training loss	0.1058
Test accuracy	100% (25/25)
Confusion matrix	13 Setosa correct; 12 Versicolor correct; 0 errors

Table 2: Reproducible result obtained with the listed split, seed and training configuration

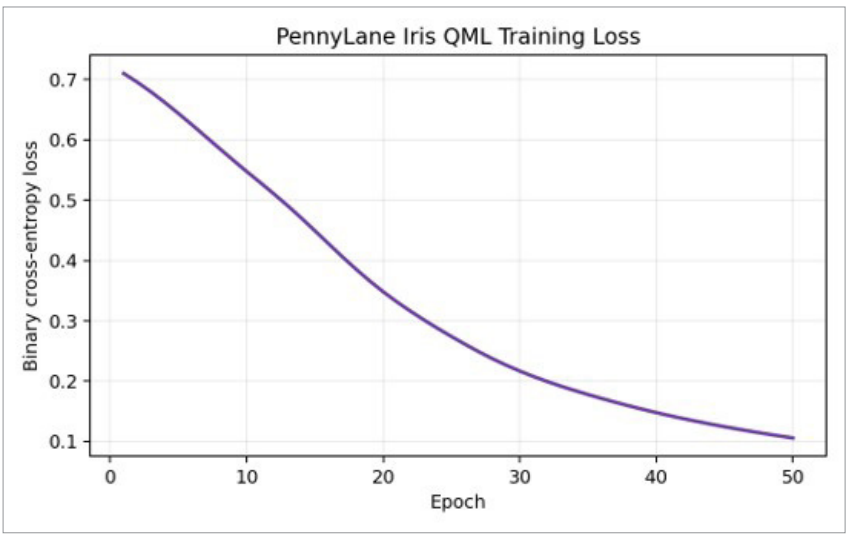


Figure 5(a): QML training loss across 50 epochs

and preferably a problem for which the quantum representation has a defensible motivation.

Limitations of the experiment

Quantum simulators become expensive as the qubit count increases because a general state-vector simulation grows exponentially. Physical quantum processors introduce noise, restricted qubit connectivity, queueing and finite measurement shots. Classical data must also be encoded into the circuit, and that cost can outweigh any benefit if the representation is not chosen carefully.

Hybrid QML is therefore an active research area rather than a drop-in performance upgrade. PennyLane's strength is that it makes controlled experimentation easier: the same conceptual model can be studied with multiple differentiation techniques, simulators, noise models and supported backends while remaining connected to familiar ML tooling.

Possible extensions are:

- Include Virginica and replace the binary output with a three-class classifier.
- Encode all four Iris features using four qubits or a data re-uploading circuit.
- Compare different embeddings, entanglement patterns and circuit depths.
- Add simulated noise and study whether accuracy and convergence change.
- Compare the hybrid model with logistic regression, SVM and a small classical neural network.
- Execute a suitably compiled version on an accessible hardware backend and compare it with ideal simulation.
- Use repeated train/test splits and report average accuracy with variability, not only one split.

PennyLane occupies a clear position in the open source quantum ecosystem. Qiskit offers a broad IBM-centred development and execution environment; Cirq provides strong circuit and NISQ-device abstractions; TensorFlow Quantum integrates Cirq with TensorFlow. PennyLane distinguishes itself by treating differentiable quantum circuits as trainable building blocks within hybrid applications.

For a newcomer to QML, that design provides a gentle and productive path: begin on a simulator, encode a small dataset, build a shallow circuit, connect it to PyTorch, observe the training loop and only then explore deeper circuits, noise and hardware. The Iris example does not demonstrate quantum advantage, but shows something equally important for a beginner—the complete route from classical data to qubits and back to a usable prediction. END

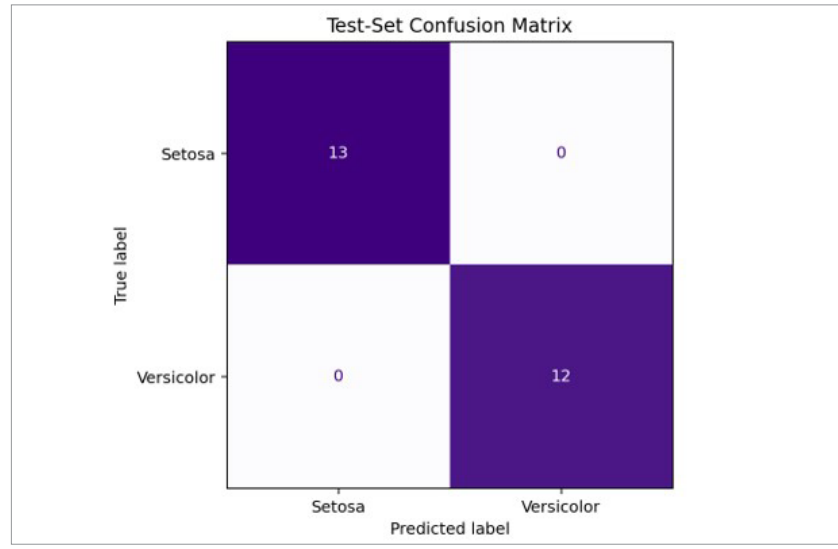


Figure 5(b): Held-out test confusion matrix

References

- PennyLane documentation, *https://docs.pennylane.ai/en/stable/*
- PennyLane PyTorch interface, *https://docs.pennylane.ai/en/stable/introduction/interfaces/torch.html*
- PennyLane gradients and interfaces, *https://docs.pennylane.ai/en/stable/introduction/interfaces.html*
- IBM Quantum and Qiskit documentation, *https://quantum.cloud.ibm.com/docs*
- Google Quantum AI Cirq documentation, *https://quantumai.google/cirq*
- TensorFlow Quantum, *https://www.tensorflow.org/quantum*
- Scikit-learn Iris dataset, *https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_iris.html*

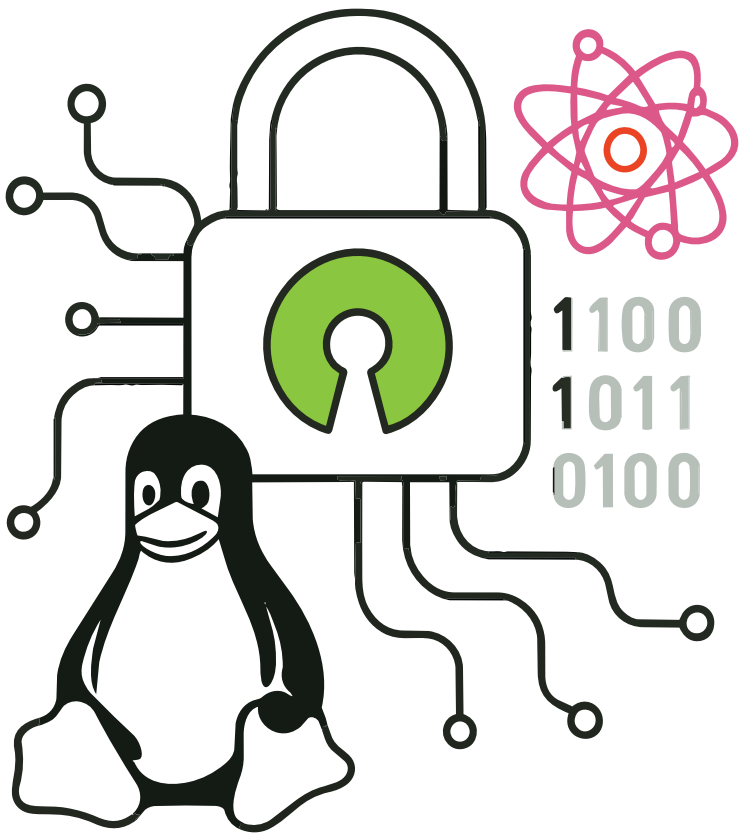
By: S. Ratan Kumar and Valli Kumari Vatsavayi

S. Ratan Kumar is associate professor in the Department of Computer Science and Engineering (AI & ML) at Anil Neerukonda Institute of Technology & Sciences(A) (ANITS), Visakhapatnam. His interests include machine learning, cybersecurity, data analytics, high-performance computing, and the practical teaching of emerging open source technologies.

Valli Kumari Vatsavayi is professor of Computer Science and Systems Engineering, Andhra University. Her research interests include quantum computing, security, privacy, machine learning, and forensics. She has to her credit over 200 research papers.

A Practical Guide to Securing Open Source in the Quantum Era

Quantum computing threatens today's cryptography. Open source must lead the migration to post-quantum cryptography (PQC) — and do it without breaking the ecosystem.



Quantum computing promises breakthroughs in medicine, materials, and AI. But the same machines that accelerate discovery will also shatter the encryption protecting the internet. Algorithms like RSA, ECDSA, and Diffie-Hellman — the backbone of TLS, VPNs, and secure messaging — will fall quickly to Shor's algorithm once large, error-corrected quantum computers arrive.

The timeline is uncertain: some predict a decade, others longer. But the threat is already here. Attackers can 'harvest now, decrypt later' — storing encrypted traffic today and waiting until quantum machines can unlock it. That means yesterday's TLS sessions, VPN tunnels, and encrypted emails could be exposed tomorrow.

Open source sits at the centre of this risk. It powers the servers, registries, operating systems, and libraries the world depends on. Proprietary vendors can push controlled upgrades; open source projects are decentralised and slower to migrate. If upstream cryptographic libraries don't evolve, every downstream user remains vulnerable.

What is post-quantum cryptography?

Post-quantum cryptography (PQC) is classical software designed to resist quantum attacks. Unlike Quantum Key Distribution (QKD), PQC runs on ordinary hardware and can be deployed today.

The main families include:

- ***Lattice-based schemes:*** Fast, compact, and built on learning with errors (LWE).
- ***Hash-based signatures:*** Conservative, relying only on hash function security.

- ***Code-based cryptography:*** Based on decoding random linear codes, resilient for decades.
- ***Multivariate and isogeny-based schemes:*** Innovative but some have already fallen to attacks.

In 2024, US-based NIST (National Institute of Standards and Technology) published the first PQC standards:

- ***ML-KEM (FIPS 203)****:* Key encapsulation, replacing ECDH.
- ***ML-DSA (FIPS 204)****:* Digital signatures, replacing RSA and ECDSA.
- ***SLH-DSA (FIPS 205)****:* Hash-based signatures, ideal for long-term archival.

These are no longer research projects. They are ready for integration into TLS, SSH, S/MIME, and code signing.

Why open source must lead

Open source is the foundation of global cryptography. Migration must begin here for three reasons:

- ***It builds the plumbing.*** OpenSSL, GnuTLS, WireGuard, OpenSSH, and the Linux kernel supply primitives used everywhere. Without PQC support upstream, downstream projects cannot secure themselves.
- ***It moves slowly.*** Enterprise distributions and embedded firmware remain in production for years. PQC must land early so maintainers can test, certify, and deploy before quantum risk becomes urgent.
- ***It sets the example.*** When open source integrates PQC, publishes benchmarks, and runs interoperability events, it shows governments and vendors how practical migration works. Patterns established here become industry norms.

Why migration is harder than it looks

Replacing cryptography is not a simple version bump. PQC introduces real engineering challenges:

- ***Bigger payloads:*** Keys and signatures are larger, affecting certificate chains, buffers, and low-bandwidth devices.
- ***Protocol negotiation:*** TLS, IKEv2, SSH, and Noise need agreed code points before PQC handshakes work.
- ***Hybrid modes:*** Many deployments will combine classical ECDH with PQC KEMs to hedge risk.
- ***Testing and certification:*** PQC requires unit, fuzz, and interoperability tests, plus FIPS or Common Criteria certification in regulated sectors.
- ***Human factors:*** Engineers must learn new names, failure modes, and configuration options without losing confidence in their systems.

A maintainer's roadmap

If your project touches cryptography, here's how to start.

Step 1: Audit your crypto surface: Catalogue every protocol, file format, and library using RSA, ECDSA, ECDH, or DSA. Identify secrets worth protecting from 'harvest now, decrypt later'.

Step 2: Adopt trusted libraries: Do not roll your own. Use vetted upstream libraries:

- *liboqs* (Open Quantum Safe) — broad C toolkit.
- *OpenSSL 3.2+* — ML-KEM and ML-DSA via provider architecture.
- *BouncyCastle* — PQC support for Java and C#.
- Reference code at pq-crystals.org and discussions on pqc-forum.

Step 3: Enable provider architecture: Use engines like OpenSSL 3 or Java's JCA to plug in new algorithms without rewriting applications.

Step 4: Build crypto agility: Keep algorithm identifiers, key formats, and handshake logic configurable. Avoid hard-coding key sizes or names.

Step 5: Test hybrids and fallbacks: Ensure your system handles peers without PQC, hybrid handshakes, and downgrade attempts. Document preferred cipher suites and deprecate weak ones.

Step 6: Engage in standards and interop: Join IETF working groups, NIST PQC forums, and Open Quantum Safe events. Interoperability testing ensures your code works across vendors.

Common pitfalls to avoid

- ***Waiting for 'quantum tomorrow':*** The real risk is traffic already being harvested.
- ***Rolling your own primitives:*** PQC math is subtle; use vetted libraries.
- ***Ignoring performance:*** Larger payloads affect latency and bandwidth. Test early.
- ***Hard-coding algorithms:*** Build crypto agility into configuration.
- ***Forgetting training:*** Migration

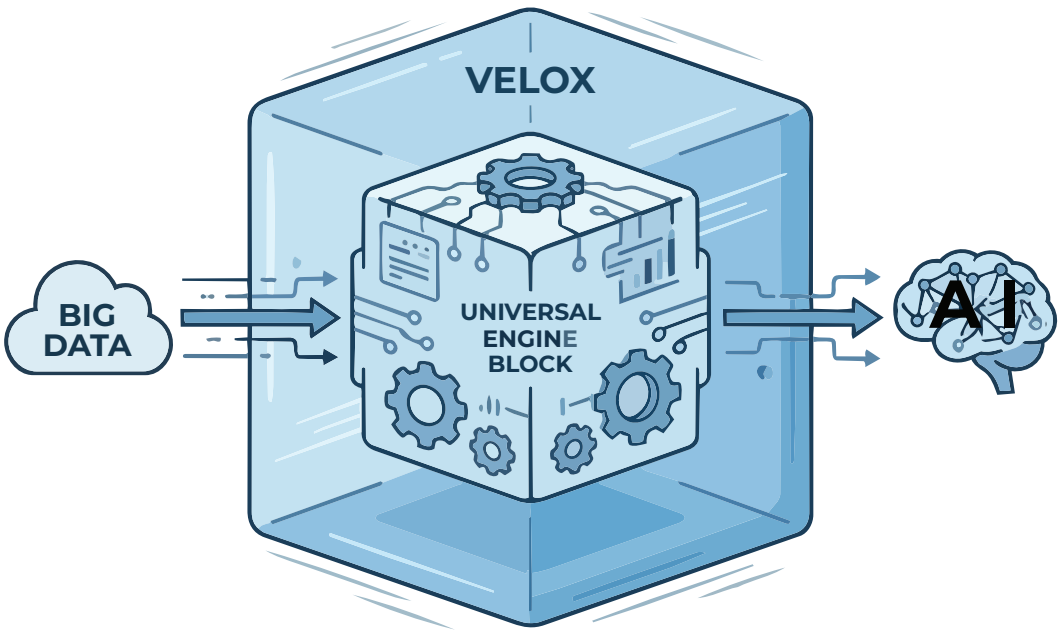
PQC standards at a glance

Standard	Purpose	Replaces	Status	Recommended use case
ML-KEM	Key encapsulation	ECDH	FIPS 203 (2024)	TLS, VPNs
ML-DSA	Digital signatures	RSA, ECDSA	FIPS 204 (2024)	Certificates, code signing
SLH-DSA	Hash-based signatures	RSA, ECDSA	FIPS 205 (2024)	Archival, long-term signatures
SHA-3 / SHAKE	Hashing/XOFs	SHA-2 (in PQC modes)	Standardised	PQC-compatible hashing

■■ ***Continue to page...72***

Inside Velox, the 'Universal Engine Block' for Big Data and AI

Velox, the high-performance open source unified execution engine developed by Meta, is enabling excellent speed, consistency and efficiency in data processing.



Imagine a world where every car manufacturer had to build a completely new engine from scratch for every type of vehicle they produced -- one engine specifically designed for a nimble sports car, another for a heavy-duty pickup truck, and a completely different one for a family SUV. Historically, this is exactly how the tech industry built data management systems, creating dozens of specialised, siloed query engines tailored for specific workloads like interactive analytics, batch processing, or machine learning.

To solve this massive fragmentation and duplicated engineering effort, Meta has developed Velox: a revolutionary, open source C++ 'universal engine block' for the data ecosystem. Rather than constantly reinventing the wheel, developers can now drop Velox's highly optimised, high-performance execution library into any system. Whether it is powering lightning-fast interactive analytics (like Presto), large-scale batch processing (like Apache Spark), or the machine learning pipelines (like PyTorch), Velox unifies the core mechanics of data processing to deliver state-of-the-art speed, unmatched consistency, and massive engineering efficiency across the entire industry.

By the early 2010s, Meta was operating one of the world's largest data platforms, processing data from billions of users while supporting thousands of engineers running analytics, A/B tests, and machine learning workloads. As data volumes grew, performance became a constant challenge.

Meta relied heavily on Presto, the distributed SQL engine it created in 2012. Although Presto was highly efficient for a JVM-based system, the JVM itself became a bottleneck at Meta's scale. Garbage collection, JIT compilation overhead, and limited access to modern CPU features such as SIMD made it increasingly difficult to improve query performance.

Prestissimo: The native execution experiment

To overcome these limitations, Meta launched Prestissimo (Presto C++), a project to rewrite only Presto's execution engine in native C++, while keeping the SQL parser and query planner unchanged.

During development, the team realised that the core components they were building—columnar memory structures, vectorised execution,

high-performance hash tables, and spill-to-disk infrastructure—were not specific to Presto. These same building blocks were needed by Spark, stream processing systems, and ML feature engineering pipelines.

Rather than implementing the same execution engine separately for every system, Meta extracted these reusable components into a standalone library. That library became Velox, a shared, high-performance execution engine that could power multiple data processing frameworks.

Why was Velox needed?

The fragmentation of data processing engines: As data workloads evolved from traditional SQL analytics to real-time stream processing and AI pipelines, the industry responded by building specialised execution engines such as Presto, Spark, Trino, Druid, ClickHouse, and DuckDB. While each engine targeted different use cases, they all implemented the same core execution primitives independently.

Every project built its own columnar in-memory data format, expression evaluation engine, hash tables for joins and aggregations, spill-to-disk infrastructure, parquet and ORC readers, SQL functions such as SUM(), COUNT(), string, date, and array operations.

This duplicated years of engineering effort across organisations. More importantly, every bug fix, optimisation, or hardware improvement had to be implemented separately in every engine, making innovation slow and expensive.

Inconsistent SQL semantics: Independent implementations also led to inconsistent behaviour across engines. Even simple functions such as *substr()*, *round()*, or *date_trunc()* often produced different results depending on the execution engine. At Meta, engineers found multiple implementations of the same string function, each using different indexing rules and error-handling behaviour.

These semantic differences made migrating workloads difficult and undermined trust in analytical results. The same SQL query could produce different answers across platforms, forcing data teams to spend significant time debugging correctness issues instead of building applications.

The JVM performance ceiling: Most large scale analytical engines, including Presto, Spark, and Hive, relied on the Java Virtual Machine (JVM). Although the JVM provides excellent portability and developer productivity, it introduces limitations for high-performance analytical execution:

- Garbage collection causes unpredictable latency during large queries.
- Java object overhead increases memory consumption and reduces cache efficiency.
- Limited control over memory management prevents advanced techniques such as memory arenas and efficient spill-to-disk.
- Access to modern CPU features like SIMD vector instructions (SSE, AVX, AVX2) is significantly more constrained than in native C++.

As datasets continued to grow, these limitations became a major barrier to improving query performance.

Velox architecture

Before diving into the architecture, it is essential to understand a critical design boundary: Velox is an execution backend library, NOT a standalone database server.

Velox deliberately does not include a SQL parser, query optimiser, distributed RPC scheduler, or cluster manager. It leaves logical planning and scheduling to host engines like Presto or Spark, which pass a serialised physical plan tree (velox::core::PlanNode) down to Velox worker nodes for ultra-fast native C++ execution.

Layer 1: Calling system (Integration layer)

- *What it does:* The host framework (Presto Coordinator or Spark Driver) acts as the brain. It parses your SQL query, optimises the execution plan, and assigns tasks across the cluster.
- *Handoff:* The calling system compiles the query into a fully optimised query plan (velox::core::PlanNode tree) and hands it down to Velox worker nodes for execution.

Layer 2: Execution framework and operators

- *Task/Pipeline execution framework:* Spawns non-blocking worker

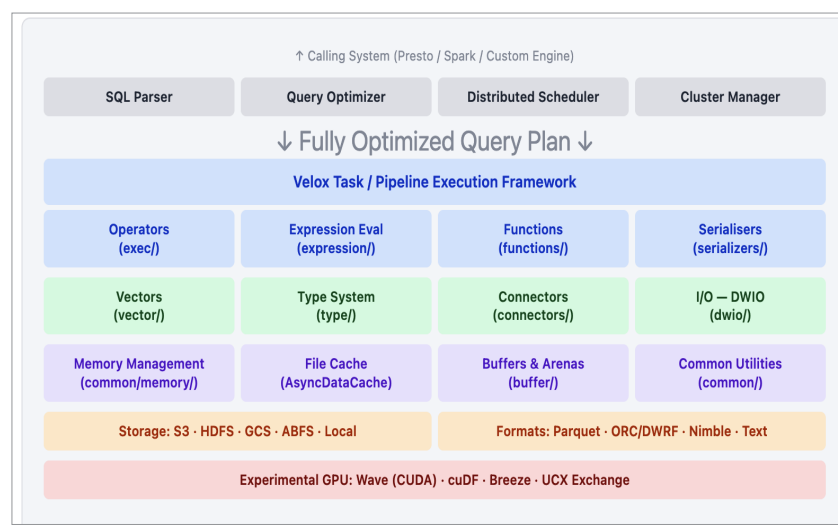


Figure 1: Architecture: How Velox is built

threads (Drivers) that stream data batches through execution pipelines.
- *Operators (exec/):* Physical processing units performing table scans (TableScan), filtering (Filter), projections (Project), multi-threaded joins (HashJoin), aggregations (HashAgg), sorting (Sort), and disk spilling (Spiller).
- *Expression Eval (expression/):* Compiles SQL math and string expressions into fast execution loops (compileExpressions()) with Common Subexpression Elimination (CSE).
- *Functions (functions/):* Built-in library of scalar functions, aggregate functions (facebook::velox::exec::Aggregate), and window functions.
- *Serializers (serializers/):* Encodes vector batches (PrestoVectorSerde, UnsafeRowSerializer) for ultra-fast network exchange between cluster nodes.

Layer 3: Core data structures and I/O
- *Vectors (vector/):* Columnar memory representation. Data is packed in vertical columns (FlatVector, zero-copy DictionaryVector, deferred LazyVector), enabling CPUs to process hundreds of values per clock cycle using SIMD instructions.
- *Type system (type/):* Manages primitive types and complex structures (arrays, maps, structs).
- *Connectors (connectors/):* Read/write data from various data sources (HiveConnector, TpchConnector).
- *I/O — DWIO (dwio/):* I/O file readers for Apache Parquet, ORC/DWRF, Meta's fast columnar format (Nimble), and Text.
- *Filter pushdown:* DWIO readers evaluate filters directly inside storage files, skipping non-matching row groups before downloading data into RAM.

Layer 4: Memory, cache and infrastructure
- *Memory management (common/memory/):* Tracks RAM usage in a strict hierarchy (MemoryPool). If a query approaches memory limits, MemoryReclaimer automatically spills temporary data to disk, preventing Out-Of-Memory (OOM) crashes.
- *File Cache (AsyncDataCache):* Asynchronously caches hot metadata and file pages in RAM, avoiding operating system page-cache thrashing.
- *Buffers and arenas (buffer/):* Manages contiguous memory blocks (Buffer, BufferView, StreamArena, MmapArena).
- *Common utilities (common/):* Process logging, metrics (TraceContext), and configuration.

Layer 5: Storage and formats

Interfacing layer connecting Velox to cloud object stores (AWS S3, Azure ADLS, Google Cloud Storage), distributed filesystems (HDFS, Local Disk), and open table formats (Hive, Apache Iceberg, Delta Lake).

Layer 6: Experimental acceleration and exchange
- *GPU acceleration:* Offloads heavy vector compute to Nvidia GPUs using Wave (Velox CUDA framework) or RAPIDS cuDF.
- *Breeze:* Low-level SIMD vector primitives.
- *UCX exchange:* High-speed RDMA network shuffle plugin for ultra-fast server-to-server data transfers.

The journey of a query inside Velox

The journey of a query inside Velox is streamlined, highly vectorized, and occurs entirely on a single host. Because Velox is an execution library rather than a full database, it does not parse SQL or perform global optimisation. It sits strictly on the local data plane, executing a physical query plan that has already been fully optimised by a parent engine like Presto or Spark.

Here is the step-by-step journey of how data moves through Velox.

Query hand-off and translation: The parent engine's coordinator parses the query, optimises the plan, and hands a specific physical query plan fragment to Velox on the local host. Velox ingests this physical plan and maps it to a *Task* (the local unit of execution).

Task and Driver execution: The task is divided into one or more *Pipelines*, each consisting of a sequence of operators that can process data continuously. Every pipeline is executed by one or more *Drivers*, which are Velox's execution objects scheduled on a shared worker thread pool. If a Driver becomes blocked, for example while waiting for I/O, it yields execution so that other Drivers can continue making progress, keeping CPU resources fully utilised.

> ***Note:*** Drivers are execution objects, not operating system threads. They are scheduled on worker threads managed by the host engine.

Pipelined scan and expression evaluation: Once execution begins, data enters the pipeline through storage connectors and file readers.

Velox reads formats such as Parquet, ORC/DWRF, Nimble, and Text using its DWIO layer. During scanning, it applies optimisations such as column pruning and predicate pushdown, ensuring that only the columns and row groups required by the query are read from storage.

The data is represented internally as columnar vectors and streamed through the execution pipeline in batches. Expression evaluation -- including filters, projections, and arithmetic operations—is performed over these vector batches, allowing the engine to leverage SIMD instructions and cache-friendly memory access for significantly higher throughput than row-by-row execution.

Memory arbitration and spill handling: As operators execute,

intermediate states such as hash tables, aggregation buffers, and sorting structures are allocated from Velox's hierarchical *MemoryPool* system.

When memory usage approaches configured limits, the *MemoryArbitrator* coordinates memory reclamation. Spill-capable operators can write intermediate state to local storage, allowing large analytical queries to continue executing even when their working set exceeds available RAM.

This approach minimises out-of-memory failures while maintaining predictable query execution.

Result delivery and network shuffle: After the final operator completes processing, the resulting columnar vectors are prepared for delivery.

For single-node execution, the vectors are returned directly to the calling engine. For distributed execution, the output may be serialised into engine-specific formats such as Presto Pages or Spark's UnsafeRow representation before being exchanged with downstream workers or returned to the parent engine.

From the perspective of Presto or Spark, Velox acts as a highly optimised execution backend that transforms an optimised physical plan into the final query results.

Where is Velox being used?

Velox is under active development but is already being integrated into more than a dozen different data systems at Meta and across the broader open source ecosystem. By providing a unified set of high-performance components, Velox is bridging the gaps between historically siloed workloads.

Here are some of the most prominent data management systems currently leveraging Velox.

Interactive analytics: Prestissimo (Presto C++): Presto is a distributed query engine that powers massive interactive SQL analytic workloads. Traditionally, Presto worker nodes ran on the Java Virtual Machine (JVM), which was historically prone to operational issues and expensive garbage collection procedures. Through a project called Prestissimo, developers are replacing these traditional Java workers with a C++ process powered entirely by Velox. Prestissimo provides a drop-in replacement that takes a Presto plan fragment from a Java coordinator and hands it directly to Velox for native execution, completely bypassing the JVM on the worker nodes and delivering massive performance improvements.

Batch processing: Apache Spark (Spruce and Gluten): For heavy-duty, large scale batch processing and ETL (Extract, Transform, Load) workloads, Velox is deeply integrated into Apache Spark. This is achieved through two primary initiatives:

- *Spruce:* Meta's internal integration, which uses Spark's built-in script transform interface to offload query plan fragments to an external C++ process running Velox.
- *Apache Gluten:* An open source project originally created by Intel. Gluten acts as a bridge, creating a Java Native Interface (JNI) API based on the Apache Arrow data format and Substrait query plans. This decouples Spark's JVM from the execution engine, allowing Spark to seamlessly delegate compute-intensive tasks to Velox.

Artificial intelligence and machine learning: One of Velox's most ambitious capabilities is bridging the gap between traditional data analytics and artificial intelligence.

- *TorchArrow:* Velox serves as the backend execution engine for TorchArrow, a Python DataFrame library for data preprocessing in PyTorch. Because data preprocessing can consume up to 50% of the resources in an ML workload, using Velox standardises these operations and provides tremendous efficiency wins.
- *F3:* Meta's feature engineering framework, F3, is also unifying its execution engine with Velox. This allows developers to seamlessly extract and generate features for ML algorithms across both offline (batch) and real-time environments.

Real-world performance: Velox in action

To prove that Velox C++ design translates to real-world speed, Velox has been subjected to rigorous benchmarks across multiple execution environments. The results demonstrate that by replacing traditional Java execution layers or leveraging specialised hardware, Velox delivers staggering performance gains and massive infrastructure cost savings.

The CPU benchmark: Presto Java vs Velox C++ (Prestissimo): The first major test of Velox's efficiency was executed at Meta, comparing the traditional Java based Presto engine against Prestissimo (Presto workers powered by Velox's C++ engine).

- *Standard TPC-H (3TB Scale):* In a warm-cache cluster of 80 nodes, CPU-bound queries saw near-order-of-magnitude improvements. Query 1 (Q1) achieved an 8.4x wall-time speedup (dropping from 42 seconds to just 5 seconds), while Query 6 (Q6) saw a 9x speedup (dropping from 9 seconds to 1 second). Shuffle-heavy queries like Q13 and Q19 also doubled in speed.
- *Shadowing real production traffic:* To move past synthetic benchmarks, Meta replayed real production traffic from various interactive analytical tools. The Velox-powered C++ engine delivered an average speedup of 6x to 7x, with a significant portion of queries running over 10x faster than the traditional Java engine.
- *The 3x server consolidation win:* Perhaps the most impactful result for infrastructure teams is cluster capacity. In shadowing tests, the Velox-based stack supported identical production workloads with equal or better performance using

3x fewer servers, allowing Meta to consolidate a 60-server Java cluster down to just 20 Velox nodes.

The distributed batch benchmark: Apache Spark + Gluten + Velox on AWS EKS: To evaluate how Velox scales for heavy-duty batch and ETL processing, Amazon Web Services (AWS) ran a TPC-DS 3TB benchmark on Amazon EKS using Graviton4 nodes (r8gd.12xlarge). This compared native Spark SQL against Spark accelerated by Apache Gluten and Velox.

- *Overall speedup:* The Gluten + Velox stack was 39% faster overall, dropping total suite runtime from 3,650.56 seconds to 2,239.93 seconds (a 1.63x overall speedup). Out of 103 queries, 81 queries saw significant improvements.
- *Deep dive on Query 93 (4.36x speedup):* In native Spark, Query 93 took 207.9 seconds, which Velox slashed to 47.7 seconds. By replacing Spark's traditional SortMergeJoin with Velox's ShuffledHashJoinExecTransformer and eliminating pre-join Sorts:
  - The hot 200-task join stage ran ~12x faster (dropping from 1m 16s to 6.5s). Peak per-task memory fell from 3.3GB (on-heap) to just 80MB (off-heap). JVM Garbage Collection time collapsed from 3.6 seconds to a mere 35 milliseconds.
  - Serialised shuffle data footprint was reduced by 24% (compressing from 214.1GB to 162.2GB), drastically reducing disk and network pressure.
- *The whole-stage codegen nuance:* The benchmark also revealed interesting structural trade-offs. A small subset of queries regressed (most notably q72, which ran slower). Because Velox does not implement SortMergeJoin, it materialised millions of small columnar batches across downstream operators. In contrast, native Spark's whole-stage codegen fused the entire stream into a single JVM function without materialisation. This highlights that while Velox is incredibly fast, understanding your query patterns and join types is key to avoiding edge-case regressions.

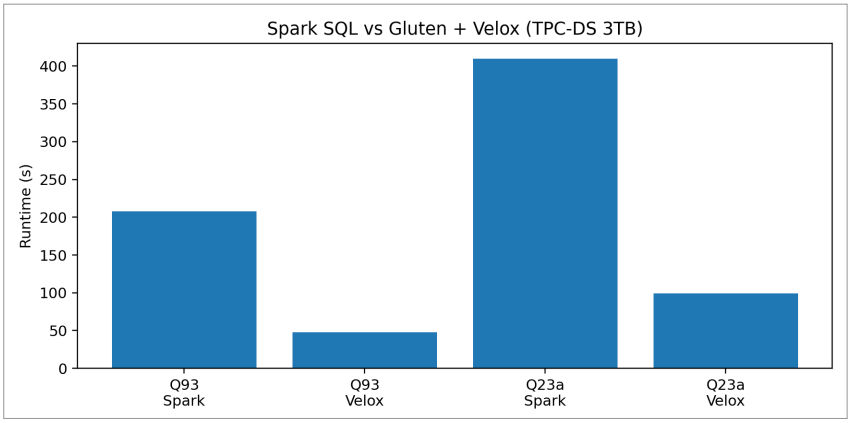


Figure 2: Distributed batch benchmark: Apache Spark + Gluten + Velox on AWS EKS

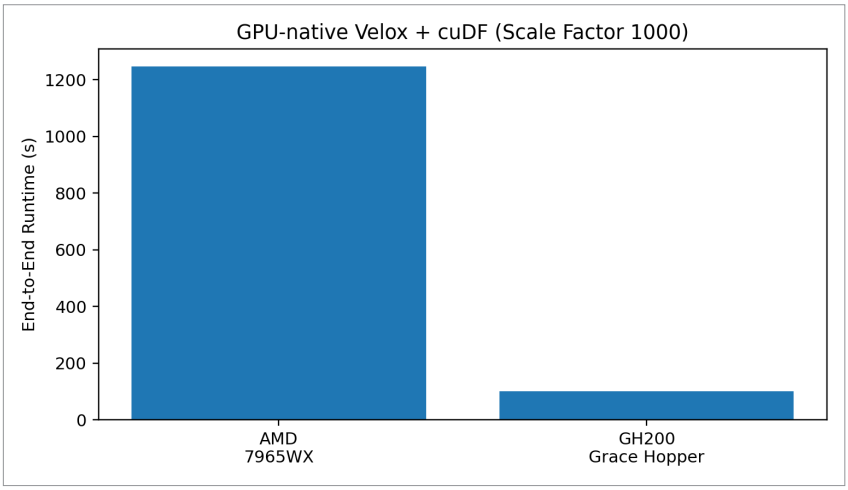


Figure 3: GPU-native Velox with NVIDIA cuDF

The hardware acceleration benchmark: GPU-native Velox with NVIDIA cuDF: By integrating NVIDIA's cuDF library directly into Velox, query plans from engines like Presto can be translated directly into GPU-accelerated pipelines.

- *Unmatched single-node performance:* On a TPC-H Scale Factor 1,000 dataset, a single node Presto C++ run on an AMD 7965WX CPU took 1,246 seconds. Shifting the entire Presto query plan to the GPU via Velox and cuDF reduced that runtime to 133.8 seconds on an NVIDIA RTX Pro 6000, and to a staggering 99.9 seconds on an NVIDIA GH200 Grace Hopper Superchip, representing an over 12x speedup.
- *Multi-GPU scaling with NVLink:* For distributed query execution, Velox supports a UCX based Exchange operator that runs the entire execution pipeline on the GPU. On an 8-GPU NVIDIA DGX A100 node, using high-bandwidth NVLink for data exchange yielded a >6x speedup compared to using Presto's baseline HTTP exchange.

Whether running on standard CPUs, modern Graviton processors, or state-of-the-art GPUs, Velox's vectorised C++ architecture delivers on its promises. By democratising elite

runtime optimisations, Velox allows organisations to slash query latencies, eliminate JVM garbage collection overhead, and achieve up to a 3x reduction in hardware footprints.

The future of unified execution

Velox represents a fundamental paradigm shift in database infrastructure. By extracting common execution primitives, vectorized data layouts, expression compilers, memory pools, and file readers into a single, highly optimised C++ library, Velox delivers on three key promises:

- ***Unmatched efficiency:*** Slashes query latencies, eliminates JVM garbage collection pauses, and delivers up to a 3x reduction in cluster server counts.
- ***Semantic consistency:*** Guarantees that SQL functions produce the exact same results whether executed in Spark SQL, Presto C++, or PyTorch.
- ***Hardware acceleration ready:*** Seamlessly scales from x86 and Graviton4 CPUs to NVIDIA GH200 Grace Hopper GPUs via cuDF. END

References

- GitHub repository, *https://github.com/facebookincubator/velox*
- Official documentation, *https://facebookincubator.github.io/velox/*
- NVIDIA blog, *https://developer.nvidia.com/blog/accelerating-large-scale-data-analytics with-gpu-native-velox-and-nvidia-cudf/*
- Apache Spark with Gluten+Velox benchmarks, *https://awslabs.github.io/data-on eks/docs/benchmarks/spark-gluten-velox-benchmark*

By: Saurabh Mahawar

The author is a developer relations engineer at IBM. He has built and nurtured developer communities, organised large scale events, and led strategic initiatives to drive developer engagement and adoption of technology.

Acknowledgment: The author would like to thank Naveen Kumar Mahadevuni, software engineer, IBM, for his support in writing and reviewing this article.

■■ *Continue from page...66*

PQC tools

Tool/Library	Role	Notes
liboqs	Reference C toolkit	Broad algorithm coverage
OpenSSL 3.2+	TLS/SSL provider	ML-KEM and ML-DSA support
BouncyCastle	Java / C#	Wide PQC support
pq-crystals.org	Reference implementations	ML-KEM and ML-DSA
pqc-forum	Community discussion	Standards updates
Go/Rust/Python/JS bindings	Language support	Rapidly maturing

- is not just for cryptographers — operators and developers need guidance too.

A deadline, not an option

The PQC transition is not optional — it is a deadline. Attackers are already harvesting encrypted traffic. When quantum machines arrive, only projects that migrated early will remain secure.

Open source must lead. Audit dependencies, adopt PQC libraries, build crypto agility, and engage in standards. The projects that act now will define the secure infrastructure of the quantum era. Those that wait risk leaving their users exposed. END

References

- NIST, 'Announcing Draft Federal Information Processing Standards (FIPS) for Post-Quantum Cryptography', 2023–2024
- Open Quantum Safe Project, *https://openquantumsafe.org/*
- IETF, 'Hybrid Post-Quantum Key Encapsulation' and related TLS/SSH drafts
- Bernstein, D.J. and Lange, T., 'Post-Quantum Cryptography', *https://pqcrypto.org/*
- ENISA, 'Post-Quantum Cryptography: Current state and quantum mitigation'
- Google Chrome Enterprise, 'PQC and the need for crypto agility', *https://security. googleblog.com/*
- Cloudflare, 'The future of PQC in TLS: A first look at ML-KEM', *https://blog. cloudflare.com/*
- PQShield, 'Post-Quantum Cryptography: State of the Art and Practical Migration'
- Trail of Bits, 'PQC migration in open source software', *https://blog.trailofbits.com/*
- SANS Institute, 'Preparing for the post-quantum cryptographic transition'
- Mozilla NSS and OpenSSL PQC Roadmaps

By: Kousik Maiti

The author has over 18+ years of industry experience. His areas of interest include Linux systems administration, security and forensics.

The Benefits and Uses of Open Source LLMs

There are several open source alternatives available today for expensive commercial large language models. Engineering teams can easily leverage open-weight models to achieve tangible ROI across production environments.



Akki Bharath Goud, cloud architect, OVHcloud

Let's begin with one important question: why do we need open source LLMs? The answer: we have well-trained open source LLMs that are trained on huge datasets, whose weights the community has publicly shared. They possess tremendous knowledge (whether they have 70 billion parameters, 20 billion parameters, or another scale) acquired through training processes that utilise significant GPU resources. Because these are available on public repositories/platforms, you can download, utilise, and reuse them without paying extra fees beyond your inference costs. While commercial models exist, open source LLMs can perform these same jobs effectively.

The evolution of enterprise AI and market trends

Earlier, AI was often treated as an experimental phase where every department was asked to start at least one AI project, regardless of its ultimate utility. That trend has changed; leaders now focus on whether an AI project aligns with their broader strategic goals.

Furthermore, the technology is moving very quickly. In 2025, the focus was on genAI applications, LLMs, and building RAG systems. Moving into 2026, the discussion has shifted towards AI agents and agentic workflows, and autonomous agents capable of managing lower-end operational tasks. Because proof-of-concept projects no longer move into production without a clear ROI, organisations need accessible AI tools. The easiest way to consume AI without massive training investments is through open source LLMs, as thousands are readily available for research and deployment in public repositories/platforms like Hugging Face.

Hardware requirements and infrastructure strategy

Regarding infrastructure and usage, there are multiple ways to leverage off-the-shelf, pre-trained models. One approach is to download the model weights directly. You must evaluate the required hardware resources -- some large language models have high memory demands or specific GPU requirements, such as an H100. Models such as gpt-oss-120b are effective for reasoning and agentic tasks. Platforms such as Hugging Face allow you to research model architectures, review code snippets, and carry out cost comparisons.

Infrastructure providers such as OVHcloud host open source models and offer ready-to-use inference endpoints built on their public cloud infrastructure, featuring top-performing models selected directly from open source repositories. OVHcloud's public cloud is built on the open source OpenStack platform, reflecting its commitment to technology sovereignty. The key advantage is that you pay strictly for inference costs rather than for model licensing. A full list of available endpoints can be found in OVHcloud's AI Endpoints catalogue.



Deployment architectures

Architecturally, you can deploy these models in a few distinct ways:

- Running directly on a public cloud GPU instance with a standard operating system.
- Containerising the open source LLM into a Docker image and deploying it within a Kubernetes cluster.
- Consuming the models directly through managed API endpoints.

Open source LLMs and their use cases

Speech-to-text: Models like Whisper Large V3 provide reliable audio transcription capabilities comparable to proprietary models.

Vision-language models (VLMs) and object detection: By using open source models like Qwen VLM, you can pass an image, process the visual features, and request structured outputs like JSON formatting to categorise elements, people, or landscape components within the frame.

RAG systems and content generation pipelines: You can construct complete retrieval-augmented generation (RAG) pipelines using strictly open source components. For instance, combining a vector database for context retrieval with an Ollama-hosted instruct model yields accurate, structured responses. This RAG core can be extended into a complex multi-model pipeline: the generated content is formatted into structured JSON, an open source text-to-speech model such as NVIDIA TTS model Nvr-tts-en-us converts that JSON into narration voice files, and a script built on python-pptx generates the matching slide deck — with tools like FFmpeg finally combining the audio and visual outputs into full video content suitable for enterprise training or knowledge bases.

Maintaining output quality in these systems relies on the underlying dataset. Your knowledge base must prioritise data quality and proper cleaning, such as structured processing of technical documentation across domain-specific topics, rather than simply relying on sheer volume before vector embedding.

Another practical use case is interacting with models directly from your terminal. Open source utilities like SHAI allow you to SSH into an AI system. By linking this CLI tool to an open-weight model like DeepSeek R1, you can use standard natural language to trigger workflows or deploy sub-agents right from your local machine, paying only for the API tokens consumed.

Similarly, developer tools like kilocode can connect to open source models, such as gpt-oss, for code generation, giving you a viable alternative to proprietary coding assistants.

Production considerations: Security, governance and fine tuning

When moving these setups into production, two operational areas are critical:

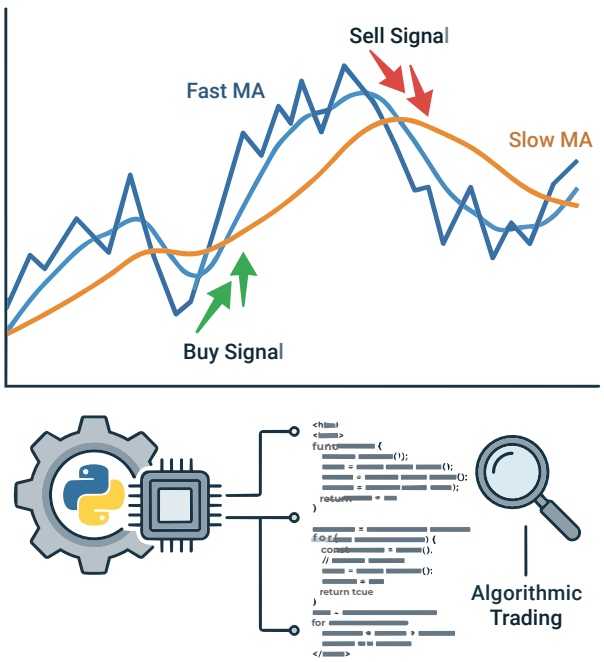
- ***Data governance and guardrails:*** Hosting models yourself, whether on-premises, via private cloud, or via public cloud, keeps your data strictly in-house. To manage this safely, you need to implement guardrails that filter content and enforce internal policies both before a prompt reaches the model and after it generates a response.
- ***Infrastructure security:*** Standard self-hosting rules apply here: you need strict firewall configurations and port management. If you are deploying via Kubernetes, that security footprint must expand to include container image scanning and strict network policies to control the traffic moving between your containers.

Fine tuning large models carries high computational costs and should be carefully evaluated against your requirements. For most standard enterprise use cases, an RAG architecture provides the necessary domain context without the need to retrain weights. Fine tuning is typically reserved for highly specialised domains such as medical imaging. When fine tuning is necessary, smaller open source models, typically under 8 to 10 billion parameters, combined with parameter-efficient methods like LoRA or QloRA, optimise resource utilisation. END

This article is based on the talk at AI DevCon India 2026 titled 'Practical Applications of Open Source LLMs', delivered by Akki Bharath Goud, cloud architect, OVHcloud. It has been transcribed and curated by Jiya Jay Singh, journalist at the EFY Group.

Moving Average Crossover Strategy: Algorithmic Trading with Python

Traders on the stock market can take advantage of the Moving Average Crossover Strategy to maximise the returns on their investments.



The Moving Average Crossover Strategy is probably the easiest and most effective trading algorithm that can help teach algorithmic trading. It uses only the price history of a stock and several lines of Python to cover nearly all the concepts used when designing a trading Python script strategy: market data, technical indicators, trading signals, positions, profits, risks, transaction costs, and performance. The main idea behind this strategy is to make use of the correlation between short-term and long-term moving averages to determine changes in market trends.

What is a moving average?

The moving average smoothens out the fluctuations in the prices of a stock by calculating the average price for 'n' periods preceding the current one. The formula used to compute the SMA (simple moving average) is:

$$SMA_t = \frac{P_t + P_{t-1} + \cdots + P_{t-n+1}}{n}$$

…where P_t represents the closing price at time 't'. For example, if the closing prices for five days are:

Day 1	100
Day 2	102
Day 3	101
Day 4	105
Day 5	107

…the 5-day moving average is:

$$SMA_5 = \frac{100 + 102 + 101 + 105 + 107}{5} = 103$$

With each new price, the old data point is dropped, and the new data point is added. This process can be carried out quite easily in Python using the Pandas' rolling function; a 20-day moving average will be computed as follows:

```
df["SMA20"] = df["Close"].rolling(20).mean()
```

…where 'df' is a historical market DataFrame.

Why use two moving averages?

A single moving average tells us about the general direction of the market, but two moving averages allow us to identify the changes in a trend. For example, if we consider 20 days and 50 days for short-term and long-term moving averages, the 20-day moving average will react relatively quickly to changes in price, while the 50-day moving average will change more slowly. The relationship between them gives us a simple trading rule:

20-day SMA > 50-day SMA → Bullish → Hold

…and:

20-day SMA < 50-day SMA → Bearish → Exit

The actual crossing of the two averages is called a crossover.

Golden Cross and Death Cross

Two popular names for the Moving Average Crossover Strategy are Golden Cross and Death Cross. When the shorter MA crosses above the longer MA the situations are known as Golden Crosses and are usually seen as a positive signal. On the other hand, when the shorter MA crosses below the longer MA the incidents known as Death Crosses and are usually seen as a negative signal.

The periods used in the calculation of the moving average are not always the same and can range from 5/20 and 10/30, to 20/50 and 50/200.

The crossover between the short-term and long-term moving averages helps us to determine the positioning strategy for trading. This has been summarised in Figure 1.

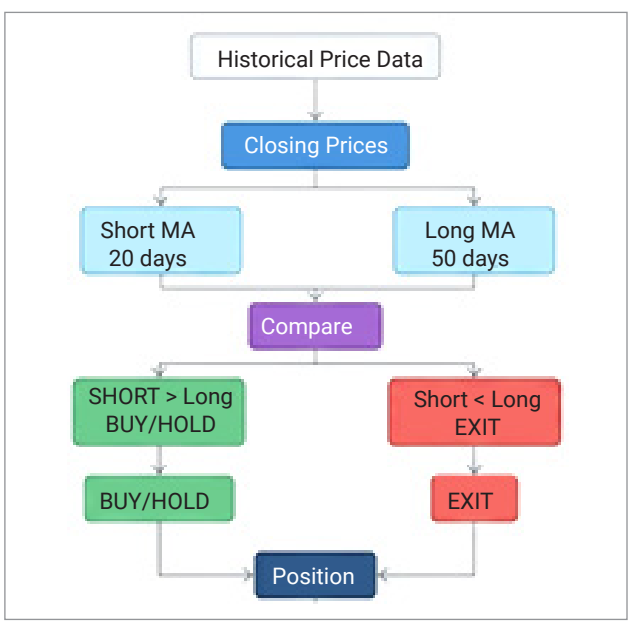


Figure 1: Positioning strategy

This simple workflow forms the basis of our Python implementation.

Obtaining market data with Python

For a study, historical market data can be obtained using *yfinance*.

```
import yfinance as yf
import pandas as pd
import matplotlib.pyplot as plt
ticker = "TCS.NS"
df = yf.download(
    ticker,
    start="2020-01-01",
    end="2025-01-01",
    auto_adjust=True,
    progress=False,
    threads=False
)
print(df.head())
```

Close	High	Low	Open	Volume
TCS.NS	TCS.NS	TCS.NS	TCS.NS	TCS.NS
1831.11	1844.879	1819.621	1831.447	1354908
1822.704	1841.543	1815.566	1841.543	2380752
1859.029	1877.91	1828.069	1828.069	4655761
1858.86	1880.401	1848.258	1862.703	3023209
1863.422	1870.856	1844.795	1858.903	2429317

Before proceeding further, always check that the download was successful:

```
if df.empty:
raise ValueError("No market data was downloaded.")
```

The resulting DataFrame may have MultiIndex columns depending upon the *yfinance* package version. In case it's needed, get the data for a single ticker/attribute using the *xs()* method.

```
if isinstance(df.columns, pd.MultiIndex):
      df = df.xs(ticker, axis=1, level="Ticker")
```

Then remove all null recordsets:

```
df = df.dropna()
```

Calculating the moving averages

We now calculate the short and long moving averages for two moving window sizes -- 20 and 50:

```
short_window = 20
long_window = 50
df[“SMA20”] = (
    df[“Close”]
    .rolling(short_window)
    .mean()
)
df[“SMA50”] = (
    df[“Close”]
    .rolling(long_window)
    .mean()
)
print(
    df[
        [“Close”, “SMA20”, “SMA50”]
    ].tail()
}
```

Close	SMA20	SMA50
3892.482	4042.155	3910.172
3882.796	4034.544	3910.964
3878.838	4030.816	3912.046
3873.203	4025.599	3913.421
3813.598	4017.13	3913.343

Plotting the moving averages

Though browsing the record set with a short- and long-term moving average shows new feature columns, visualisation plays a special role in understanding the price movement pattern.

```
plt.figure(figsize=(15, 7))
plt.plot(
    df.index,
    df[“Close”],
    label=”Close Price”,
    linewidth=1.2
)
plt.plot(
    df.index,
    df[“SMA20”],
    label=”20-Day SMA”,
    linewidth=2
)
plt.plot(
    df.index,
    df[“SMA50”],
    label=”50-Day SMA”,
    linewidth=2
)
plt.title(
    f”{ticker} – Moving Average Crossover”
)

plt.xlabel(“Date”)
plt.ylabel(“Price”)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Figure 2: Moving average crossover for a 20/50-day period

To study the crossover, it's important to look for the points at which the short-term average crosses the long-term average (Figure 2).

Detecting Buy and Sell crossover points

The critical difference lies in whether the price is above/below the long-term moving average or crosses the moving average.

A Buy crossover occurs when:

Yesterday: SMA20 <= SMA50
Today: SMA20 > SMA50

In Python this can be implemented as:

```
buy = (
    (df["SMA20"] > df["SMA50"]) &
    (df["SMA20"].shift(1) <= df["SMA50"].shift(1))
)
```

Similarly, a Sell crossover occurs when:

Yesterday: SMA20 >= SMA50
Today: SMA20 < SMA50

This can be implemented as:

```
    sell = (
    (df["SMA20"] < df["SMA50"]) &
    (df["SMA20"].shift(1) >= df["SMA50"].shift(1))
)
```

The *shift(1)* is important because it allows us to compare today's relationship with yesterday's relationship.

Creating the trading position

We now convert the moving average relationship into a position by identifying the Buy/Sell signals:

```
df["Signal"] = 0
df.loc[
    df["SMA20"] > df["SMA50"],
    "Signal"
] = 1
```

Thus:

```
Signal = 1 → Holding the stock

Signal = 0 → Not holding the stock
```

We can visualise the position:

```
print(
    df[
        ["Close", "SMA20", "SMA50", "Signal"]
    ].tail(20)
)
```

Close	SMA20	SMA50	Signal
4007.268	3890.032	3879.731	1
4055.371	3908.205	3881.22	1
4157.49	3931.148	3885.056	1
4140.215	3945.39	3888.494	1
4146.408	3959.419	3891.992	1

Marking all crossovers on the chart

After knowing the position indicator, it is possible to integrate the price, the moving averages, the Buy indicators, and the Sell indicators into a visual report on the entire trading process.

```
fig, ax = plt.subplots(figsize=(15, 7))
ax.plot(
    df.index,
    df["Close"],
    label="Close Price",
    linewidth=1.2
)
ax.plot(
    df.index,
    df["SMA20"],
    label="20-Day SMA",
    linewidth=2
)
ax.plot(
    df.index,
    df["SMA50"],
    label="50-Day SMA",
    linewidth=2
)
# BUY signals
ax.scatter(
    df.index[buy],
    df.loc[buy, "Close"],
    marker="^",
    s=120,
    label="BUY",
    zorder=5
)
# SELL signals
```



Figure 3: 20/50 moving average crossover

```
ax.scatter(
    df.index[sell],
    df.loc[sell, "Close"],
    marker="v",
    s=120,
    label="SELL",
    zorder=5
)
ax.set_title(
    f"{ticker} – 20/50 Moving Average Crossover"
)
ax.set_xlabel("Date")
ax.set_ylabel("Price")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

The chart now provides a visual representation of the positioning algorithm (Figure 3).

How the strategy helps to hold stock

Finally, for a clear demonstration of the Hold and Sell periods along the entire moving average cycle we can shade the holding periods of the stock. This is particularly useful for understanding the trading pattern.

```
holding = df["Signal"] == 1
```

The shading of the periods during which the strategy helps to hold the stock is implemented here and shown in Figure 5:

```
fig, ax = plt.subplots(figsize=(15, 7))
ax.plot(
    df.index,
    df["Close"],
    label="Close Price",
    linewidth=1.2
)
ax.plot(
    df.index,
    df["SMA20"],
    label="20-Day SMA",
    linewidth=2
)
ax.plot(
    df.index,
```

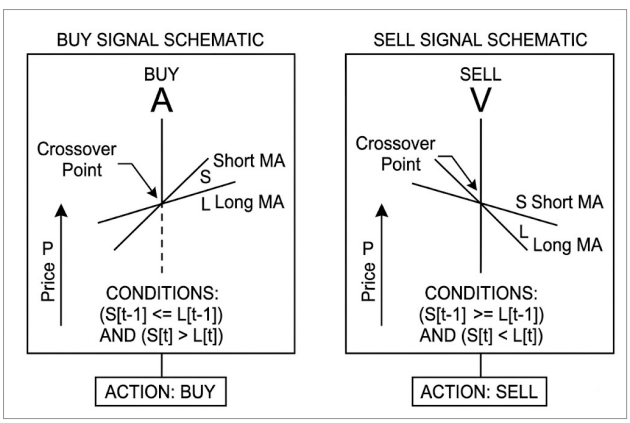


Figure 4: Buy signal and Sell signal schematics

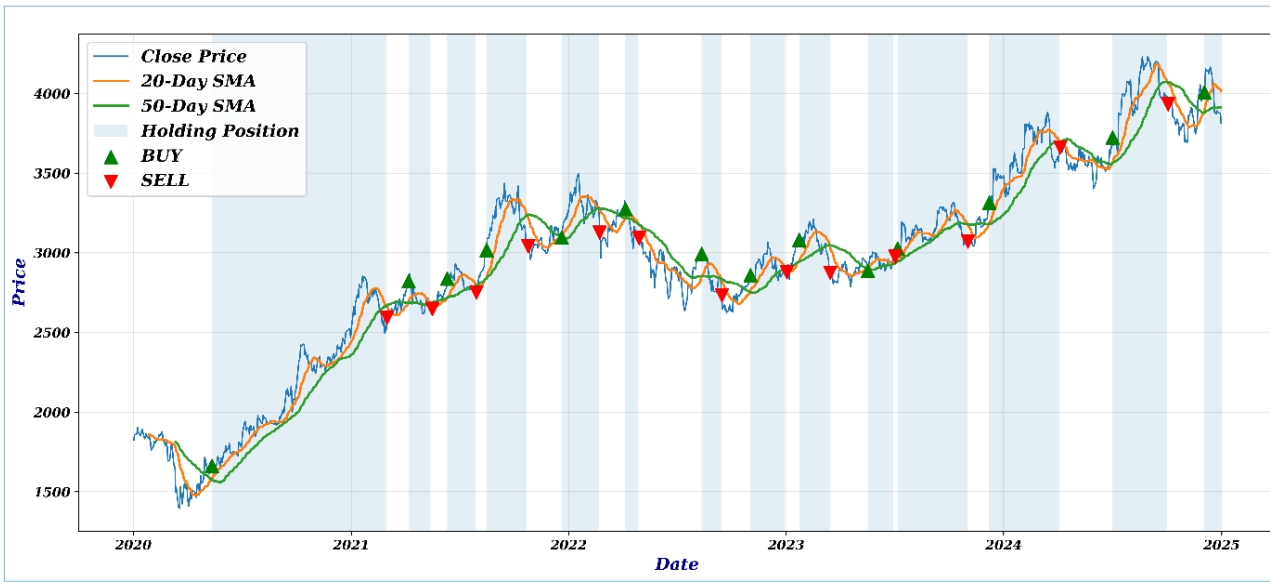


Figure 5: Buying, selling and holding stocks using the Moving Average Crossover Strategy

```
    df["SMA50"],
    label="50-Day SMA",
    linewidth=2
)
# Holding periods
ax.fill_between(
    df.index,
    0,
    1,
    where=holding,
    transform=ax.get_xaxis_transform(),
    alpha=0.12,
    label="Holding Position"
)
# BUY
ax.scatter(
    df.index[buy],
    df.loc[buy, "Close"],
    marker="^",
    s=120,
    label="BUY",
    zorder=5
)
# SELL
ax.scatter(
    df.index[sell],
    df.loc[sell, "Close"],
    marker="v",
    s=120,
    label="SELL",
    zorder=5
)
ax.set_title(
    f"{ticker} – Moving Average Crossover Strategy"
)
ax.set_xlabel("Date")
ax.set_ylabel("Price")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Buy zone: In Figure 5, when the short moving average crosses over the long moving average, the stock should be bought.

Hold zone: When the short moving average stays over the long moving average, the stock should be held.

Sell zone: When the short moving average crosses under the long moving average, the stock should be sold.

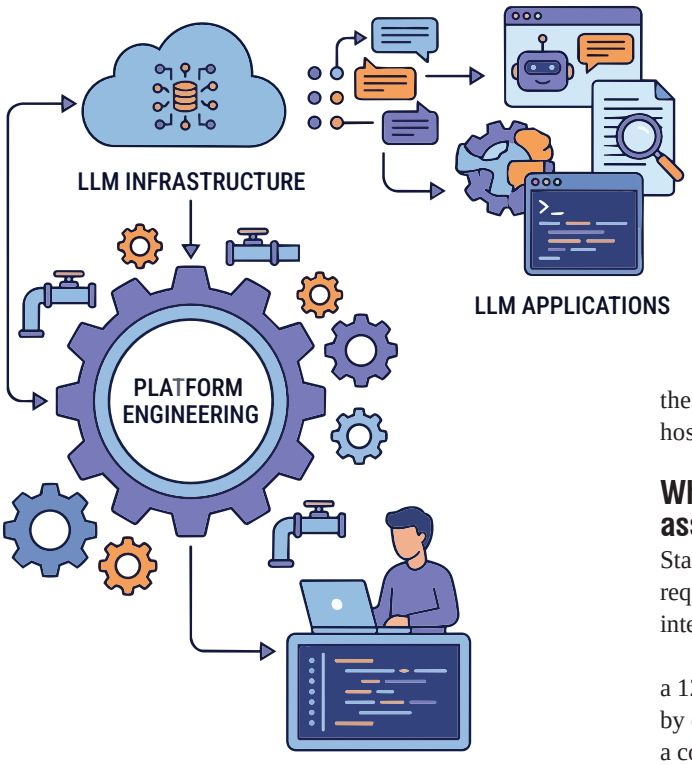
This provides traders with a direct visual relationship between the crossover and the position, and between the position and the holding period.

Traders can optimise the entire process of SMA crossover through backtesting using different time periods of moving averages -- for example, 5/10, 10/30, and 50/200. However, algorithmic trading is not just about producing Buy and Sell signals. It needs much more than that to build a robust strategy. END

By: Dr Dipankar Ray

The author is a technology enthusiast with interests in artificial intelligence, machine learning, natural language processing, and data-driven research. He has been involved in academic projects, technical writing, and training programmes that focus on the practical applications of AI.

A Peek at Platform Engineering for LLM Applications



Glitches in an LLM service do not necessarily show up because of a poorly engineered platform. Even platforms built well can falter because LLM workloads violate reliable assumptions.

A platform team that has spent five years perfecting Kubernetes golden paths, Terraform modules, and GitOps pipelines can still get blindsided by its first production LLM service. The pipelines work. The clusters are healthy. And the p95 latency chart looks fine while users complain, the bill triples after a one-line prompt edit, and a security review asks a question nobody can answer: what can this agent do if someone hides an instruction inside a support ticket?

None of that happens because the platform was built badly. It happens because LLM workloads violate assumptions that were so reliable they stopped being visible. This article works through three of them:

- Observability for a system where latency has two phases and correctness is a distribution.
- Security for a system where input, retrieved data, and model output are all untrusted.
- Cost for a system that bills per token and behaves like the cost of goods sold.

The tools named are open source or open standards, and the patterns hold whether you self-host models or call a hosted API.

Why do LLM workloads break your platform assumptions?

Start with the request. In a conventional service, one request costs roughly what the last one costs, replicas are interchangeable, and adding capacity is a scaling decision.

An LLM request breaks all three. A 200 token prompt and a 120,000 token prompt travel the same code path and differ by orders of magnitude in compute. A replica already holding a conversation's key value cache answers far faster than an identical replica that does not. And the instance you want during a spike may not exist, because accelerator capacity is contracted rather than elastic.

Classic assumption	What holds for LLM workloads	Platform consequence
Requests cost about the same	Compute varies 100x by prompt and output length	Quotas must count tokens, not requests
Replicas are interchangeable	Each replica holds its own KV cache state	Load balancing must be cache aware
Latency is one number	Time to first token and inter-token delay differ	Two SLOs, measured separately
Correctness is binary	Output quality is a distribution	Testing becomes statistical evaluation
Input is data	Input is also instruction	The model is a confused deputy by design

Read this table as a work plan. Every row implies a control the platform must provide, or each team will invent its own and none will agree.

What does an LLM platform actually look like?

Most LLM platforms converge on the same seven layers, whether the team planned it or arrived there by accretion. Naming them helps, because it tells you where a given control belongs.

The gateway earns its place first, because only four controls can sit together there:

- Identity and tenancy, so every request carries an owner.
- Token metering, so cost is attributable before the invoice arrives.
- Guardrails on input and output, applied once instead of per service.
- Routing and fallback, so you can switch providers without touching application code.

Teams that skip it end up with no cost attribution, no kill switch, and no failover.

What happens inside a single request?

The cost and latency levers all sit somewhere on the request path, so it is worth tracing once end to end.

Three facts from that path explain most of what you will later see on dashboards:

- Prefill processes the whole prompt at once, is compute bound, and sets time to first token. Long context and slow first tokens travel together.
- Decode generates one token at a time, is bound by memory bandwidth, and sets the inter-token delay users experience as typing speed.
- The key value cache holding attention state for every active request grows with context length and consumes GPU memory.

That third point is the one to keep. Concurrency on an inference server is limited by memory, not by CPU, so when the cache fills, tail latency degrades sharply rather than gradually.

What has to be versioned and deployed?

A conventional service has code and config. An LLM service has at least eight things that change behaviour in production, and a change to any one of them is a deployment:

- Model weights and version, plus the quantization and engine config used to serve them.

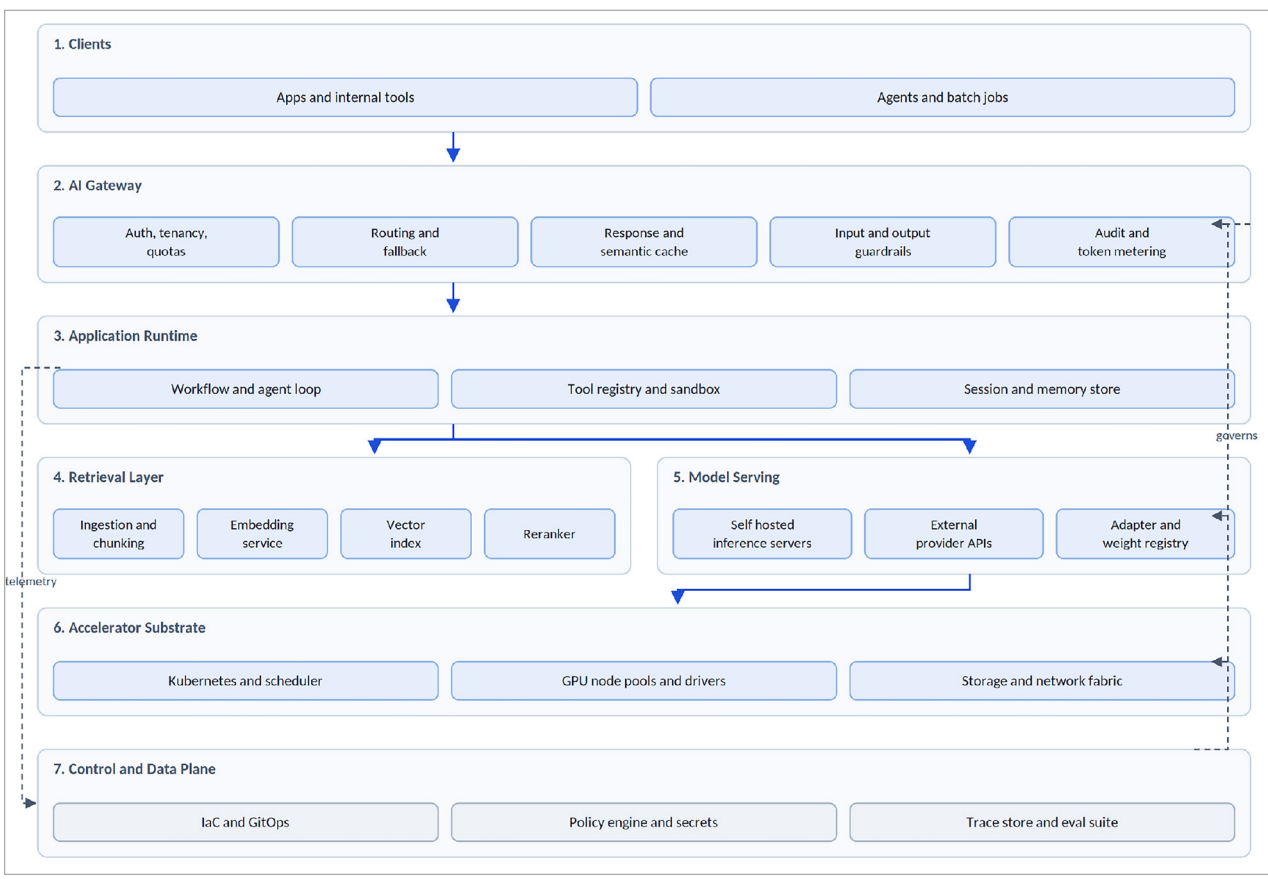


Figure 1: Reference architecture. The gateway and the control plane are the two layers platform teams most often skip, and the two they most often have to retrofit under pressure

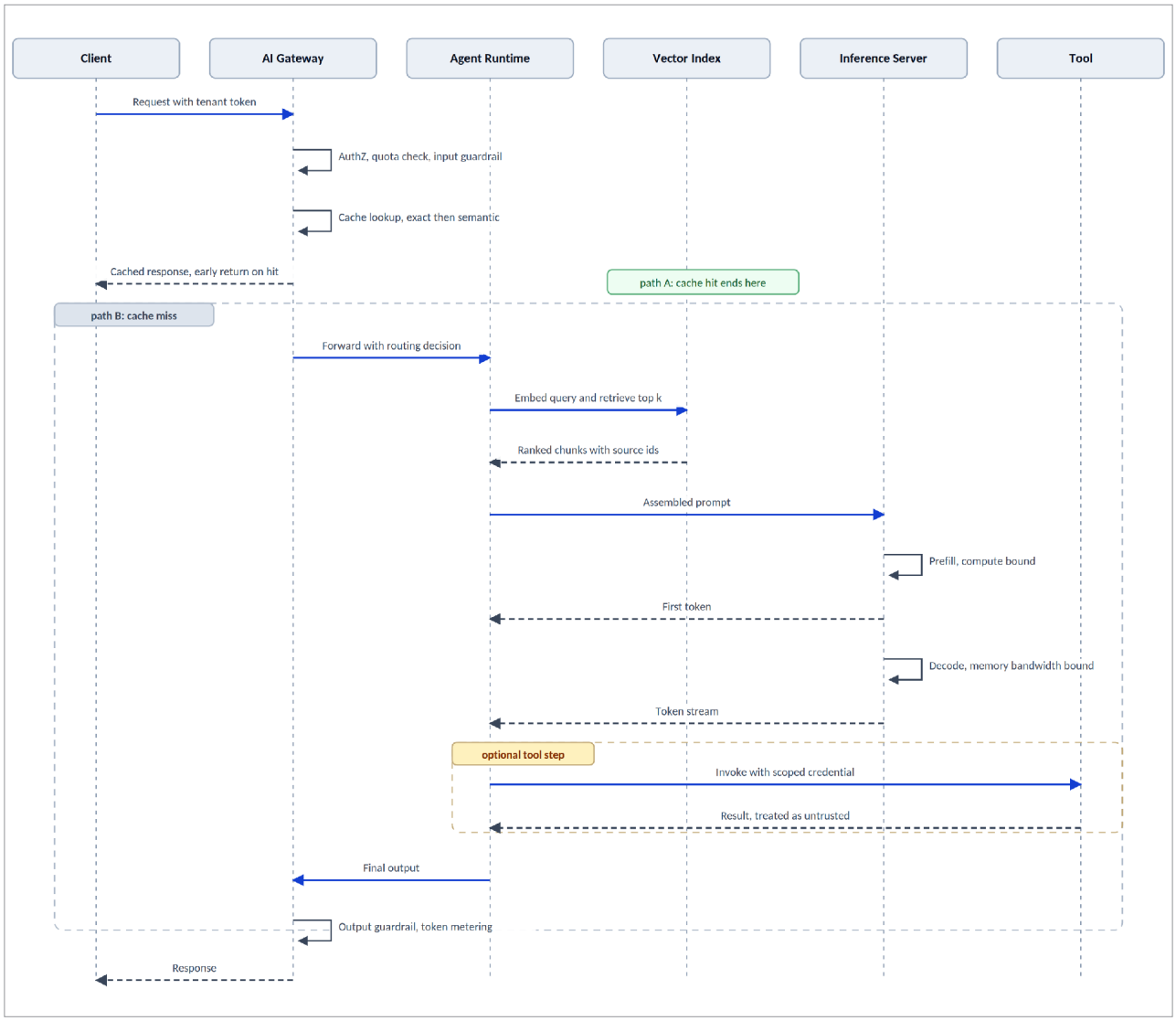


Figure 2: Request lifecycle. Note that the gateway touches the request three times, and that tool output re-enters the model as new untrusted input

- Prompt templates and the tool schemas the model is allowed to call.
- The embedding model, the chunking rules, and the index built from both.
- Guardrail policies and the evaluation datasets that define what passing means.

The embedding model bites hardest. Vectors from one model are not comparable to vectors from another, so swapping it invalidates the entire index. If rebuilding ten million chunks takes six hours, that belongs in a runbook long before someone proposes the upgrade in sprint planning.

Common advice says to treat prompts as code. That is half right. A prompt without an evaluation set is untested code, and untested code in a non-deterministic system is closer to a configuration guess than a change you can defend in a postmortem.

Hardware scheduling has caught up. Dynamic Resource Allocation reached general availability in Kubernetes 1.34 (released August 2025) and is on by default from 1.35, letting workloads declare the device attributes they need instead of an opaque GPU count. One thing to resist: self-service on accelerator quota.

How do you observe a probabilistic system?

Three audiences need three views, and a single dashboard serving all of them will serve none.

Plane	Representative signals	Question it answers
Accelerator	Memory used, bandwidth utilisation, tokens per second per GPU	Is the hardware earning its cost?
Serving	TTFT, inter-token latency, queue wait, KV cache utilisation, prefix cache hit rate	Is the model server keeping up?
Application	Tokens by tenant and model, cost per request, tool failures, guardrail blocks, eval scores	Are users getting good answers at a sane price?

Here are the signals that predict user pain:

- GPU utilisation percentage is the most misleading number in the stack, because it reports whether any kernel ran rather than whether the device did useful work.
- Queue wait time is the leading indicator, since time to first token rises non-linearly once requests wait for cache capacity.
- End-to-end averages hide streaming stalls, because a response that pauses halfway does complete eventually.
- Goodput, the rate of requests served within SLO, is the figure that tells you when to add capacity.

A server can sit at 95 percent utilisation while producing a fraction of its possible tokens per second, so you must track tokens per second per accelerator alongside it.

How do you trace a request end to end?

One user question can fan out into a retrieval call, three model calls, and two tool invocations. The interesting question is always which of those consumed the time and the tokens.

```
trace 4f2a... chat.completion    total 8.4s     4,180 tokens $0.021
├─ gateway.guardrail.input          22ms     blocked=false
├─ cache.semantic.lookup            41ms     hit=false similarity=0.71
├─ retrieval.embed                  88ms     model=embed-v3
├─ retrieval.search                134ms     k=8 index=support-kb-v11
├─ gen_ai.chat attempt 1         3,900ms     in=3,102 out=210 ttft=740ms
│  └─ tool.ticket_lookup           610ms     status=ok scope=read:tickets
├─ gen_ai.chat attempt 2         3,510ms     in=3,590 out=278 ttft=690ms
└─ gateway.guardrail.output         55ms     redactions=1
```

Figure 3: A span tree for one request. Token counts and cost belong on spans, not only on dashboards, because attribution is what makes a spike diagnosable

The vendor-neutral way to produce that structure is the OpenTelemetry genAI semantic conventions, which standardise span names, token attributes, and metrics so an LLM call looks the same regardless of which library emitted it. OpenTelemetry graduated from the CNCF in May 2026. Adopt the conventions with your eyes open:

- They remain in development status as of mid-2026 and moved to a dedicated repository in the v1.42.0 release of June 2026, which changed release cadence rather than granting stability.
- Pin the convention version and the instrumentation packages, then inspect a stored trace rather than trusting a documentation example.
- Leave prompt and completion content uncaptured by default, store it separately with its own retention and access controls, and sample rather than capturing everything.

Quality needs its own signals. Automated evaluators score a sample of live traffic, and implicit signals, such as retries, edits, and escalations to a human, cost nothing to collect. Using a model as judge means paying for a second inference and inheriting its variance, so sample rather than judging everything.

The most useful first step to secure an LLM application is drawing the trust boundaries, because the intuitive drawing is wrong (Figure 4).

Prompt injection has no single fix

Prompt injection has held the top position in the OWASP Top 10 for LLM Applications since the list began, and the 2025 edition keeps it at LLM01. The reason is structural: models receive instructions and data through one channel with no reliable separation.

Direct injection is the familiar version. Indirect injection is the dangerous one, where an attacker plants instructions inside a document or a support ticket, and they execute when retrieval pulls that content into context. Since no filter catches everything, design so that a successful injection cannot accomplish much:

- Scope every tool credential to least privilege and authorise downstream calls as the end user rather than as a shared agent identity.
- Validate output against a schema before anything acts on it, and never pass model output into an interpreter, a shell, or a query untreated.
- Require human approval for actions that move money, send external communication, or delete data.
- Restrict outbound network access from tool sandboxes to an allowlist, which turns many exfiltration attempts into blocked connections.
- Cap loop depth, tool calls, and output tokens per session, because unbounded consumption is both denial of service and denial of wallet.

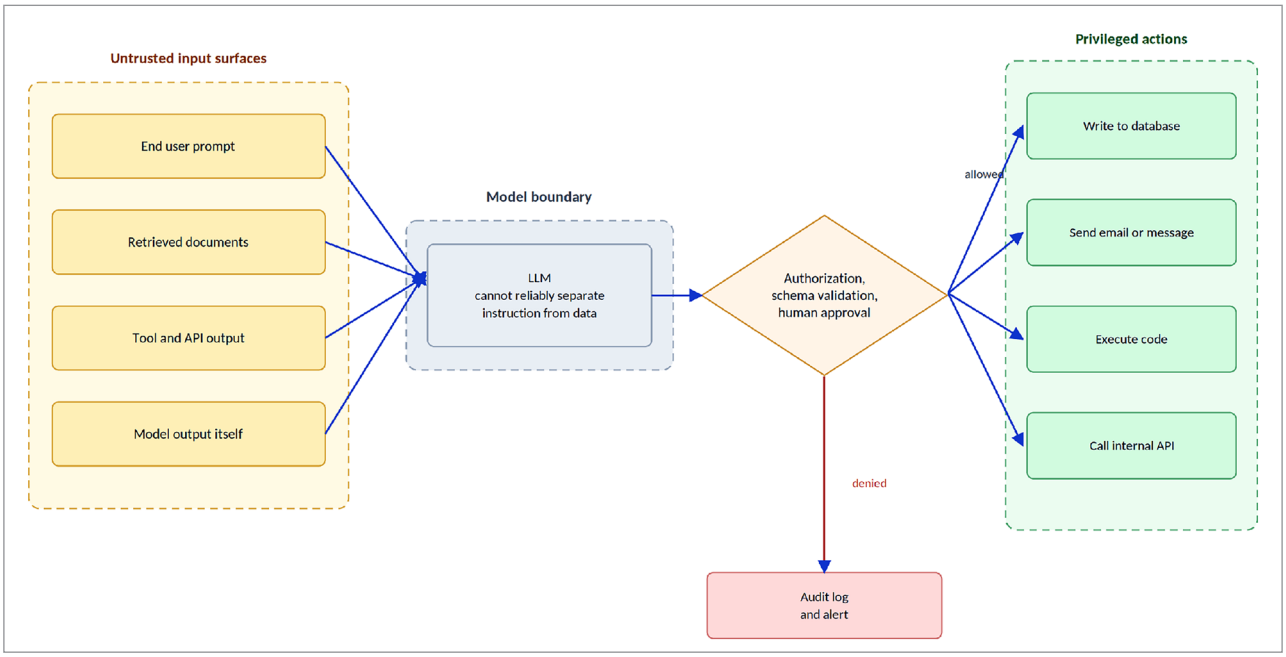


Figure 4: Trust boundaries. Retrieved documents and tool results are attacker-influenced input, and model output is untrusted until validated

The model supply chain is a binary supply chain

Model files are executable artifacts. Serialisation formats based on Python pickle can run arbitrary code at load time. This is not theoretical, and documented CVEs have covered remote code execution when loading untrusted models. Scanners have their own weaknesses, with bypass vulnerabilities in a widely used pickle scanner disclosed in late 2025.

Build an ingestion gate and make it the only path into production:

- Prefer formats that cannot execute code during deserialisation, such as safetensors, and refuse pickle-based artifacts unless there is a documented exception.
- Verify provenance cryptographically. Sigstore-backed model signing gives you a tamper-evident record of who produced the artifact.
- Scan artifacts and their dependency graph, then record the result as an attestation rather than a one-time check.
- Publish an AI bill of materials capturing model lineage, training data description, and evaluation results, using the ML-BOM extensions now present in the major SBOM formats.
- Mirror approved models into an internal registry and block direct pulls from public hubs in production networks.

The dependency side matters as much as the weights. Package ecosystems around ML tooling have been targeted through dependency confusion and through package names hallucinated by coding assistants, which attackers then register.

Policy as Code at the admission boundary

Written standards do not stop a pod from starting. Admission policy does.

Two open source engines dominate. Both are worth understanding because they fit different teams.

Dimension	OPA with Gatekeeper	Kyverno
Policy language	Rego, general purpose	YAML, Kubernetes native
Learning curve	Steeper	Lower for Kubernetes teams
Scope	Kubernetes and beyond	Kubernetes focused
Mutation support	Available	First class
Best fit	Policy shared across systems	Cluster guardrails and defaults

A representative rule for an inference workload looks like this.

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: restrict-model-sources
spec:
  validationFailureAction: Enforce
  rules:
```

```
    - name: models-from-internal-registry-only
      match:
        any:
          - resources:
              kinds: ["Pod"]
              namespaces: ["inference-*"]
      validate:
        message: "Model artifacts must come from the internal
registry."
        pattern:
          spec:
            containers:
              - image: "registry.internal.example.com/*"
                env:
                  - name: MODEL_URI
                    value: "s3://models-approved/*"
```

Useful companion policies are: deny pods in inference namespaces without a cost-attribution label, require a resource claim rather than a bare GPU count, block egress-enabled tool sandboxes outside an allowlisted namespace, and require a signature attestation on any image that loads model weights.

How do you control the cost of inference?

Inference cost scales with usage and lands in gross margin. Most teams reach for model selection first because it feels like the engineering lever, then find that caching and prompt structure return more, sooner, with less risk.

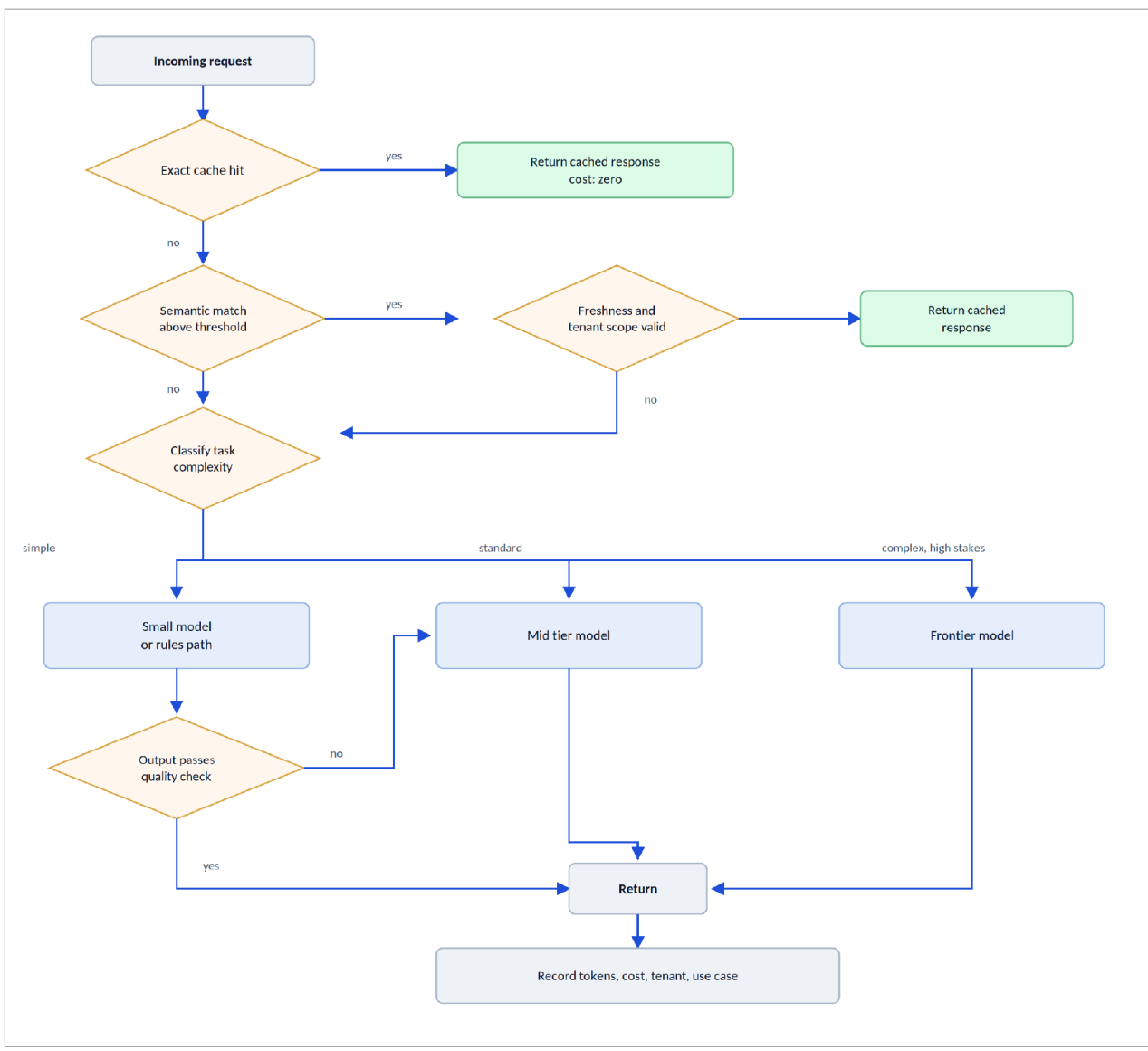


Figure 5: A routing cascade. Escalation on failure keeps quality intact while making the expensive model the exception rather than the default

So where does the money actually go?

- Continuous batching is the highest-return change on a self-hosted stack, commonly delivering two to three times the throughput of static batching.
- Quantization increases the batch size that fits in a given GPU, and the only trustworthy way to judge the quality cost is your own evaluation set.
- Spot capacity suits evaluation runs and batch scoring, and suits interactive serving poorly, because reloading tens of gigabytes of weights after a preemption is not recoverable within a user's patience.
- Scale to zero fails for the same reason, so keeping one replica warm per model with daytime traffic usually beats saving a few dollars an hour.

Two newer patterns are worth tracking. Disaggregated serving splits prefill and decode into independently scaled pools, and prefix-aware routing sends each request to the replica already holding a matching cached prefix. The Kubernetes Gateway API Inference Extension does exactly that, scoring endpoints on queue depth and cache locality instead of round robin.

Why should you route before you generate?

The cheapest inference is the one you do not run.

Routing needs three things: a classifier or rule set, an evaluation set proving cheaper paths hold quality, and observability to catch drift. Without the evaluation set, routing is a quality regression you have not measured yet.

Cache at several levels

Caching is not one technique. It is four, with different hit profiles and different failure modes.

Cache layer	What is stored	Where it helps	Main risk
Exact response	Output keyed on prompt hash	High-repetition workloads	Stale answers when data changes
Semantic	Output keyed on embedding similarity	Paraphrased questions	False hits when threshold is loose
Provider prefix cache	Repeated static prompt prefix	Long system prompts, few-shot examples	Needs stable prompt ordering
KV prefix reuse	Attention state on the replica	Multi-turn conversations	Needs cache-aware routing

Semantic caching fails in the most interesting ways, because similarity is not equivalence and small wording changes carry large intent changes. Set the threshold conservatively, scope keys by tenant, and log near misses so you can tune with evidence.

What should FinOps actually measure?

Cost per token is an accounting metric. It falls when you switch to a cheaper model even if that model needs three attempts to get the answer right. Cost per successful task captures retries, escalations, and quality failures in one number.

- Tag every request at the gateway with team, use case, model, and environment.
- Design those tags before the first production call, since they cannot be added retroactively without breaking continuity.
- Alert on rate of spend rather than monthly totals, because a runaway agent loop can burn a month's budget long before a cumulative threshold fires.

How do you ship model and prompt changes safely?

LLM platforms need three delivery tracks running at different cadences against one declared state.

What changes is the promotion gate:

- Alongside tests and security scanning, the pipeline blocks on evaluation thresholds, projected cost per task, and canary quality signals.
- Shadow traffic compares two model versions on identical production inputs without exposing users to either result.
- Canary by request class beats canary by percentage, because routing all queries of one low-risk type reveals more than five percent of everything.
- Rollback needs the previous model pinned and warm, with the prompt versioned alongside it, since the pair was evaluated together.

What changes when you run it in production?

Conventional high availability assumes you can add a replica. With accelerators, redundancy means paying for an idle GPU, so multi-provider fallback at the gateway is usually cheaper insurance than duplicate capacity, provided the path is exercised. Untested failover is a hypothesis, not a control.

A degradation ladder, agreed in advance and implemented at the gateway, converts a hard outage into a slower service.

Level	Trigger	Behaviour
1	Queue wait above target	Reduce max output tokens, trim retrieval k
2	Primary model degraded	Route to secondary provider or smaller model
3	Sustained capacity shortfall	Serve cached answers for common intents
4	Inference unavailable	Disable AI paths, expose underlying tools directly

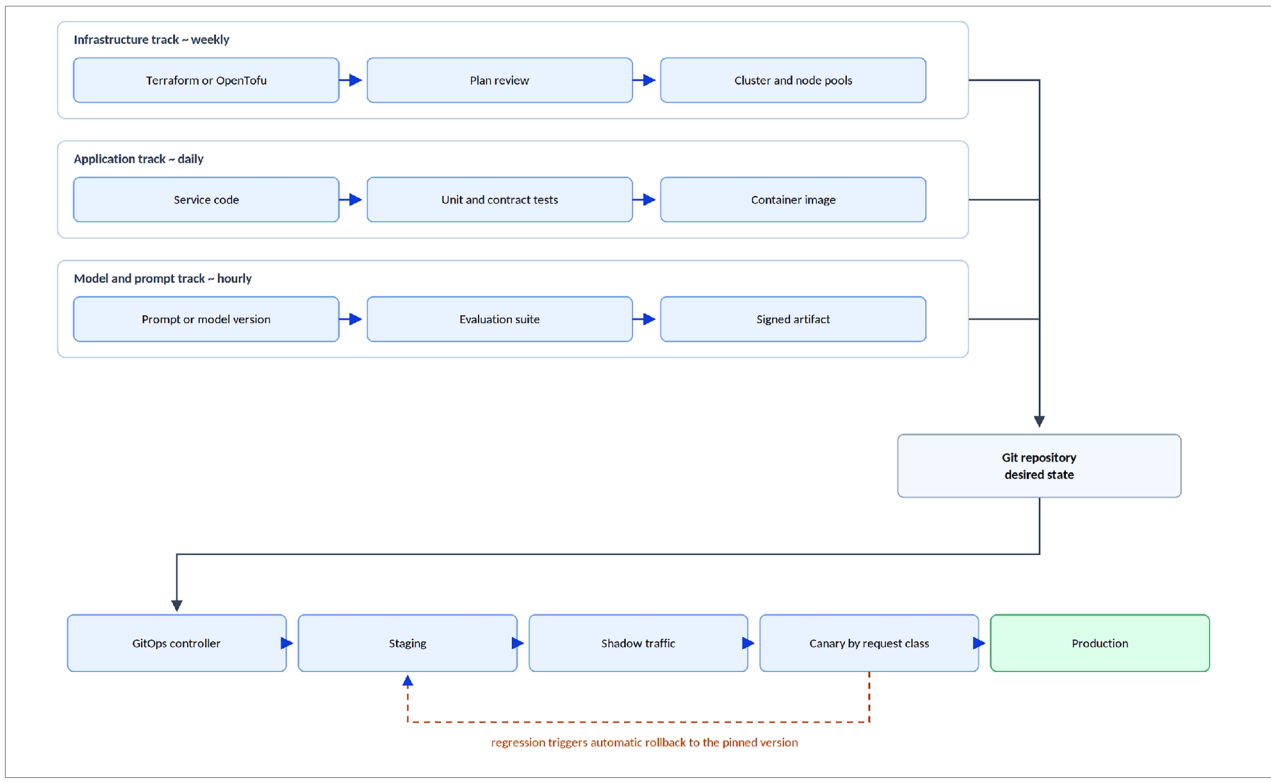


Figure 6: Three tracks, one desired state. The model and prompt track moves fastest and therefore needs the strictest automated gates.

Level 4 matters more than it seems to. If the product has no non-AI path, an inference outage is a full product outage, which is a decision worth making deliberately.

Regulated deployments need one more thing: an immutable record linking the request, prompt and model version, retrieved document IDs, tool calls, output, and any approver.

Which failure patterns show up most often?

Most of these are observability failures first. Each was detectable earlier with a signal the platform was not collecting.

Pattern	Underlying cause	Fix
Green dashboards, unhappy users	Streaming stalls hidden by end-to-end averages	Measure TTFT and inter-token latency separately
High utilisation, low output	Small batches, memory bandwidth bound	Continuous batching, tune KV cache size
Retrieval quality collapse	Embedding model changed without reindex	Version the embedding model with the index
Provider outage takes everything down	Fallback configured but never tested	Scheduled failover exercises

Where should you start?

Sequence matters more than completeness:

- Put the gateway in first, because it gives you identity, metering, routing, and a kill switch in one component, and every later control has somewhere to live.
- Instrument to the genAI conventions next, with content capture off.
- Build the evaluation suite third, since it is the prerequisite for routing, caching, and any model change you want to make safely.
- Optimise cost last, because caching and routing without evaluations is optimisation without a safety net.

The honest caveat is that this order is slower than shipping against a provider API and dealing with the platform later. Plenty of products started that way. The difference shows up at the third or fourth AI service, when nobody can say which team is spending what, or what an agent could reach if asked nicely enough.

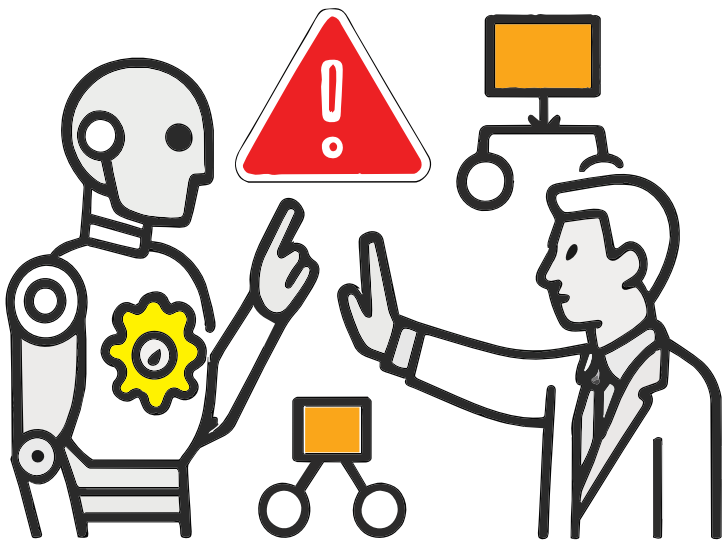
The discipline that made microservices manageable still applies. It just has to account for workloads that cost per token, hold state in GPU memory, treat their input as instruction, and answer differently to the same question twice. END

By: Raj Patel

The author is a SaaS enthusiast and technical content creator.

Stopping Dangerous AI Agent Actions Before Execution

A Linux-first control pattern can stop risky agent actions before they reach the shell or filesystem. You can enforce a policy before AI agent tool execution using open source and Linux security controls.



Imagine a coding agent working inside a Linux project. It can inspect files, run tests, edit source code, install packages, and call network tools. Most of those actions are ordinary. The dangerous moment is the one where a bad instruction, compromised dependency, or poisoned context turns an ordinary task into a command that reads a secret, deletes the wrong directory, or sends data somewhere it should not go.

If the security control only notices the problem after the command has run, the log may be useful for an investigation, but it is too late to prevent the action. A safer design puts a policy decision directly in front of the tool call.

The dangerous gap is before execution

AI agents do not need root access to cause meaningful damage. A developer account can usually modify a repository, read configuration files, run a package manager, invoke Git, and reach the network. An agent that inherits those permissions inherits the same blast radius. The model may be the component deciding what to do, but the shell, filesystem, package manager, browser, and API clients are the components that actually do it.

That distinction matters. Monitoring model output is helpful, but a model can describe one action and a tool adapter can execute another representation. Post-execution telemetry is also helpful, but it records the event after the side effect. The reliable control point is the boundary where a proposed action has been converted into a concrete tool call but has not yet executed.

Put policy at the action boundary

At that boundary, normalise the proposed action into a small, deterministic record. For a shell tool, that can include the executable, arguments, working directory, environment references, and whether the command can reach the network. For a filesystem tool, capture the requested path and operation. For Git, capture whether the action is a read, commit, push, force-push, or history rewrite.

A policy check can then return one of three simple decisions: allow, deny, or require review. The important part is that the decision happens before the tool receives permission to execute.

```
action = {
  "tool": "shell",
  "command": "git push --force origin
main",
  "cwd": "/srv/project"
}
decision = policy.evaluate(action)
if decision == "deny": stop()
if decision == "review": request_
approval()
if decision == "allow": execute()
```

This pattern keeps the security decision outside the model. The agent can propose an action, but it cannot grant itself permission. That separation is the same reason Linux administrators do not treat an application process as the authority for its own access controls.

Use Linux controls as backstops

Pre-execution agent policy should

complement Linux security rather than replace it. File permissions, containers, namespaces, seccomp, AppArmor, and SELinux can reduce what a process can do even if the agent-level control misses something. The two layers solve different problems.

Linux controls are strong at enforcing operating system boundaries. Agent-level policy can reason about higher-level intent before those boundaries are reached. It can distinguish a normal 'git status' from a force-push, a read from a write, or a package inspection from an installation that may run lifecycle scripts. A useful deployment has both: semantic checks close to the tool call and operating system limits underneath.

Treat tools as capabilities

A practical policy starts by treating every tool as a capability rather than giving an agent a single all-or-nothing trust level. Most agents need far fewer permissions than the human account that launches them.

For example, a coding agent may be allowed to read the project tree, write only inside the current repository, run a known test command, and inspect Git state. It may require review before installing a package, pushing to a remote, or making a network request. It should normally be denied access to SSH keys, cloud credentials, browser profiles, password stores, and unrelated home-directory files unless the task specifically requires them.

This capability model also makes policies easier to audit. Instead of asking whether an agent is 'trusted', you can answer concrete questions: which directories may it modify, which commands may it launch, which domains may it contact, and which operations require a human decision?

Keep policy deterministic

Security policy should be predictable. Avoid making the final allow-or-deny decision with another language model. A model can help classify context or explain why a rule fired, but the enforcement decision should come from deterministic rules whenever possible.

Teams can implement those rules directly in application code, in a small configuration format, or with an open policy engine such as Open Policy Agent. What matters is that the same proposed action produces the same decision under the same policy. That makes failures reproducible and lets administrators review changes in version control.

Escalate ambiguous actions

Blocking every mutation makes an agent safe but not very useful. Allowing every mutation creates the opposite problem. The review state gives teams a middle path for actions that are legitimate but unusually risky.

Good review candidates include a force-push, a recursive delete outside the workspace, a command that reads from a credential directory, a package installation with lifecycle scripts, or a network request that carries environment-derived secrets. The goal is not to ask a human about every command. It is to reserve review for the small set of actions where the cost of a mistake is high, or the policy lacks enough context to decide safely.

Test policy like application code

A policy that is not tested will drift. Treat security rules as code and build a small regression suite around the actions your agents actually perform. Each test should describe a proposed action and the decision you expect.

Useful tests include allowing a read inside the repository, denying reads from *~/.ssh*, allowing the project test runner, reviewing a new outbound network destination, and denying a destructive command outside the workspace. Add a regression test whenever a real incident or near miss exposes an unexpected path. This turns operational lessons into durable controls rather than one-time warnings.

Where HOL Guard fits

HOL Guard on open source implementation of this pre-execution pattern for AI agents and coding-tool workflows. It intercepts supported actions before execution and applies policy so a risky operation can be allowed, denied, or stopped for review. The broader pattern is not specific to one product. Teams can build the same architecture with their own wrappers, agent hooks, policy middleware, and Linux controls.

The important design choice is where enforcement lives. If the check sits after the command, it is monitoring. If the model can bypass the check, it is advisory. If every side-effecting tool call must cross a policy boundary before execution, it becomes an actual control.

Build the smallest useful boundary

Start with the actions that have the largest blast radius: shell execution, filesystem writes, credential access, package installation, Git pushes, and outbound network calls. Normalise those actions, apply deterministic policy, and make review a first-class result rather than an exceptional path.

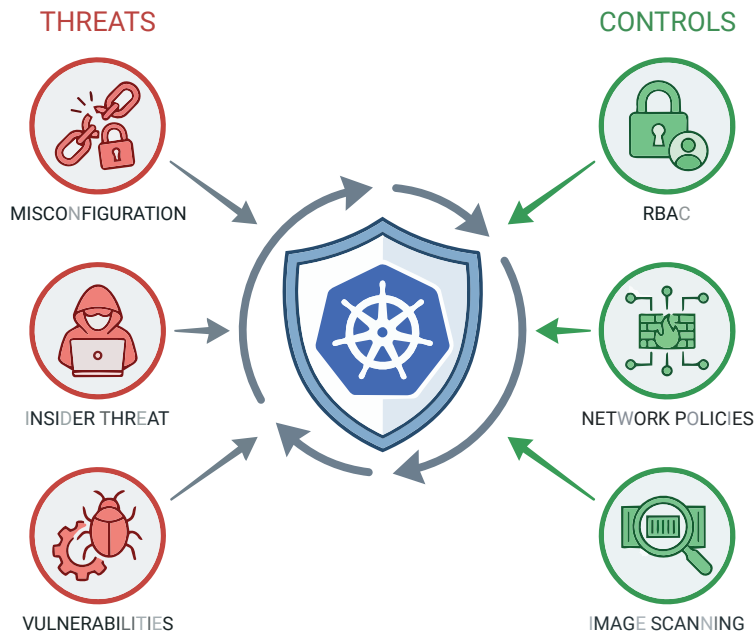
AI agents will keep getting more capable. The useful security question is not whether a model can be perfectly trusted. It is whether a proposed action can be checked before the system commits to it. On Linux, that boundary can be small, testable, open source, and layered with controls administrators already understand. END

By: Michael Kantor

The author is the president of Hashgraph Online. He works on open standards and open source security tooling for AI agents.

Kubernetes Security: Threats and Controls

Kubernetes has become the *de facto* standard for orchestrating containerised workloads at scale, but its distributed architecture introduces a broad attack surface spanning the control plane, worker nodes, network, and the software supply chain that builds container images. Thankfully, there are a number of controls that can help to secure systems.



Kubernetes (K8s) automates the deployment, scaling, and management of containerised applications. Its power comes from a highly programmable API server that mediates every action inside the cluster — from creating a pod to modifying network rules. This same programmability, however, means that a single misconfiguration (an overly permissive role, an open network path, an unscanned image) can expose the entire cluster to compromise. Industry data cited in recent reports shows that most organisations now run Kubernetes in hybrid-cloud environments and are increasing their reliance on it as a primary application platform, which raises the stakes for getting security right from the outset.

Kubernetes architecture overview

A Kubernetes cluster is divided into a control plane, which makes global decisions about the cluster (scheduling, detecting/responding to events), and worker nodes, which run the actual application workloads inside pods (Figure 1). The control plane's kube-apiserver is the single-entry point for all requests and is therefore the most security-critical component: every authentication, authorisation Role-based access control (RBAC), and admission-control check happens here before a request is persisted to etcd, the cluster's key-value store of records.

Why Kubernetes security matters

The risks described in this article are not theoretical. In 2018, security researchers at RedLock discovered that Tesla's Kubernetes Dashboard had been left exposed to the public internet with no authentication at all (Figure 2). Attackers found the console, used it to obtain AWS credentials embedded in a pod's environment, and deployed cryptocurrency-mining containers across Tesla's cloud infrastructure — deliberately throttling CPU usage to stay under the radar of monitoring systems. The incident illustrates, in one real example, how a single missing control (authentication on a management interface) cascades into credential exposure, unauthorised workload deployment, and sustained resource theft. We will revisit this and two further real-world cases in detail later.

Core controls of Kubernetes security

Role-based access control (RBAC): RBAC governs who (a user, group, or ServiceAccount) can perform which verbs (get, list, create, delete, etc) on which resources (pods, secrets, deployments) in the cluster. A Role or ClusterRole defines a set of permissions;

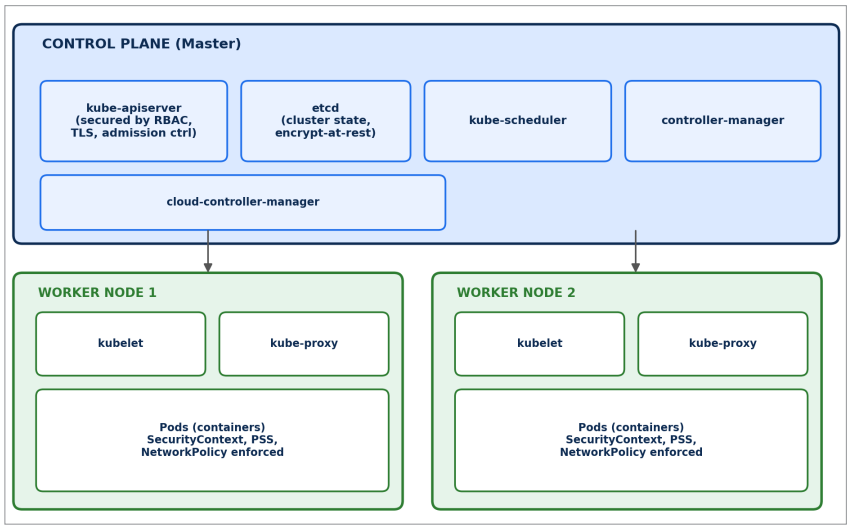


Figure 1: Kubernetes cluster architecture – control plane and worker nodes (Source: *https://kubernetes.io/docs/concepts/overview/components/*)

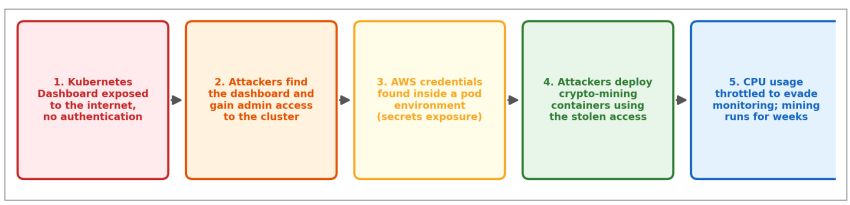


Figure 2: Public incident reporting by RedLock and Wired on the exposure of Tesla's Kubernetes Dashboard to the public internet (Source: *https://www.wired.com/story/cryptojacking-tesla-amazon-cloud/*)

a RoleBinding or ClusterRoleBinding attaches that set of permissions to a subject. The Principle of Least Privilege (PoLP) is the guiding rule: subjects — including CI/CD service accounts — should be granted only the permissions strictly required for their function, since granting cluster-admin broadly makes lateral movement trivial for an attacker who compromises any single credential.

Table 1 summarises the core RBAC objects and how they form an access-control decision.

The five objects in this table always work together as a chain, never in isolation. Consider a CI/CD pipeline that needs to deploy to the 'staging' namespace. The correct pattern is to create a dedicated ServiceAccount for the pipeline, define a narrowly scoped Role that permits only 'create' and 'update' on Deployments and Services within 'staging', and connect the two with a RoleBinding — nothing more. A ClusterRole and ClusterRoleBinding would only be justified if the same pipeline genuinely needed to act across every namespace (for example, a cluster-wide monitoring agent). A well-documented real-world failure of this pattern is the legacy Helm 2 'Tiller' component: its simplest install path granted the Tiller ServiceAccount a full cluster-admin ClusterRoleBinding and exposed an unauthenticated in-cluster endpoint, meaning any pod that could reach Tiller could deploy arbitrary resources and inherit cluster-admin rights — exactly the over-broad-binding failure mode Table 1's least-privilege pattern is designed to prevent.

Network policies: By default, all pods in a Kubernetes cluster can communicate with all other pods — a flat network with no isolation. NetworkPolicy resources restrict this by defining allowed ingress/egress traffic based on pod and namespace labels. Current security checklists recommend starting with a default deny-all policy per namespace and then explicitly allowing only the traffic paths an application needs, combining this with RBAC and secrets management to build a defence-in-depth architecture.

Pod and container hardening: Pod Security Standards (PSS) define three levels of restriction that can be

Table 1: Core Kubernetes RBAC objects *(Source: https://kubernetes.io/docs/reference/access-control/rbac/)*

Object	Scope	Purpose	Example
Role	Namespace	Defines allowed verbs on resources within one namespace	Allow 'get,list' on pods in 'dev'
ClusterRole	Cluster-wide	Defines allowed verbs on resources across all namespaces, or on cluster-scoped resources	Allow 'list' on nodes
RoleBinding	Namespace	Grants a Role (or ClusterRole) to a subject within one namespace	Bind 'dev-viewer' Role to user 'alice'
ClusterRoleBinding	Cluster-wide	Grants a ClusterRole to a subject across the whole cluster	Bind 'cluster-admin' to a break-glass account
ServiceAccount	Namespace	Identity used by pods/CI pipelines to call the API server	CI pipeline deploying to 'staging'

enforced per namespace: Privileged (unrestricted), Baseline (blocks known privilege escalations), and Restricted (enforces current pod-hardening best practice). Recommended container-level controls include running as a non-root user, dropping all Linux capabilities and adding back only what is required, using minimal base images, and scanning images for vulnerabilities in the CI/CD pipeline before they ever reach the cluster.

PSS levels are applied per namespace via labels (e.g., pod-security.kubernetes.io/enforce: restricted), which means a cluster can safely run mixed workloads — a legacy system namespace at Baseline while every new application namespace is locked to Restricted. The practical difference between levels is best seen through container-escape vulnerabilities such as CVE-2019-5736 in runc, which allowed a malicious or compromised container to overwrite the host's runc binary and escape to the underlying node. A Restricted-level pod (non-root, all capabilities dropped, no privilege escalation) closes off most of the preconditions that class of exploit depends on, while a Privileged-level pod offers no resistance at all — which is why production namespaces are increasingly defaulted to Restricted rather than Baseline.

Secrets management, admission control, and monitoring: Secrets encrypt Kubernetes Secrets at rest in etcd, and prefer an external secrets manager (e.g., HashiCorp Vault, AWS Secrets Manager) over storing credentials directly in manifests.

Admission controllers intercept API requests after authentication/authorisation to enforce policy before an object is persisted; policy engines such as OPA Gatekeeper and Kyverno are commonly used to block non-compliant workloads.

Audit logging and monitoring continuously monitor API server audit logs, RBAC changes, and NetworkPolicy changes; eBPF-based tools (e.g., Cilium Hubble, Tetragon) provide runtime visibility into traffic that static policy review alone would miss.

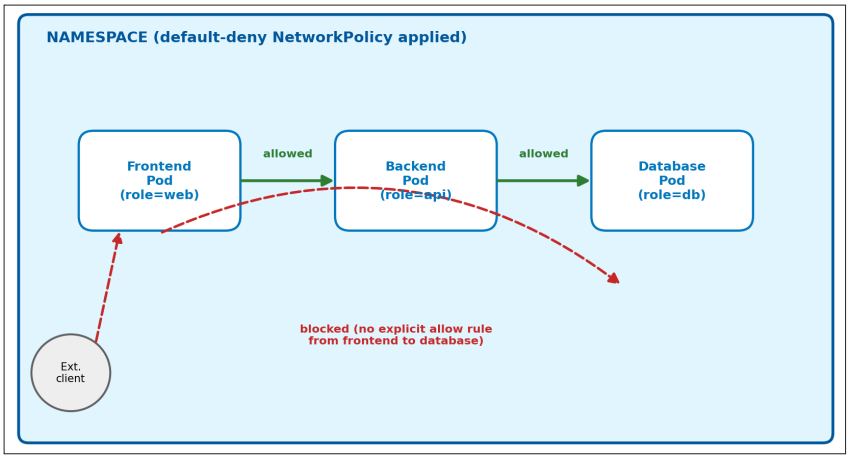


Figure 3: Network policy enforcing micro-segmentation between tiers (Source: *https://www.networkershome.com/fundamentals/cloud-security/kubernetes-security-rbac-policies-admission/*)

Software supply chain security: Because Kubernetes workloads are built from container images assembled through CI/CD pipelines, cluster security is inseparable from software supply chain security. A best practice is to scan and sign every image before deployment (e.g., with Trivy for vulnerability scanning and Cosign for cryptographic signing), block images with unpatched high/critical CVEs from being admitted to the cluster and verify image provenance at admission time so that only artifacts built by a trusted pipeline can run. This closes the gap between "the code is secure" and "the running container is secure."

Real-world scenarios

The following three publicly documented incidents each map to one of the control areas discussed above, showing what happens in practice when a single layer of the (Cloud, Cluster, Container, Code) 4C's-style defence is missing.

- ***Tesla — Exposed dashboard and credential theft (2018)***

As introduced in Figure 2, security firm RedLock discovered that Tesla's Kubernetes Dashboard was reachable from the public internet with no authentication configured. Attackers used the console to reach a pod's environment variables, which contained live AWS credentials, including access to an Amazon S3 bucket holding telemetry data. Rather than exfiltrate that data, the attackers used the stolen access to deploy cryptocurrency-mining containers, tuning CPU usage and routing traffic through Cloudflare to avoid detection. Tesla fixed the issue within hours of RedLock's disclosure. This was a failure of authentication and secrets hygiene — the exact gap that RBAC and external secrets management are designed to close.

- ***Azurescape — Cross-tenant container escape on Azure Container Instances (2021)***

Palo Alto Networks' Unit 42 team found that Microsoft's Azure Container Instances service was running containers on an outdated, vulnerable version of the runc container runtime. By exploiting known runc container-escape flaws, researchers broke out of their own container, reached the underlying multi-tenant Kubernetes node, obtained a privileged Kubernetes service-account token, and used it to take over the cluster's API server — gaining the theoretical ability to access other customers' containers on the same shared infrastructure. Microsoft

Table 2: Kubernetes Pod Security Standards (PSS) levels (Source: *https://kubernetes.io/docs/concepts/security/pod-security-standards/*)

Level	Description	Typical controls enforced
Privileged	Unrestricted policy, provides the widest possible level of permissions	None — intended only for system/infra workloads
Baseline	Minimally restrictive; prevents known privilege escalations	Blocks host namespaces, privileged containers, most host mounts
Restricted	Heavily restricted; follows current pod-hardening best practice	Requires runAsNonRoot, drops all capabilities, blocks privilege escalation

patched the issue and notified affected customers to rotate credentials; no in-the-wild exploitation was reported. This demonstrates why container and node hardening (Table 2) and keeping the underlying container runtime patched are just as critical in managed and multi-tenant Kubernetes offerings as in self-managed clusters.

- ***Dero cryptojacking campaign — Anonymous API access (2024)***

More recently, an automated campaign distributing the Dero cryptominer was observed scanning the internet for Kubernetes API servers that still permitted anonymous, unauthenticated access. Where such clusters were found, the attackers used that open access to pull and deploy malicious container images directly from public registries, turning the victim's own compute capacity into a mining botnet without ever needing to steal a credential. This maps directly to the RBAC authorisation flow in Figure 3 — a cluster with anonymous access effectively has no authorisation step at all — and to the admission-control guidance, since restricting which registries and images may be deployed would have blocked the attack outright.

Kubernetes security is best understood as a layered, ongoing discipline rather than a single setting. Securing the cluster requires coordinated controls at every level of the 4C's (Cloud, Cluster, Container, Code) model: hardened cloud/IAM foundations, least-privilege RBAC, default-deny network segmentation, restricted pod security standards, encrypted and externally managed secrets, policy-enforcing admission controllers, continuous audit logging, and a secured software supply chain feeding trusted images into the cluster.

Organisations that treat these controls as a connected system — rather than isolated checkboxes — are best positioned to operate Kubernetes safely at production scale. END

References

- Kubernetes Documentation — 'Kubernetes Components', *https://kubernetes.io/docs/concepts/overview/components/*
- Kubernetes Documentation — 'Overview of Cloud Native Security', *https://kubernetes.io/docs/concepts/security/overview/*
- 'Kubernetes Documentation — 'Using RBAC Authorization', *https://kubernetes.io/docs/reference/access-control/rbac/*
- Kubernetes Documentation — 'Pod Security Standards', *https://kubernetes.io/docs/concepts/security/pod-security-standards/*
- Wiz Academy — 'Kubernetes RBAC Best Practices — From Basic to Advanced', *https://www.wiz.io/academy/container-security/kubernetes-rbac-best-practices*
- Sysdig — 'Kubernetes Security 101: 10 Best Practices and Fundamentals', *https://www.sysdig.com/learn-cloud-native/kubernetes-security-101*
- Networkers Home — 'Kubernetes Security — RBAC, Policies, Admission', *https://www.networkershome.com/fundamentals/cloud-security/kubernetes-security-rbac-policies-admission/*
- Atmosly — 'Kubernetes Security Checklist: 50 Best Practices (2025)', *https://atmosly.com/blog/kubernetes-security-checklist-50-best-practices-2025*
- KodeKloud — 'Kubernetes Best Practices 2025: Optimize, Secure, and Scale,' *https://kodekloud.com/blog/kubernetes-best-practices-2025/*
- Emircan Agac (Medium) — 'Kubernetes Security Best Practices Part 1: RBAC, Secrets & Pod Security,' *https://medium.com/@emircanagac/kubernetes-security-best-practices-part-1-1d7ef1566f25*
- Greenberg, Andy — 'How Hacked Water Heaters Could Trigger Mass Blackouts'/ Tesla cryptojacking coverage, Wired, *https://www.wired.com/story/cryptojacking-tesla-amazon-cloud/*
- Unit 42, Palo Alto Networks — 'Azurescape: Cross-Account Container Takeover in Azure Container Instances', *https://unit42.paloaltonetworks.com/azure-container-instances/*
- Vectra AI — 'Kubernetes Security Risks and How to Detect Them' (Dero cryptojacking campaign, 2024), *https://www.vectra.ai/topics/kubernetes-security*
- ropnop Blog — 'Attacking Default Installs of Helm on Kubernetes', *https://blog.ropnop.com/attacking-default-installs-of-helm-on-kubernetes/*

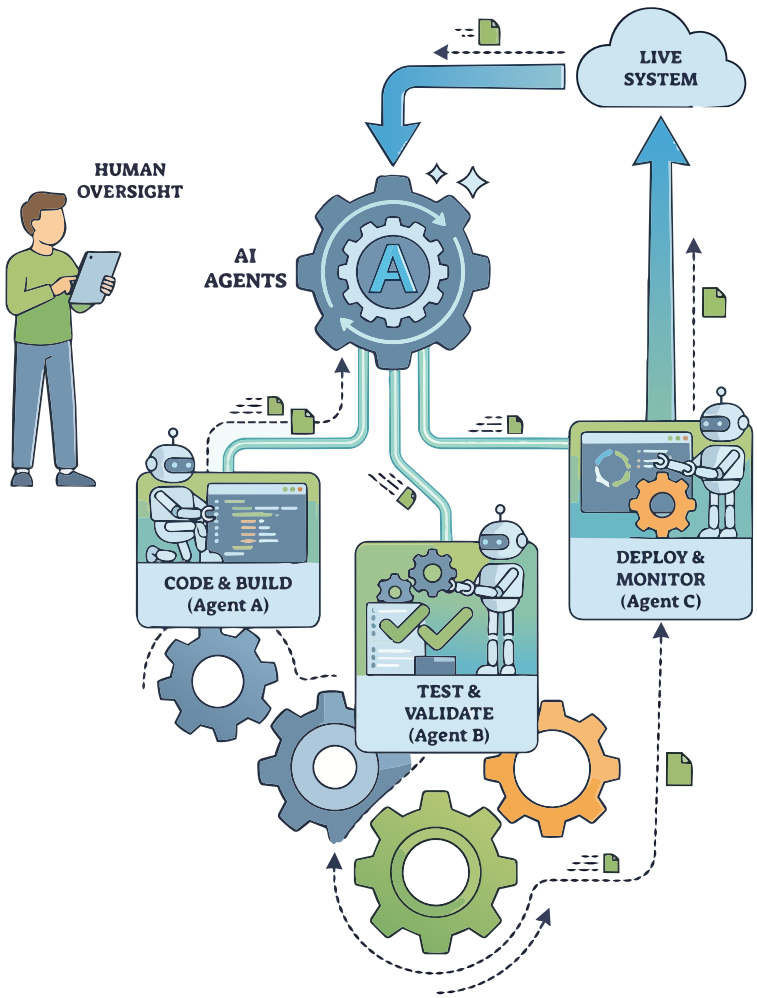
By: Dr Geetali Saha and Kush Patel

Dr Geetali Saha is a faculty member at the Department of Electronics & Communication, GCET, CVM University, Anand, Gujarat.

Kush Patel is a member of the faculty in the Department of Information Technology, GCET, a PhD Scholar at CVM University, Anand, Gujarat. He is an expert in security applications.

Agentic DevOps in Action

The complexity of modern-day digital infrastructure can no longer be handled by traditional DevOps. Agentic DevOps is the way forward.



DevOps fundamentally changed the way modern software is put together, tested, shipped, and cared for. It helped to replace separate teams, manual handovers, and long release cycles with automated pipelines, Infrastructure as Code (IaC), cloud platforms, containers, and continuous monitoring. But things have become a lot more complicated today with the use of multiple clouds, containers, microservices, APIs, databases, and all that distributed infrastructure. And as this complexity grows, automating the same predefined tasks isn't enough. That's where agentic DevOps is making its mark.

Agentic DevOps basically blends DevOps principles with generative AI and autonomous AI agents. Instead of using AI only to produce code or answer questions, these agentic setups can watch what's happening, interpret what the issue is, plan out a sequence of moves, work with the existing technical tools, and then check whether the outcome is good. In other words, the focus is shifting from systems that just follow orders, towards systems that can grasp the end goal and figure out the right actions to take.

From DevOps to agentic DevOps

The road to agentic DevOps has been built gradually. Traditional DevOps brought automation into how software gets shipped and how operations run. Then continuous integration and continuous deployment (CI/CD) helped teams build, test, and deploy apps automatically whenever changes showed up. After that, Infrastructure as Code made it easier to manage infrastructure using version-controlled configuration, instead of relying on more manual ways. Monitoring and observability then gave teams real-time visibility into how applications and infrastructure were doing, not just what they hoped was happening.

So yes, these improvements cut down a lot of the grunt work. But the core workflows stayed predefined. A pipeline could run through a sequence of steps very efficiently, but usually couldn't figure out why a strange failure happened, or it couldn't decide what to do when something entirely new showed up.

Agentic DevOps improves on this considerably. An AI agent can take in the available information, reason through a situation, interact

Table 1: The evolution of software operations

Stage	Primary focus	Role of AI	Decision making
Traditional IT	Manual operations	None	Human
DevOps	Automation and CI/CD	Limited	Predefined rules
AIOps	Intelligent monitoring	Analytics and ML	AI-assisted
Agentic DevOps	Autonomous operations	Reasoning + Agents	Agent + Human
Autonomous platform	Self-managing systems	Multi-agent intelligence	Controlled autonomy

with different systems, and carry out actions aligned to a specific objective. That creates a more dynamic way of operating, where the whole system is not locked into one fixed sequence of instructions.

Here's an example of a traditional pipeline vs an agentic workflow:

Traditional CI/CD approach:

```
deploy:
  steps:
    - build
    - test
    - deploy

rollback:
  if: deployment_failed
  action: rollback
```

Agentic approach:

```
def agentic_devops(event):

    context = observe(
        logs=True,
        metrics=True,
        deployment_history=True
    )

    problem = analyze(context)

    plan = create_plan(
        objective="restore_application_
health",
        problem=problem
    )

    if requires_approval(plan):
        request_human_approval(plan)

    result = execute(plan)

    verify(result)
```

Here's another smaller code example:

```
@tool
def check_kubernetes_pods(namespace):
    """Check the health of pods in a
Kubernetes namespace."""
    return kubectl(
        f"get pods -n {namespace}"
    )

agent = create_agent(
    tools=[check_kubernetes_pods],
    goal="Investigate unhealthy
application pods"
)
```

This demonstrates a key idea of Agentic DevOps: AI + reasoning + tools.

Automation and agentic intelligence

Automation and agentic intelligence seem similar but are not the same. Automation tends to follow instructions that already exist — the path is pretty much written ahead of time. An agent, on the other hand, aims at some goal and then figures out what steps are needed to get there.

Think about a simple deployment rule: 'If the deployment fails, automatically roll back to the previous version'. That's meant for automation because the trigger and the reaction are known ahead of time, and there are no surprises.

Picture a case where the deployment technically 'works', yet application performance starts to slip right after. CPU usage climbs, database connections begin failing, and users get hit with high latency. There may not be any single predefined rule to cope with this scenario.

So you move to a different operational loop: observe, understand, plan, act, and verify. First, the system observes what's happening, then it builds an understanding of the situation, makes a plan, takes action, and finally checks whether the action actually delivered the desired outcome.

Table 2: Traditional automation vs agentic DevOps

Traditional automation	Agentic DevOps
Follows predefined rules	Works towards defined goals
Executes fixed workflows	Dynamically plans actions
Handles known scenarios	Can investigate unfamiliar scenarios
Limited contextual understanding	Context-aware reasoning
Requires predefined responses	Can select from multiple actions
Usually stops after execution	Verifies the outcome
Human handles exceptions	Agent can handle approved exceptions

This is the real value of agentic DevOps — not just in doing tasks automatically, but in turning automation into something context-aware and goal driven.

The DevOps agent

At the centre of agentic DevOps sits the DevOps agent. You can think of it as an AI driven software component that talks to the tools and systems used — right from the idea to the release.

But a useful agent is more than just a big language model. It needs access to relevant information, plus some well-controlled tools. For example, it may read the source code sitting in a Git repository, look over CI/CD pipeline outcomes, query Kubernetes, or inspect cloud resources. It could also sift through monitoring signals, and work with an incident-management platform when things go sideways.

Table 3: The different levels of agent capability

Level	Agent capability	Human involvement
Level 1	Provides recommendations	Full
Level 2	Executes low-risk tasks	High
Level 3	Handles predefined incidents	Moderate
Level 4	Plans and executes complex workflows	Selective
Level 5	Operates autonomously within policies	Strategic oversight

However, the agent must have context. Without it, an AI model may give answers that are technically correct, but operationally not a great fit. A production environment isn't generic. It has its own architecture, its own dependencies, security rules, deployment steps, and business requirements. The agent should understand these limits first, before it starts processing anything.

All of this creates a kind of intelligent layer that stitches different pieces of the DevOps ecosystem into a single flow.

From intelligent assistance to autonomous operations

One of the most interesting aspects of agentic DevOps is the gradual shift towards autonomy.

Right now, a lot of developers and DevOps engineers already lean on AI assistants to knock out scripts, describe



what an error message really means, draft infrastructure configurations, or come up with solutions that look plausible. Of course, the engineer still has the final say — reviewing the output and executing it.

In the next stage, AI will be able to carry out several connected tasks, with only slight oversight. For instance, an agent may be given the goal of investigating a deployment that failed. Instead of just explaining the error, it could gather the necessary details, reason through the failure, surface a few likely causes, and then put together a remediation plan.

Human-in-the-Loop

The idea of autonomous systems often brings concerns about safety and control, and honestly, that matters more than people think. In DevOps, these worries become especially important, because a wrong move can hit production systems, mess with customer data, weaken security, and even disrupt business operations.

An agent may be allowed to restart a development service on its own, but a change in the production infrastructure may still need a human sign-off. Likewise, an agent can automatically investigate a security alert, but it would pause for authorisation before it touches a firewall rule, or a production access policy, depending on the scope.

So you end up with this Human-in-the-Loop model, where people and agents collaborate. The AI deals with repetitive investigation and operational work, while engineers make the high impact calls.

The goal isn't to remove humans from DevOps at all. It's more to shift human attention towards the areas where judgment counts the most.

The rise of the multi-agent DevOps team

Agentic DevOps gets even more powerful when several specialised agents team up. A release agent obsesses over deployments and CI/CD, making sure version changes don't slip. An infrastructure agent handles the cloud side and Kubernetes objects, all the way from namespaces to scaling decisions. The observability agent watches logs, metrics, and traces, looking for weird patterns. A security agent goes after vulnerabilities and policy violations that look small but aren't. And an SRE agent zeroes in on reliability plus incident response when things start acting up.

Picture a production incident where application latency suddenly spikes. The observability agent catches the anomaly first, and then the deployment agent checks whether a recent release was involved. The infrastructure agent digs into resource utilisation, comparing current load with the baseline. Meanwhile, the security agent looks for suspicious activity, connections, or any strange authorisation behaviour. After that a coordinating agent bundles everything together, synthesizes the signals, and recommends what to do next.

This setup pushes DevOps closer to the idea of an AI-powered engineering team, not just automation for automation's sake.

What agentic DevOps means for engineers

The emergence of agentic DevOps will also nudge the role of DevOps engineers. A lot of the repetitive operations work that used to pile up can increasingly be assisted or even handled by intelligent systems. Engineers may find themselves spending less time manually digging through logs, running routine commands, crafting repeat configs, and following common alert storms.

At the same time though, fresh duties will show up. Engineers will need to grasp how to design agentic workflows, equip the agents with the right tools, set up permissions properly, assess what the agents decide, keep an eye on how they behave, and build solid safety mechanisms that hold up when things get weird.

The road to autonomous DevOps

Agentic DevOps should not be treated like some sudden, full stop replacement for what we already do in DevOps. It is more like an evolution stacked on top of existing practices, and not a clean swap. CI/CD, Infrastructure as Code, cloud platforms, Kubernetes, observability, GitOps and security all stay non-negotiable. What AI agents add is a new intelligent layer that can interact with these pieces in a more proactive way.

You can think of the path as going from manual operations to automation, then to automation with smarter help, and finally towards something closer to controlled autonomy.

The end game is a software delivery and operations environment where systems can notice problems, grasp the surrounding context, pick a fitting response, perform only the actions that are approved, and then confirm the outcome.

Agentic DevOps is not only about fitting AI onto a pipeline. It is a wider shift in how software platforms get designed, and how they run day to day. END



By: D. Banerjee and Sourav K.

D. Banerjee is an IT professional with 15+ years of experience in Tier-1 IT companies, specialising in delivering scalable enterprise solutions and leading complex technology initiatives.

Sourav K. is an IT professional with around 5 years of experience in Tier-1 IT companies, skilled in modern technologies and focused on building efficient, data-driven solutions.

An EFYGROUP EVENT

Discover Gujarat's Electronics Power *& Profit From It!*

If You Are Into Electronics, You Have To Be There.

EFY Expo @Gujarat provides the only platform to discover and tap into opportunities being generated by Gujarat's electronics industry, which is growing exponentially. Connect with an entirely new set of customers, prospects, channel partners, principles, investors, and even suppliers at the only electronics expo of Gujarat.







Electronics ForYou expo
GUJARAT 2026

3-5 DEC 2026
HELIPAD EXHIBITION CENTER, GANDHINAGAR

Enabling New Product Development to Manufacturing of Innovations in Electronics

SPECIAL OFFER AND 100% MONEY BACK GUARANTEE FOR EXHIBITORS

Contact:
growmybiz@efy.in OR 98111-55335

SCAN THIS

TO REGISTER, visit:
gujarat.efyexpo.com/register

R N I No. DELENG/2012/49440, Mailed on 27/28th of Advance month
Published on 27th September 2026

Posted at Magazine Post RMS Delhi 110006

An EFY GROUP EVENT

BITS Pilani DIGITAL
Excellence made yours

Presents

OPEN SOURCE INDIA

7-8 October 2026

NIMHANS CONVENTION CENTER, BENGALURU

23rd edition

Asia's #1 Conference on Open Source

ACCELERATING DEVELOPMENT AND DEPLOYMENT OF OPEN SOURCE

tracks

- **Open Source For Everyone** (India contributes)
- **AI / ML** (Opening up the Tech)
- **IT Infra** (Security, Storage & Cloud)
- **Developers' Meet** (AI and Open Source)
- **DevOps and DevSecOps**
- **Data Management**





For more details, call on
+91 98111-55335
or write to
info@opensourceindia.in

Know More

www.OpenSourceIndia.in